Minor exclusion characterized by bounded twin-width of rooted spanning forest orders

Nikolas Mählmann, Patrice Ossona de Mendez, Nicole Schirrmacher, Sebastian Siebertz, Giannos Stamoulis, Dimitrios M. Thilikos, Alexandre Vigny

---

Editorial note. Complete original mathematical body466–4086, including every supporting structural proof, all native figures and appendix, is retained; only original standalone bibliography command4043 is supplied by the assembly bibliography backend using the unchanged ref.bib. Native author preamble and elsarticle class retained.

Editorial note. Authors retain responsibility for mathematics. No proof certification, author review, added assumptions, repaired mathematics or reconstructed proofs.

Editorial note. Three principal objects are distinct; derived model-checking bounds, algorithmic variants and application corollaries are retained as context rather than split into extra counts. Efficient minimal-minor-host construction hypothesis in the hardness theorem is preserved exactly.

Editorial note. Rendering dependency: original elsarticle class and package distribution/source/license files supplied as local assets from official TeX Live archive; no class or mathematical source modification, no global installation. Failed r001 preserved.

Editorial note. Rendering source0 preserves line positions and delegates original plainurl bibliographystyle at raw162 to the assembly backend once, preventing duplicate bibstyle aux commands. Official original urlbst style distribution supplied locally; unchanged original ref.bib and raw main.tex retained.

Editorial note. Actual ec-lmr10 missing glyph: two Unicode c-acute instances in original prose/theorem attribution at main.tex1636/1639 rendered with equivalent TeX accent in source0. Original mathematical text and author attribution unchanged; raw main.tex and ref.bib retained.

Allowed standard prerequisites (editorial).

- Graph minors, weak sparsity, twin-width, first-order transductions and monadic dependence. Complete native definitions and all source-local supporting structural constructions/translations are included. Bibliographic pointers identify the author-used standard external meta-theorems and previous characterizations; no independent proof reconstruction is supplied.. Retains precise source hypotheses, graph/forest/relational-signature conventions and effective encoding requirements.

1. Introduction

Bridging algorithmic graph theory and finite model theory, a central line of research seeks efficient graph algorithms on well-behaved classes, together with logical and structural characterizations of the tractable problems and the underlying classes. The resulting statements are commonly expressed as *algorithmic meta-theorems* of the following form.

Every problem expressible in ⟨**logic**⟩ *can be solved in time* ⟨**runtime**⟩ *on every structure satisfying* ⟨**tameness condition**⟩.

Two prototypical examples of this pattern are the following.

- Model checking monadic second-order logic (MSO) can be done in time $f(|\varphi|,t)\cdot|G|$ on graphs G of tree-width at most t. This is known as Courcelle's theorem. [2, 25, 26]
- Model checking first-order logic (FO) can be done in time $f_{\mathscr{C}}(|\varphi|,\varepsilon)\cdot|G|^{1+\varepsilon}$ on all graphs G from every fixed nowhere dense graph class $\mathscr{C}$. [56]

In both of these examples, the involved structural tameness condition coincides with the notion of *monadic dependence*, a key notion from model theory that we will define later. For every graph class $\mathscr{C}$ that is monotone (i.e., closed under taking subgraphs), we have that $\mathscr{C}$ is monadically dependent ...

- ... for MSO if and only if $\mathscr{C}$ has bounded tree-width; [57]
- ... for FO if and only if $\mathscr{C}$ is nowhere dense. [1]

In this paper, we investigate the connection between efficient model checking and monadic dependence for minor-free graph classes. As minor-free graph classes are strictly sandwiched between bounded tree-width and nowhere dense classes, this calls for a logic that is less expressive than MSO and more expressive than FO.

We introduce an extension of first-order logic, called *sub-connectivity logic* and denoted by FO+subconn. Formulas are evaluated on a graph G with a specified set $Z\subseteq E(G)$ of *admissible edges*. In addition to the usual first-order atoms, the logic has an atom $\mathsf{subconn}(x,y;\bar{z})$ asserting that x and y are not among the vertices in $\bar{z}$ and are connected by a path using only edges of Z and avoiding those vertices.

We translate FO+subconn on (G,Z) into first-order logic on an expansion by a forest order induced by a rooted depth-first spanning forest of the graph $(V(G),Z)$. Combining this translation with our structural results, we show that there are a computable function f and an absolute constant c such that an FO+subconn-sentence φ can be model-checked on (G,Z) in time

$$f(|\varphi|,h(G))\cdot|G|^c,$$

where $h(G)$ is the *Hadwiger number* of G, that is, the largest order of a clique minor of G. Thus, FO+subconn model checking is fixed-parameter tractable on every graph class excluding a fixed minor. The algorithm extends to binary relational structures $\mathbf{M}$, with running time $f(|\varphi|,h(\mathrm{Gaif}(\mathbf{M})))\cdot\|\mathbf{M}\|^c$, where $\mathrm{Gaif}(\mathbf{M})$ is the Gaifman graph of $\mathbf{M}$.

For a graph class $\mathscr{C}$, let $\mathscr{C}_Z := \{(G,Z) : G\in\mathscr{C},\ Z\subseteq E(G)\}$. We call $\mathscr{C}$ *weakly sparse* if it excludes some complete bipartite graph as a subgraph. For every weakly sparse class $\mathscr{C}$, we prove that $\mathscr{C}_Z$ is monadically dependent for FO+subconn if and only if $\mathscr{C}$ excludes a fixed minor.

1.1. Context

Model checking. For a logic $\mathcal{L}$ and a class $\mathscr{C}$ of finite structures, the associated computational problem in the broader program of algorithmic meta-theorems is the *model checking* problem: given $\mathbf{M} \in \mathscr{C}$ and a sentence $\varphi \in \mathcal{L}$, decide whether $\mathbf{M} \models \varphi$. Reformulated from above, algorithmic meta-theorems assert that model checking for a logic is tractable[1] on classes satisfying suitable structural tameness conditions, thereby linking logical definability, (graph) structure, and efficient computation in a uniform way. Classical milestones of this program include tractable model checking for MSO on bounded tree-width graphs [2, 25, 26] and for FO on sparse graph classes such as bounded degree, excluded minor, bounded expansion, and nowhere dense classes [82, 44, 37, 56]. More recently, *twin-width* has led to model-checking algorithms that also apply to dense graphs [13]. In particular, and particularly important for this paper, FO model checking is fixed-parameter tractable on classes of linearly ordered binary structures of bounded twin-width [10]. Beyond unifying large families of tractable problems, such results repeatedly lead to new structural viewpoints and algorithmic techniques. It has therefore become one of the central problems in algorithmic model theory to determine the exact tractability frontier of efficient model checking for various logics.

The limits of tractability. The limits of tractability for model checking are closely related to *dependence*, one of the main dividing lines in classical model theory. A formula $\varphi(\bar{x};\bar{y})$ is *independent on a class* $\mathscr{C}$ if, for every $n \in \mathbb{N}$, there are a structure $\mathbf{M} \in \mathscr{C}$, tuples $\bar{a}_i \in M^{|\bar{x}|}$ for $i \in [n]$, and tuples $\bar{b}_J \in M^{|\bar{y}|}$ for $J \subseteq [n]$ such that

$$\mathbf{M} \models \varphi(\bar{a}_i;\bar{b}_J) \quad \iff \quad i \in J.$$

Intuitively, this means there are arbitrarily large sets, from which φ can pick out all the subsets. A formula is *dependent on* $\mathscr{C}$ if it is not independent on $\mathscr{C}$, and a class is *dependent* for a logic if every formula of that logic is dependent on it. A class is *monadically dependent* if every expansion of the class by finitely many arbitrary unary predicates is dependent.

For *hereditary classes* (i.e., classes closed under taking induced substructures), the notion of monadic dependence for FO is known to imply fixed-parameter tractable FO model checking for:

- weakly sparse classes of graphs [37, 56],
- edge-stable[2] classes of graphs [32, 31],
- classes of linearly ordered graphs [10],
- tournaments [49].

Moreover, model checking is known to be AW[⋆]-hard on every hereditary graph class that is not monadically dependent [34]. Strikingly, in many cases, monadic dependence corresponds to a graph-theoretic notion:

- nowhere-denseness for weakly sparse classes [1],
- flip-flatness for edge-stable classes [33],
- bounded twin-width for linearly ordered graphs and tournaments [10, 49].

The connection between monadic dependence of linearly ordered structures, twin-width, and model checking will be central to our results for graphs equipped with forest orders. We refer to the recent survey [72] for a broader overview.

[1] When we speak of tractable model checking on a class $\mathscr{C}$, we mean fixed-parameter tractability with respect to the formula size, that is, a running time of the form $f(|\varphi|) \cdot \|\mathbf{M}\|^c$ for some computable function f and an absolute constant c, on every $\mathbf{M} \in \mathscr{C}$ and formula φ. For the general sparsity and stability results recalled below, we assume effective bounds on the corresponding class properties, as required for computable parameter dependence.

[2] A graph class is *edge-stable* if it excludes a half-graph as a semi-induced subgraph.

First-order logic with connectivity and the quest for a logic for classes with excluded minors. A basic limitation of first-order logic is that it cannot express connectivity. Several natural extensions of first-order logic were introduced to address this gap. A recent example is *separator logic*, also denoted by $\mathsf{FO}{+}\mathsf{conn}$, which extends FO on graphs by allowing the use of the atomic predicate $\mathsf{conn}(x, y; \bar{z})$, which evaluates to true if there is a path in the graph from x to y that avoids the vertices of $\bar{z}$ [5, 81]. The following theorem summarizes the resulting frontier on monotone classes.

Theorem 1.1. *Let $\mathscr{C}$ be a monotone class of graphs. Then the following are equivalent:*
(1) $\mathscr{C}$ excludes some graph as a topological minor;
(2) $\mathscr{C}$ is monadically dependent (for $\mathsf{FO}{+}\mathsf{conn}$);
(3) $\mathscr{C}$ is dependent (for $\mathsf{FO}{+}\mathsf{conn}$).

Moreover, any of these conditions imply that $\mathsf{FO}{+}\mathsf{conn}$ model checking is FPT on $\mathscr{C}$, while if $\mathscr{C}$ is independent and admits efficient encoding[a] of graphs, then model checking on $\mathscr{C}$ is $\mathsf{AW}[*]$-hard.

[a] For logics more expressive than first-order logic, independence alone need not rule out fixed-parameter tractable model checking, even on hereditary graph classes. For example, sufficiently long subdivisions can make the encoded graph small relative to the input, leaving enough time to evaluate a sentence by brute force on the encoded instance. Such phenomena occur for monadic second-order logic [46, 61] and separator logic [73].
To obtain hardness, we therefore impose an efficiency condition on the encoding. An *efficient encoding of graphs* consists of a polynomial-time construction that maps every finite graph to a monadic expansion of a structure in the class, from which a fixed simple interpretation in the logic recovers the input graph. The interpretation then gives an $\mathsf{AW}[\star]$-hardness reduction for model checking on these monadic expansions. Our hardness theorem uses a specific efficient minor-encoding hypothesis, stated below.

The algorithmic statements of the theorem were proved in [73], and we give a brief sketch of the dependence characterizations in Appendix A. Further works on extensions of FO for topological minor-free classes include [51, 79, 80, 7, 78]

The next natural graph-theoretical dividing line corresponds to classes excluding a minor, hence the following question.

Problem 1. *Is there a natural logic $\mathcal{L}$ such that a monotone or weakly sparse class of graphs is monadically dependent for $\mathcal{L}$ if and only if it excludes a minor?*

Understanding the structure of monadically dependent classes. A complementary approach to monadically dependent classes studies *tree-ordered weakly sparse* (TOWS) representations of such graph classes. Many successful structural representations come with an underlying tree order. Exposing this order as a part of the structure makes global structure accessible to first-order logic. This viewpoint is central in the study of classes of many tameness notions for graph classes [10, 13, 14, 19, 21, 23, 27, 35, 45, 47, 48], and has very recently led to new characterizations of minor exclusion through tree-ordered weakly sparse graphs [20], which is a first step toward a solution of Problem 1 and the departure point of the present paper. Let us give the background.

Let G be a graph. A *spanning forest* of G is an acyclic subgraph $F \subseteq G$ with $V(F) = V(G)$. Following [58], we call F *full* if F and G induce the same partition of $V(G)$ into connected components. A *rooted spanning forest* has one distinguished vertex, called a *root*, in each component. It defines a forest order $\preceq_F$ on $V(G)$ by $x \preceq_F y$ if x lies on the unique path from y to the root of

its component. For a graph class $\mathscr{C}$, let

$$\mathscr{C}^{\Upsilon} := \{(G, \preceq_F) : G \in \mathscr{C},\ F \text{ is a full rooted spanning forest of } G\},$$
$$\mathscr{C}^{F} := \{(G, \preceq_F) : G \in \mathscr{C},\ F \text{ is a rooted spanning forest of } G\}.$$

These are the classes of graphs in $\mathscr{C}$ expanded by full and arbitrary rooted spanning forest orders, respectively. Two results established a logical characterization of minor exclusion in this setting.

Theorem 1.2 ([20, Theorem 1.4]). *Let $\mathscr{C}$ be a weakly sparse class of graphs. Then the following are equivalent:*
(1) $\mathscr{C}$ excludes some graph as a minor;
(2) $\mathscr{C}^{\Upsilon}$ is monadically dependent (for FO*);*
(3) $\mathscr{C}^{F}$ is monadically dependent (for FO*).*

The equivalence of the first two conditions is [20, Theorem 1.4]. The equivalence of (2) and (3) is an easy observation, see Remark 3.1 in the preliminaries.

Theorem 1.3 ([12, Theorem 2]). *Let $\mathscr{C}$ be a monotone class of graphs. Then the following are equivalent:*
(1) $\mathscr{C}$ excludes some graph as a minor;
(2) $\mathscr{C}$ has bounded spanning twin-width*, that is: there exists a constant t such that every graph $G \in \mathscr{C}$ has a full rooted spanning forest F such that $\mathrm{tww}(G, \preceq_F) \leqslant t$.*

1.2. Results

Structural and algorithmic results. If $\mathscr{C}$ excludes a fixed minor, then Theorem 1.2 shows that $\mathscr{C}^{F}$ is monadically dependent. We strengthen this conclusion to bounded twin-width.

Theorem 5.1 (see page 27). *Let $\mathscr{C}$ be a weakly sparse class of graphs. Then the following are equivalent:*
1. *$\mathscr{C}$ excludes some graph as a minor;*
2. *$\mathscr{C}^{\Upsilon}$ has bounded twin-width;*
3. *$\mathscr{C}^{F}$ has bounded twin-width.*

Moreover, if these conditions hold, an expansion of $\mathscr{C}^{F}$ by a linear order with bounded twin-width can be computed in polynomial time.

This is significant as, for linearly ordered binary structures, bounded twin-width *exactly* captures monadic dependence and governs the tractability of first-order model checking. While the above theorem strengthens the (1) ⇒ (2) direction of Theorem 1.3, we also obtain a strengthening of the (2) ⇒ (1) direction on monotone classes (Theorem 5.16).

The following is an algorithmic version of Theorem 5.1. The exclusion of a minor is restated using the notion of Hadwiger number $h(G)$ of a graph G.

Theorem 5.2 (see page 27). *There exist computable functions ξ, f, a constant c and an algorithm that, for a given graph G and a rooted spanning forest F of G, computes in time $f(h(G)) \cdot |G|^c$ a linear order $<$ on G, such that $(G, \preceq_F, <)$ has twin-width at most $\xi(h(G))$.*

This theorem extends to binary relational structures (Theorem 5.4). As FO model checking is known to be fixed-parameter tractable on bounded twin-width classes of linearly ordered binary structures [10], it also implies the following model checking result.

Corollary 5.5 (see page 29)**.** *Fix a finite relational signature σ of arity at most two, and let $\preceq$ be a fresh binary relation symbol. There are a computable function f and an absolute constant c such that the following holds. Given a finite σ-structure $\mathbf{M}$, a rooted spanning forest F of* $\mathrm{Gaif}(\mathbf{M})$*, and an* FO*-sentence φ over $\sigma \cup \{\preceq\}$, one can decide whether*

$$(\mathbf{M}, \preceq_F) \models \varphi$$

in time

$$f\big(|\varphi|, h(\mathrm{Gaif}(\mathbf{M}))\big) \cdot \|\mathbf{M}\|^c.$$

In the above, for a structure $\mathbf{M}$ with universe M, its *Gaifman graph* $\mathrm{Gaif}(\mathbf{M})$ has vertex set M, with distinct vertices adjacent if they occur together in a tuple of a relation of $\mathbf{M}$.

At first glance, Corollary 5.5 may seem somewhat technical, since it concerns first-order logic over a more general signature obtained by enriching graphs with an additional structure, namely a forest order. We next show that this is in fact a very natural viewpoint: first-order logic over this enriched signature admits an exact characterization in terms of a natural extension of first-order logic on graphs by suitable connectivity predicates.

Solving Problem 1: A logic for classes that exclude a minor. We introduce *sub-connectivity logic*, denoted by FO+subconn. We now directly work in the setting of binary structures. Let σ be a finite binary relational signature containing a distinguished binary relation symbol Z. Formulas of *sub-connectivity logic*, denoted by $\mathsf{FO{+}subconn}[\sigma]$, are evaluated on σ-structures $\mathbf{M}$ in which $Z(\mathbf{M})$ is symmetric and irreflexive. We identify the Z-reduct $\mathbf{M}\restriction_Z$ with the simple undirected graph on M having edge relation $Z(\mathbf{M})$. In addition to the usual first-order atoms over σ, the logic has atoms

$$\mathsf{subconn}(x, y; \bar{z}),$$

where $\mathbf{M} \models \mathsf{subconn}(x, y; \bar{z})$ if and only if x and y lie outside the set of vertices occurring in $\bar{z}$ and belong to the same connected component of $\mathbf{M}\restriction_Z - \bar{z}$.

Hence, when $\sigma = \{E, Z\}$ and $\mathbf{M}$ is a σ-structure whose E-reduct is a graph and which satisfies $Z(\mathbf{M}) = E(\mathbf{M})$, the atom subconn is exactly the connectivity atom of separator logic FO+conn. However, allowing the interpretation of Z to specify a second independent edge layer makes it possible to reason about connectivity in, for example, a spanning forest or the branch sets of a minor model. This explains why the frontier moves from topological-minor exclusion for FO+conn to *minor exclusion* for FO+subconn.

Our first main result about FO+subconn is an exact characterization of its expressive power by relating FO+subconn to the spanning forest orders discussed above.

Theorem 6.2 (see page 45)**.**

FO+subconn *is equivalent to* FO *with DFS spanning forest orders in the following sense.*

FO with DFS spanning forest orders can simulate FO+subconn:

Let σ be a finite binary relational signature containing a distinguished binary relation symbol Z and not containing $\preceq$.
For every formula $\varphi(\bar{x})$ of $\mathsf{FO{+}subconn}[\sigma]$, there is an effectively computable formula $\psi(\bar{x}) \in \mathsf{FO}[\sigma \cup \{\preceq\}]$ such that the following holds. Let $\mathbf{M}$ be a σ-structure in which $Z(\mathbf{M})$ is symmetric and irreflexive, and let F be any rooted depth-first search forest of $\mathbf{M}\restriction_Z$. Then, for every tuple $\bar{v} \in M^{|\bar{x}|}$,

$$\mathbf{M} \models \varphi(\bar{v}) \quad \iff \quad (\mathbf{M}, \preceq_F) \models \psi(\bar{v}).$$

FO+subconn can simulate FO with spanning forest orders:

Let τ be a finite binary relational signature containing neither Z, R, nor $\preceq$.
For every formula $\chi(\bar{x}) \in \mathsf{FO}[\tau \cup \{\preceq\}]$, there is an effectively computable formula $\theta(\bar{x})$ of $\mathsf{FO{+}subconn}[\tau \cup \{Z, R\}]$, where Z and R are fresh binary and unary relation symbols, respectively, with the following property. Let $\mathbf{M}$ be a τ-structure, let $\preceq$ be a rooted forest order on M, let $Z_\preceq$ be the set of undirected cover edges of $\preceq$, and let $R_\preceq$ mark exactly the roots of $\preceq$. Let $\mathbf{M}_\preceq$ be the expansion of $\mathbf{M}$ in which $Z(\mathbf{M}_\preceq) = Z_\preceq$ and $R(\mathbf{M}_\preceq) = R_\preceq$. Then, for every $\bar{v} \in M^{|\bar{x}|}$,

$$(\mathbf{M}, \preceq) \models \chi(\bar{v}) \quad \iff \quad \mathbf{M}_\preceq \models \theta(\bar{v}).$$

Combining the forward translation with the structural theorem yields monadic dependence whenever the Gaifman graphs exclude a fixed minor. In the graph special case, the converse holds on every weakly sparse class, answering Problem 1.

Theorem 6.4 (see page 46)**.** *Let $\mathscr{K}$ be a class of finite binary relational structures over a fixed signature containing the distinguished binary relation symbol Z, such that $Z(\mathbf{M})$ is symmetric and irreflexive for every $\mathbf{M} \in \mathscr{K}$. If $\mathrm{Gaif}(\mathscr{K})$ excludes a fixed minor, then $\mathscr{K}$ is monadically dependent for* FO+subconn*.*
Moreover, if $\mathscr{C}$ is a weakly sparse class of graphs, then

$$\mathscr{C}_Z \text{ is monadically dependent for } \mathsf{FO{+}subconn} \quad \iff \quad \mathscr{C} \text{ excludes a fixed minor.}$$

Combining the same translation with the algorithmic version of our structural theorem gives the promised meta-theorem for arbitrary binary structures.

Theorem 6.3 (see page 46)**.** *There exist a computable function f and an absolute constant c such that the following holds. Given a finite binary relational structure $\mathbf{M}$ over a finite signature containing the distinguished binary relation symbol Z, where $Z(\mathbf{M})$ is symmetric and irreflexive, and an* FO+subconn*-sentence φ over its signature, one can decide whether $\mathbf{M} \models \varphi$ in time*

$$f\big(|\varphi|, h(\mathrm{Gaif}(\mathbf{M}))\big) \cdot \|\mathbf{M}\|^c,$$

where $\|\mathbf{M}\|$ denotes the size of the input encoding. In particular, FO+subconn *model checking is fixed-parameter tractable, parameterized by formula length, on every class of binary structures whose Gaifman graphs exclude a fixed minor.*

We also prove a hardness counterpart to Theorem 6.3. Let $\mathscr{C}$ be monotone and exclude no fixed minor. Assume that, given a graph H, one can compute in polynomial time a graph $F \in \mathscr{C}$ that contains H as a minor and is minimal under taking subgraphs with this property. Then FO+subconn model checking on $\mathscr{C}_Z$ is AW[⋆]-hard, parameterized by the sentence size. Thus, under this encoding hypothesis and the assumption FPT $\neq$ AW[⋆], minor exclusion is the tractability frontier in this monotone setting. See [42, Section 8.6] for the complexity class.

Theorem 6.5 (see page 47)**.** *Assume that $\mathscr{C}$ is monotone, does not exclude any fixed minor, and admits efficient encoding, that is, there is a polynomial-time algorithm that, given a graph H, computes a graph $F \in \mathscr{C}$ such that $H \leqslant_m F$ and $H \not\leqslant_m F'$ for every proper subgraph F' of F. Then deciding, given a graph $F \in \mathscr{C}$, a subset $Z_F \subseteq E(F)$, and an* FO+subconn*-sentence φ, whether $(F, Z_F) \models \varphi$ is* AW[⋆]*-hard, parameterized by $|\varphi|$.*

An algorithmic application. As an algorithmic application, we consider combinatorial reconfiguration and solution discovery for vertex-deletion problems whose feasibility conditions are expressible in separator logic FO+conn. The idea behind combinatorial reconfiguration is to transform one configuration of a system into another while keeping the system running throughout the transformation. For vertex-subset problems on graphs, we can imagine a token placed on each vertex of a solution. We change the configuration by moving one token at a time to an unoccupied vertex, either by sliding it along an edge or by allowing it to jump to any unoccupied vertex. The requirement that the system keeps running means that the vertices occupied by tokens must form a feasible solution after every move. The algorithmic question is whether two given feasible solutions can be transformed into one another by a sequence of such moves. This viewpoint has been applied to a wide range of graph problems. We refer for example to the surveys [17, 70, 88].

A particularly relevant family of examples arises from reconfiguring vertex sets that hit or separate prescribed structures. For instance, Feedback Vertex Set Reconfiguration asks to transform one deletion set into another while maintaining that the remaining graph is acyclic. Parameterized algorithms and kernels for several such vertex-set reconfiguration problems were studied in [68]. More recently, Feedback Vertex Set Reconfiguration has been studied in its own right on planar graph classes [15]. Vertex Separator Reconfiguration under the token addition/removal, token jumping, and token sliding rules has been studied in [29], while [52] considers the reconfiguration of minimum s-t separators.

Since individual reconfiguration problems often require tailored arguments, there has also been substantial interest in algorithmic meta-theorems for reconfiguration. Mouawad, Nishimura, Raman, and Wrochna gave an MSO-based meta-theorem parameterized by treewidth and the length of the reconfiguration sequence [69]. Gima, Ito, Kobayashi, and Otachi obtained meta-theorems for reconfiguration under other structural parameters, such as neighborhood diversity and treedepth [50]. Further meta-algorithmic and structural results are known for sparse graph classes, for example for independent set and dominating set reconfiguration on sparse graphs [64, 83].

Our logic provides a new meta-algorithmic viewpoint. We consider *modulator reconfiguration* problems in which the current token set S is a deletion set and feasibility is expressed by a logical condition on the remainder $G - S$. By using FO+conn as the specifying logic, this captures, among others, Vertex Cover Reconfiguration, Feedback Vertex Set Reconfiguration, and s-t-Cut Reconfiguration. Let us stress again that we use separator logic to specify the property of $G - S$, problem, while we use the methods for subconnectivity logic to resolve the algorithmic reconfiguration problem.

Theorem 7.1 (see page 50). *There exist a computable function f and an absolute constant c such that, given a possibly unary-colored graph G, sets $S, T \subseteq V(G)$, a budget $b \in \mathbb{N}$, and a sentence $\psi \in \mathsf{FO+conn}$, one can decide the ψ-Modulator-Reconfiguration problem in time*

$$f(|\psi|, h(G), b) \cdot |G|^c.$$

The same method applies to *solution discovery*, where the initial token set need not be feasible and feasibility is required only at the end of the sequence. We refer to Section 7 for the precise definitions and the corresponding meta-theorem for Modulator Discovery.

Extending to regular path queries is not possible. The logic $\mathsf{FO+subconn}$ can be further extended by introducing a recursive scheme while keeping the fixed-parameter tractability property on every class of graphs excluding a minor; see Section 8.1.

We investigate how far one can push this line of ideas. In database theory and graph querying, *connectivity with regular constraints on the witnessing path* is a central notion. It appears in regular path queries, conjunctive regular path queries, and their extensions with regular relations on tuples of paths [28, 66, 4, 63, 43, 90, 77]. One might therefore hope to strengthen $\mathsf{FO+subconn}$ by requiring the connecting paths to satisfy simple regular constraints. We show that already with two colors and very simple regular constraints, the corresponding extension has, on monotone classes, bounded tree-width as its exact dependence frontier. The hardness direction additionally requires that the relevant grid-minor encodings, including their minor models, can be constructed efficiently.

2. Overview of the structural and algorithmic proofs

We now give an overview of the proof of our main structural theorems, Theorems 5.1 and 5.2.

2.1. From minor exclusion to bounded twin-width

Theorems Theorems 5.1 and 5.2 are derived from the following theorem, Theorem 5.3, which treats connected graphs equipped with rooted spanning trees. The passage to arbitrary rooted spanning forests uses Remark 3.1.

Theorem 5.3 (see page 28). *Let $\sigma = \{E, \preceq, <\}$ be the signature obtained by expanding the signature $\{E\}$ of graphs with two binary relations $\preceq$ and $<$. There exist computable functions ξ, f, a constant c, and an algorithm that, for a given connected graph G and a rooted spanning tree Y of G, computes in time $f(h(G)) \cdot |G|^c$ a σ-structure $\mathbf{G}$ such that*

- *the $\{E\}$-reduct of $\mathbf{G}$ is G,*
- *the $\preceq$-reduct of $\mathbf{G}$ is the tree order $\preceq_Y$,*
- *the $<$-reduct of $\mathbf{G}$ is a linear order,*
- *the twin-width of $\mathbf{G}$ is at most $\xi(h(G))$.*

The proof of this derivation as well as the proof of Theorem 5.3 will be the subject of Section 5. We now give an outline of the proof of Theorem 5.3.

Our proof is organized around *transductions*. Informally, a transduction is a uniform logical transformation between classes of structures. It may take a bounded number of copies of the input, add finitely many unary colors and then define the universe and relations of the output by formulas. Transductions therefore allow us to encode the desired structures inside a better-behaved auxiliary class.

Two preservation results are central. First, on classes of linearly ordered binary structures, monadic dependence is equivalent to bounded twin-width. Second, bounded twin-width is preserved under fixed transductions, with a computable change in the bound. We therefore do not prove bounded twin-width directly for every desired tree-ordered expansion. Instead, we construct an auxiliary linearly ordered class that is monadically dependent, and hence has bounded twin-width, and recover the desired structures from it by a fixed transduction.

Fix $t \in \mathbb{N}$, and let $\mathscr{H}_t$ be the class of connected graphs with Hadwiger number at most t. For every graph $G \in \mathscr{H}_t$ and every rooted spanning tree Y of G, we construct an auxiliary graph H, together with a rooted spanning tree Y_H and a linear order $<$, and we write the resulting tree- and linearly ordered structure as $(H, \preceq_{Y_H}, <_H)$. The construction is designed so that the class

$$\mathscr{M}_t = \{(H, \preceq_{Y_H}, <_H) \mid G \in \mathscr{H}_t, Y \text{ is a rooted spanning tree of } G\}$$

is monadically dependent. At the same time, $(H, \preceq_{Y_H}, <_H)$ is constructed so that it contains enough information to recover, by one fixed transduction, the original graph G together with the spanning-tree order defined by Y and a linear order $<_G$. We denote this expansion by $(G, \preceq_Y, <_G)$.

It follows qualitatively that $\mathscr{M}_t$ has bounded twin-width, because it is a linearly ordered monadically dependent class of binary structures. Since the class of the desired structures $(G, \preceq_Y, <_G)$ is obtained from $\mathscr{M}_t$ by a fixed transduction, preservation of twin-width under transductions then gives a qualitative bound for the desired class as well. To obtain the computable bound required by the algorithmic theorem, we additionally use the effective quantitative content of the tree-ordered weakly sparse characterization: the auxiliary spanning trees have maximum degree 3, and the graph reducts of all their tree-ordered minors exclude a biclique whose order is computable from t. This yields a computable twin-width bound for $\mathscr{M}_t$. The constructive linearization and recovery transductions then preserve computability of the final bound.

The proof of Theorem 5.3 is accordingly split into two phases (see Figure 1). In Phase I, starting from the pair (G, Y), we construct the auxiliary linearly ordered tree-ordered structure $(H, \preceq_{Y_H}, <_H)$ and prove that the class $\mathscr{M}_t$ of obtained structures forms a monadically dependent class. In Phase II, we show that no information needed for recovering the original graph and tree order has been lost: from $(H, \preceq_{Y_H}, <_H)$ we transduce back a linearly ordered expansion $(G, \preceq_Y, <_G)$ of the original tree-ordered graph. The following paragraphs give a more detailed informal description of these two phases before the full formal construction is presented.

- **Phase I.** Given a connected graph $G \in \mathscr{H}_t$ with a rooted spanning tree Y, we construct in polynomial time a tree-ordered graph $(H, \preceq_{Y_H})$ with a linearly ordered expansion $(H, \preceq_{Y_H}, <_H)$ of $(H, \preceq_{Y_H})$ such that the class

  $$\mathscr{M}_t = \{(H, \preceq_{Y_H}, <_H) \mid G \in \mathscr{H}_t, Y \text{ is a rooted spanning tree of } G\}$$

  is monadically dependent, and hence, as a linearly ordered class, has bounded twin-width.
- **Phase II.** From $(H, \preceq_{Y_H}, <_H)$ we transduce a linearly ordered expansion $(G, \preceq_Y, <_G)$ of the original tree-ordered graph. The linear order $<_G$ is inherited from the order $<_H$.

Consequently, the linear order $<_G$ is computable in polynomial time from (G, Y), and the class of all resulting structures $(G, \preceq_Y, <_G)$ has bounded twin-width because it can be transduced from $\mathscr{M}_t$.

Phase I. Our starting point is Theorem 1.2, stating that a weakly sparse class $\mathscr{C}$ of graphs excludes some graph as a minor if and only if the associated class $\mathscr{C}^F$ of expansions by rooted spanning-forest orders is monadically dependent.

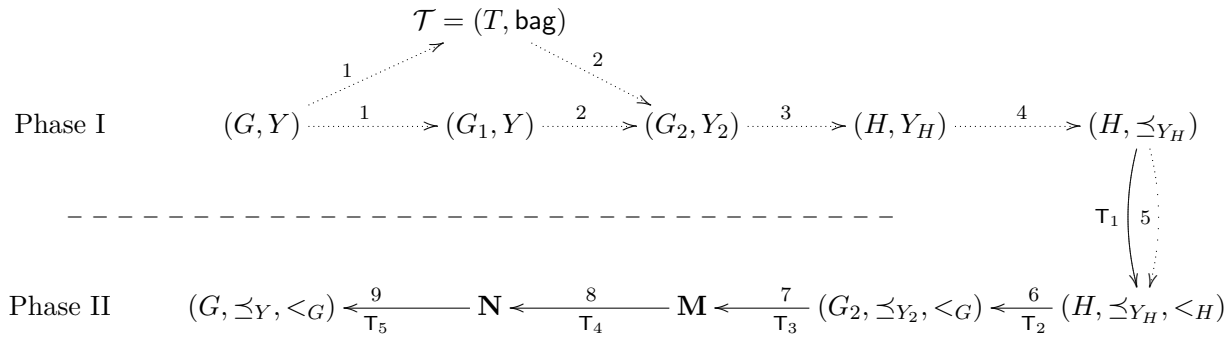


Figure 1: Outline of the proof of Theorem 5.3. Dotted arrows are polynomial-time computations and plain arrows are transductions. (Precisely, $G \xrightarrow[\mathsf{T}]{1} H$ means that at step 1 of the proof, we prove $H \in \mathsf{T}(G)$.) The main steps of the proof are: **1.** construction of a neat tree decomposition of G based on the structure theorem of Robertson and Seymour, as well as an augmentation of G so that every adhesion is a clique; **2.** construction of a subgraph whose nontrivial blocks have explicit no-apex almost-embedding witnesses; **3.** effective blockwise translation into a subcubic graph of bounded Hadwiger number; **4.** computation of the tree-ordered expansion; **5.** computation of a linear order that can be obtained by transduction T_1; **6.** transduction of the linearly ordered tree-ordered expansion of G_2 by T_2; **7.** transduction of a master structure that permits the graph to be handled margin-wise by T_3; **8.** recovery of the target tree order on the margins by transduction T_4; **9.** full recovery of the tree order by transduction T_5.

This observation gives monadic dependence, but it does not yet explain how to obtain bounded twin-width. The missing step is to pass from forest orders to linear orders. Here we use an idea inspired by a result of [20]: if the underlying spanning tree has bounded degree, then a suitable linearization of this tree order can be obtained by a transduction. Of course, graphs in a minor-closed class need not have bounded degree and therefore their rooted spanning trees may also have unbounded degree. The main technical point of the proof is to overcome this obstacle. We prove the following result, which may be of independent interest.

Theorem 2.1. *There exist computable functions ξ and f, a universal constant c, a fixed transduction L, effectively computable families of transductions $(\mathsf{T}_t)_{t\in\mathbb{N}}$ and $(\mathsf{R}_t)_{t\in\mathbb{N}}$, and an algorithm with the following properties, for every $t \in \mathbb{N}$.*

1. *Given a graph G with $h(G) \leqslant t$, the algorithm computes in time $f(t) \cdot |G|^c$ a connected subcubic graph H and a rooted spanning tree Y_H of H such that $h(H) \leqslant \xi(t)$ and*

$$G \in \mathsf{T}_t(H, \preceq_{Y_H}).$$

2. *If an arbitrary rooted spanning forest F of G is also given, the algorithm computes, within the same time bound, such a pair (H, Y_H) together with a DFS preorder $<_H$ of Y_H and a linear order $<_G$ on $V(G)$ such that*

$$(H, \preceq_{Y_H}, <_H) \in \mathsf{L}(H, \preceq_{Y_H}) \qquad \text{and} \qquad (G, \preceq_F, <_G) \in \mathsf{R}_t(H, \preceq_{Y_H}, <_H).$$

In particular, $(G, \preceq_F)$ can be transduced from $(H, \preceq_{Y_H})$ by some transduction effectively computable from t.

Thus, for every fixed t, all graphs of Hadwiger number at most t are interpretable by a transduction in tree-ordered subcubic graphs whose Hadwiger number is bounded by a computable function of t. In particular, every class excluding a fixed minor is a transduction of a tree-ordered subcubic class that again excludes a fixed minor.

Note that, by contrast, the analogous statement for the minor relation instead of transduction fails already for graphs of Hadwiger number at most 5: although every graph is a minor of a subcubic graph, there is no class $\mathscr{D}$ of subcubic graphs that excludes a fixed minor and whose minor closure contains every K_6-minor-free graph; this follows e.g. from [67, Theorem 17].

For every fixed t, let $\mathscr{M}_t$ be the class of all structures $(H, \preceq_{Y_H}, <_H)$ produced by the second part of Theorem 2.1 from connected graphs G with $h(G) \leqslant t$ and rooted spanning trees Y of G. The graph reducts of the structures in $\mathscr{M}_t$ are subcubic and have Hadwiger number at most $\xi(t)$, and each of their tree orders is induced by a rooted spanning tree. Thus, the corresponding tree-ordered class is a subclass of the class of full rooted spanning-tree expansions of a minor-excluded class, and is monadically dependent by Theorem 1.2. Applying the fixed linearization transduction L shows that $\mathscr{M}_t$ is monadically dependent. Since $\mathscr{M}_t$ consists of linearly ordered binary structures, it has bounded twin-width.

2.2. Sketch of the proof of Theorem 2.1

Let us sketch how we prove Theorem 2.1. We first give an informal introduction to tree decompositions and the structure theorem of Robertson and Seymour for graphs excluding a fixed graph as a minor, on which the proof of the theorem is based.

Very roughly, a tree decomposition represents a graph by arranging overlapping pieces in the shape of a tree. These pieces are called *bags*. The bags cover the graph: every vertex occurs in a bag, and every edge has both endpoints together in some bag. Moreover, the bags containing a fixed vertex induce a connected subtree of the decomposition tree. Thus, a vertex may occur in several bags, but those bags form one contiguous region.

If the decomposition tree is rooted, every non-root bag has a parent. For a bag x, its intersection with the parent bag is the *adhesion* of x. The vertices in x but not in its adhesion form the *margin* of x. Intuitively, the margin is the new part contributed by the bag, while the adhesion is the part shared with its parent. These overlaps are separators that control how the local pieces interact.

The Robertson-Seymour graph minor structure theorem states, informally, that every graph excluding a fixed minor has a tree decomposition of bounded adhesion whose pieces are almost embeddable in a surface of bounded Euler genus. Up to deleting a bounded number of exceptional vertices, called *apices*, and handling a bounded number of controlled local complications, called *vortices*, each piece behaves like a graph drawn on a surface.

We begin with a connected graph $G \in \mathscr{H}_t$. In Step 1, we use the Robertson-Seymour structure theorem to compute a tree decomposition and augment G to a graph G_1 in which every adhesion is a clique. We may further arrange that the decomposition is *neat*, meaning that it satisfies additional structural properties specified later.

The margins of a neat tree decomposition partition the vertex set. For each non-root node x of the decomposition tree, we choose a representative $\beta(x)$ in the margin of x and a neighbor $\alpha(x)$ in the adhesion of x that belongs to the margin of the parent of x; see Figure 2. A representative is also chosen for the root node, for which no vertex $\alpha(x)$ is needed.

In Step 2, we delete all edges between distinct margins except the selected edges $\alpha(x)\beta(x)$. We also delete the apex-incident edges and then add back an inclusion-minimal set that preserves connectivity inside each margin. After a corresponding local modification of the original spanning tree, we obtain a spanning tree Y_2 of the new graph G_2; see Figure 3. We prove that G_2 excludes a fixed apex graph as a minor. Recall that an apex graph is a graph that becomes planar after deleting one vertex.

In Step 3, we apply the degree-reduction construction of [67] to G_2, according to which every graph in an apex-minor-free minor-closed class is a minor of a subcubic graph in a fixed proper

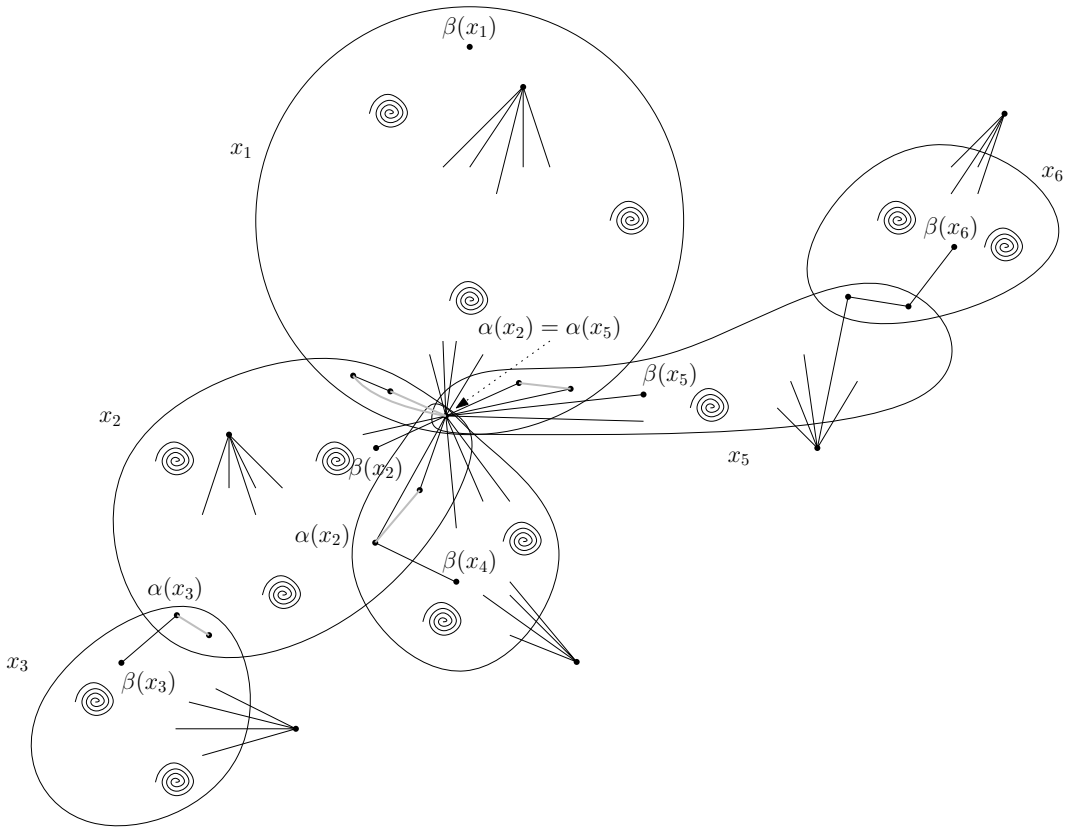


Figure 2: In the first step, we apply the decomposition theorem of Robertson and Seymour and process the decomposition so that it becomes neat. For each node x of the decomposition tree, we select a representative vertex $\beta(x)$ from its margin $\mathsf{mrg}(x)$. If x is not the root, we also select a neighbor $\alpha(x) \in \mathsf{adh}(x) \cap \mathsf{mrg}(\mathsf{parent}(x))$ of $\beta(x)$.

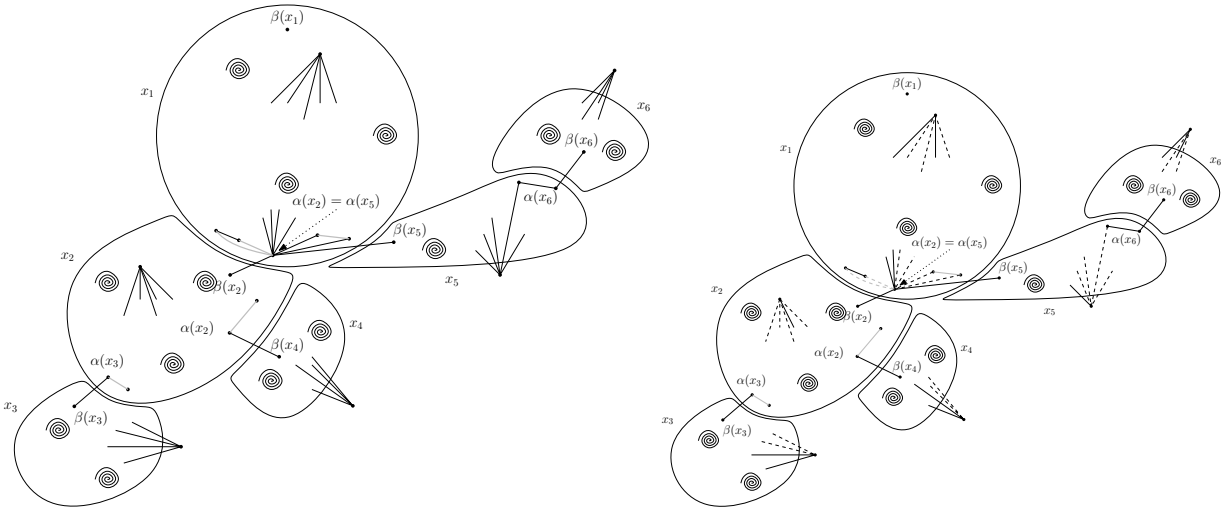


Figure 3: Illustration of the second step. We delete all edges between margins, except for the single edge $\alpha(x)\beta(x)$ keeping the tree structure of the tree decomposition intact. We also keep only a minimal set of edges incident with apex vertices so as to preserve the connectivity of the margins and such that the resulting blocks fall into an apex-minor-free class.

minor-closed superclass. Applied to G_2, the construction produces a connected subcubic graph $H \in \mathscr{H}_{\xi_c(t)}$ such that G_2 is a minor of H, where ξ_c is computable, and transfers the spanning tree Y_2 to a spanning tree Y_H of H. It runs in time $a(t) \cdot |G_2|^{c_0}$ for a computable function a and an absolute constant c_0; see Figure 4.

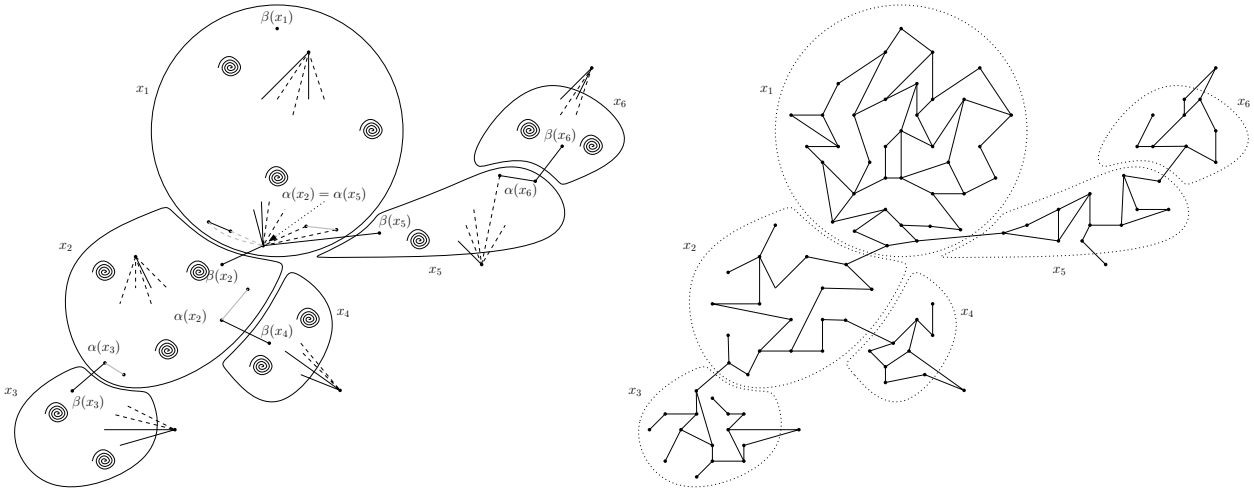


Figure 4: Illustration of the third step. We move to a connected subcubic graph H from a class excluding a minor.

In Step 4, we form the tree-ordered lift $(H, \preceq_{Y_H})$. The relevant class of forest-ordered subcubic graphs is monadically dependent by [20]. In Step 5, we compute a linear order $<_H$ of $V(H)$ from a depth-first traversal of Y_H, which is also obtainable by a transduction of $(H, \preceq_{Y_H})$. Hence, the resulting class $\mathscr{M}_t$ of linearly ordered structures $(H, \preceq_{Y_H}, <_H)$ is monadically dependent and therefore has bounded twin-width.

Phase II. Phase II begins by transducing $(G_2, \preceq_{Y_2})$ from $(H, \preceq_{Y_H}, <_H)$ and transforming the computed linear order $<_H$ to a linear order $<_G$ of $V(G_2)$ (Step 6). We then construct a master structure $\mathbf{M}$ that lets us work one margin at a time while retaining access to the order of the decomposition tree (Step 7); see Figure 5.

In Step 8, a transduction restores the edge relation of G_1 inside each margin and the local trace order there; see Figure 6. The remaining edges between margins and the remaining order comparisons are recovered in Step 9.

Finally, we use the tree version of Simon's factorization forest theorem and the fact that simple MSO interpretations of colored rooted trees can be simulated by first-order transductions of the associated tree orders [23]. This restores the complete tree order $\preceq_Y$ in Step 9 and yields $(G, \preceq_Y, <_G)$ as a transduction of $(H, \preceq_{Y_H}, <_H)$. A related margin-wise reconstruction was used in [81] for separator logic model checking on classes excluding a topological minor.

3. Preliminaries

This section contains definitions and preliminary results that we use in our proofs.

3.1. Graph theory

Let G be a graph. A *forest of* G is an acyclic subgraph $F \subseteq G$. A forest is *spanning* if $V(F) = V(G)$ and it is *full* if the components of F are equal to the components of G (in this paper, connected

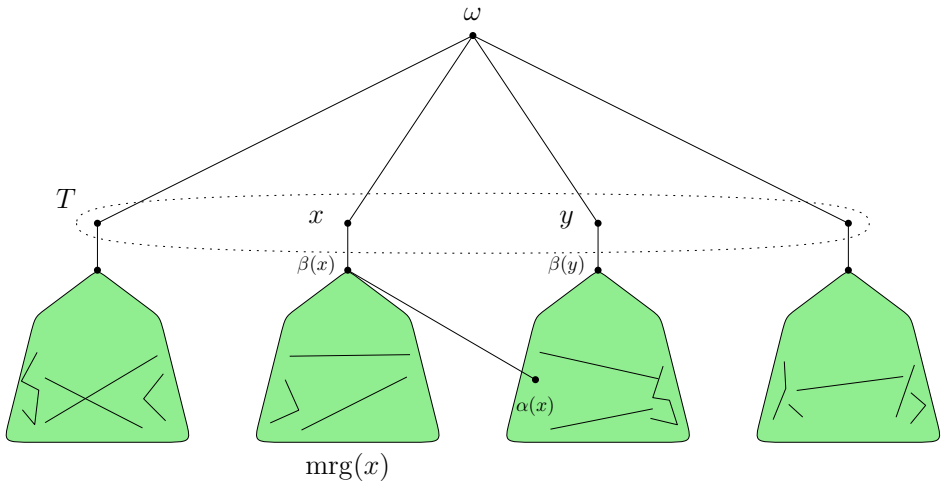


Figure 5: In $\mathbf{M}$, the children of the root ω are copies of the decomposition nodes, and the separate relation $\trianglelefteq$ records their tree-decomposition order. Under each such node x, a subtree rooted at $\beta(x)$ contains the vertices of $\mathsf{mrg}(x)$.

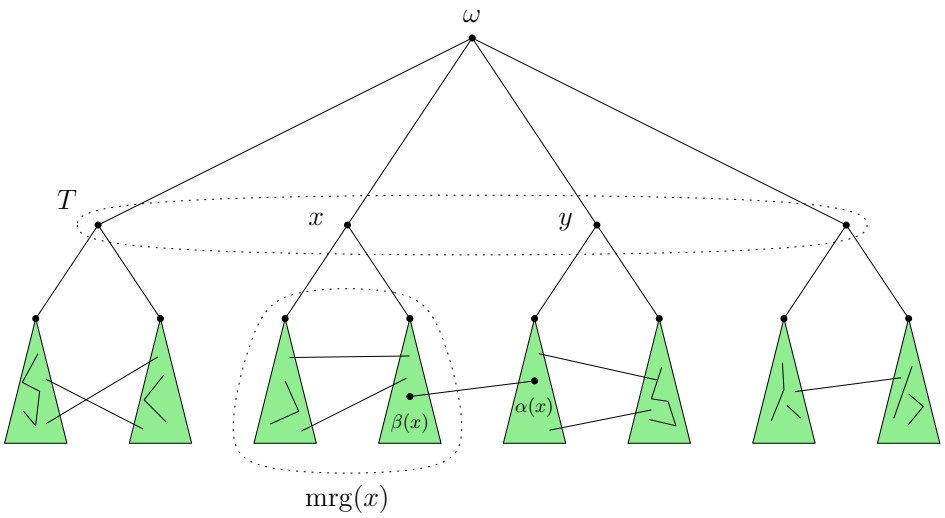


Figure 6: In $\mathbf{N}$, each margin carries the restriction of the edge relation of G_1 and the local trace order of Y. The remaining order comparisons are recovered in Step 9.

components are vertex sets). It is a *tree* if it is connected. A *rooted spanning forest* is a spanning forest with one distinguished vertex, called *root*, in each connected component. If we want to make the roots explicit, we write (F, R), where R denotes the set of roots and, if F is a tree, we write (F, r), where r is the unique root of F.

Let G be a graph, and let $uv \in E(G)$ be an edge. The *contraction* of uv is the operation that deletes the edge uv, identifies its endpoints u and v into a new vertex adjacent to every vertex that was adjacent to at least one of u and v and then removes any parallel edges.

We say that a graph H is a *minor* of G and write $H \leqslant_m G$, if H can be obtained from a subgraph of G by a sequence of edge contractions.

Equivalently, $H \leqslant_m G$ if there exists a family $(B_v)_{v \in V(H)}$ of non-empty pairwise vertex-disjoint connected subsets of $V(G)$, called *branch sets*, such that for every edge $uv \in E(H)$, there is at least

one edge of G with one endpoint in B_u and the other endpoint in B_v. Such a family is called a *minor model* of H in G. A minor model is *surjective* if its branch sets partition $V(G)$. Equivalently, a surjective minor model is specified by a surjective map $f\colon V(G) \to V(H)$ such that each branch set $f^{-1}(v)$ induces a connected subgraph of G and, for every edge $uv \in E(H)$, some edge of G joins $f^{-1}(u)$ and $f^{-1}(v)$.

The *Hadwiger number* $h(G)$ of a graph G is the maximum integer t such that $K_t \leqslant_m G$. Using the minor containment algorithm given in [60], the Hadwiger number of a graph G can be computed in time $f(h(G))\,|G|^{1+o(1)}$, where f is a computable function.

3.2. Order theory

Let X be a finite set. A binary relation $\preceq$ on X is a *partial order* if it is reflexive (for all $x \in X$, $x \preceq x$), antisymmetric (for all $x, y \in X$, if $x \preceq y$ and $y \preceq x$, then $x = y$), and transitive (for all $x, y, z \in X$, if $x \preceq y$ and $y \preceq z$, then $x \preceq z$). In this case, the pair $(X, \preceq)$ is called a *partially ordered set* (or *poset*). A *cover* of a poset $(X, \preceq)$ is a pair (u, v) of elements of X such that $u \prec v$ and there exists no z with $u \prec z \prec v$. We denote by $u \prec\colon v$ the (definable) property that (u, v) is a cover. The *cover graph* of a poset $(X, \preceq)$ is the oriented graph with vertex set X, whose arcs are the covers of $(X, \preceq)$. The *undirected cover graph* of $(X, \preceq)$ is the underlying undirected graph of the cover graph of $(X, \preceq)$. Given two elements $x, y \in X$ in a partial order, we write $x \parallel y$ if x and y are incomparable. For $B \subseteq X$, the restriction of $\preceq$ to B is $\preceq \cap (B \times B)$.

A partial order $\preceq$ on X is a *forest order* if, for every $y \in X$, the set $\{x \in X \mid x \preceq y\}$ is linearly ordered by $\preceq$. The minimal elements of $\preceq$ are called the *roots* of $\preceq$. A forest order is a *tree order* if it has a minimum (i.e., a unique root r). Equivalently, a partial order $\preceq$ is a forest order (resp. a tree order) if its cover graph is a rooted forest (resp. a rooted tree) with all arcs oriented away from the roots.

As any upper-bounded set of elements in a forest order is a chain, we will allow in formulas statements like $w = \max\{u \preceq v\colon P(u)\}$ and $w = \max\{u \prec v\colon P(u)\}$ (where P is a first-order definable property, possibly with parameters), and these notations represent first-order definable statements. (We do not define $\max\{u \preceq v\colon P(u)\}$ directly, as $\{u \preceq v\colon P(u)\}$ might be empty.)

An *ancestor* of an element v in a forest order F is an element u with $u \preceq v$ and the *parent* of v is

$$\pi_F(v) := \begin{cases} v & \text{if } v \text{ is a root (i.e., a minimal element),} \\ w & \text{if } w \prec\colon v. \end{cases}$$

Note that we use the convention $\pi_F(v) = v$ at a root. This makes π_F a total map on a forest-ordered set. For decomposition trees, we use the separate notation parent and the convention $\mathsf{parent}(r) = \bot$, with $\mathsf{bag}(\bot) = \emptyset$.

If $(X, \preceq)$ is a tree-ordered set, then the *infimum* (or least common ancestor) of two elements of X always exists and is first-order definable. We denote it by $x \curlywedge y$. For forest orders, we define the *least common ancestor* relation by

$$\mathsf{lca}(a; x, y) := (a \preceq x) \wedge (a \preceq y) \wedge \forall a'\big((a' \preceq x \wedge a' \preceq y) \to (a' \preceq a)\big).$$

Using the notation introduced above, we can also define $\mathsf{lca}(a; x, y)$ as

$$\mathsf{lca}(a; x, y) := (a = \max\{v \preceq x\colon v \preceq y\}).$$

Every rooted tree (Y, r) naturally defines a tree order $\preceq_Y$ on $V(Y)$ by setting

$$x \preceq_Y y \quad \iff \quad \text{there is no path connecting } y \text{ and } r \text{ in } Y - x.$$

Analogously, every rooted forest (F, R) (where R denotes the set of the roots) defines a forest order $\preceq_F$ on $V(F)$ by setting

$$x \preceq_F y \quad \iff \quad \begin{array}{c} x \text{ and } y \text{ belong to the same component of } F, \text{ and, for the unique} \\ \text{root } r \in R \text{ of that component, there is no path from } y \text{ to } r \text{ in } F - x. \end{array}$$

Conversely, every finite forest order $\preceq$ is induced in this way by a rooted forest, which is the undirected cover graph of $\preceq$ rooted at minimal elements of $\preceq$.

A binary relation $<$ on X is a *(strict) linear order* if it is irreflexive (for all $x \in X$, not $x < x$), transitive, and total (for all distinct $x, y \in X$, either $x < y$ or $y < x$). In this case, the pair $(X, <)$ is called a *linearly ordered set*.

Let G be a graph. We use the notation

- $F, F_1, F_2, \ldots$ for rooted spanning forests of G,
- $Y, Y_1, Y_2, \ldots$ for rooted spanning trees of G (which requires that G is connected),
- T for the decomposition tree of a tree decomposition of G (defined below), and
- $\preceq$ and $<$ for forest and linear orders on the vertex set of G (possibly unrelated to the edges of the graph).

Let $\mathscr{C}$ be a class of graphs. We define

$$\mathscr{C}^{\Upsilon} := \big\{(G, \preceq_F) \;\big|\; G \in \mathscr{C},\ F \text{ is a full rooted spanning forest of } G\big\},$$

and

$$\mathscr{C}^{F} := \big\{(G, \preceq_F) \;\big|\; G \in \mathscr{C},\ F \text{ is a rooted spanning forest of } G\big\}.$$

Remark 3.1. We will use the following reductions between forests and trees, and between disconnected and connected structures.

Given a graph G with a rooted spanning forest F, add a universal vertex u and extend F to a spanning tree Y by joining u to every root of F and choosing u as the new root. For the resulting connected graph $\widehat{G} = G \oplus K_1$, we have $(G, \preceq_F) = (\widehat{G}, \preceq_Y)[V(G)]$. If G excludes K_t as a minor, then $\widehat{G}$ excludes K_{t+1} as a minor, since at most one branch set of a clique-minor model can contain u. Deleting u is a first-order interpretation, which preserves monadic dependence, and taking an induced substructure does not increase twin-width. The construction and the restriction take polynomial time. Thus, for the results below whose bounds depend only on an excluded minor, it suffices to consider connected graphs equipped with rooted spanning trees.

For classes of tree-ordered graphs, the hereditary closure obtained by retaining the root and the unrestricted hereditary closure are dependent simultaneously, and monadically dependent simultaneously. Indeed, every arbitrary induced substructure is obtained from a root-preserving induced substructure by possibly deleting its root, which is the definable least element. Here we take induced substructures in the relational signature, the restricted order need not be induced by a spanning forest of the restricted graph.

Finally, a class closed under taking connected components is dependent if and only if its connected members are dependent, and the same holds for monadic dependence. This follows from the first-order composition theorem due to Feferman and Vaught [40] for disjoint unions and applies in particular to hereditary classes. For relational structures, connectedness refers to the Gaifman graph of the full signature.

3.3. Basic finite model theory

3.3.1. Binary relational structures

A *binary relational signature* is a finite set of unary and binary relation symbols. A σ*-structure* $\mathbf{M}$ consists of a finite set M, called its *domain* or *universe*, and an interpretation $R(\mathbf{M}) \subseteq M^r$ for every relation symbol $R \in \sigma$ of arity $r \in \{1, 2\}$. Unary relations are also called *colors*. For every formula $\varphi(\bar{x})$, we write $\varphi(\mathbf{M}) := \{\bar{a} \in M^{|\bar{x}|} : \mathbf{M} \models \varphi(\bar{a})\}$.

The *Gaifman graph* $\mathrm{Gaif}(\mathbf{M})$ of a binary relational structure $\mathbf{M}$ is the simple undirected graph with vertex set M in which two distinct elements u, v are adjacent if and only if there is a binary relation symbol $R \in \sigma$ such that $(u, v) \in R(\mathbf{M})$ or $(v, u) \in R(\mathbf{M})$.

We regard a simple undirected graph G as a structure with signature $\{E\}$, universe $V(G)$, and with E interpreted as the symmetric adjacency relation of G.

Let $\mathbf{M}$ be a σ-structure and let $\tau \subseteq \sigma$. The τ*-reduct* of $\mathbf{M}$, denoted by $\mathbf{M}\restriction_\tau$, is obtained by forgetting all relations outside τ. If $\tau = \{R\}$, we abbreviate $\mathbf{M}\restriction_{\{R\}}$ by $\mathbf{M}\restriction_R$. Conversely, if $\tau \subseteq \sigma$ and $\mathbf{M}$ is a τ-structure, a σ*-expansion* of $\mathbf{M}$ is a σ-structure whose τ-reduct is $\mathbf{M}$. It is *monadic* if all symbols in $\sigma \setminus \tau$ are unary. We write $\mathbf{M}_<$ for an expansion of $\mathbf{M}$ by a linear order $<$. Finally, if $U \subseteq M$, then $\mathbf{M}[U]$ denotes the substructure induced by U.

Whenever a binary relational signature σ contains a distinguished binary relation symbol Z, we consider only σ-structures $\mathbf{M}$ in which $Z(\mathbf{M})$ is symmetric and irreflexive. Hence, the $\{Z\}$-reduct $\mathbf{M}\restriction_Z$ is a simple undirected graph with vertex set M and edge relation $Z(\mathbf{M})$. Connectivity with respect to Z always means connectivity in this graph.

3.3.2. Logic and transductions

Let σ be a binary relational signature. The set $\mathsf{FO}[\sigma]$ (or simply FO if σ is clear from the context) of first-order formulas over σ is obtained from atomic formulas over σ by closing under the Boolean connectives and the quantifiers $\exists x$ and $\forall x$ ranging over elements. Monadic second-order logic MSO extends first-order logic by quantifiers $\exists X$ and $\forall X$, ranging over sets of elements. The semantics are defined as usual and we refer to [62] for more background on logic.

Let σ and τ be finite binary relational signatures. A *simple interpretation* I of τ-structures in σ-structures is given by a σ-formula $\nu(x)$ and, for every relation symbol $R \in \tau$ of arity $r \in \{1, 2\}$, a σ-formula $\rho_R(x_1)$, or $\rho_R(x_1, x_2)$, respectively. For a σ-structure $\mathbf{M}$ with domain M, the structure $\mathsf{I}(\mathbf{M})$ has domain $\nu(\mathbf{M})$ and interprets each such symbol R by $R(\mathsf{I}(\mathbf{M})) := \rho_R(\mathbf{M}) \cap \nu(\mathbf{M})^r$. Unless another logic is specified, interpretations and transductions in this paper are first-order.

If Σ is a finite set of unary relation symbols disjoint from σ, then a Σ*-coloring* of a σ-structure $\mathbf{M}$ is any $(\sigma \cup \Sigma)$-expansion of $\mathbf{M}$. We write $\Gamma_\Sigma(\mathbf{M})$ for the set of all Σ-colorings of $\mathbf{M}$.

A *copy operation* transforms a structure $\mathbf{M}$ with domain M into a structure $\mathrm{Copy}_m(\mathbf{M})$ with domain $M \times [m]$, where $[m] = \{1, \ldots, m\}$. Each relation R of arity r in the original signature is interpreted component-wise, that is,

$$R(\mathrm{Copy}_m(\mathbf{M})) := \big\{((a_1, i), \ldots, (a_r, i)) : (a_1, \ldots, a_r) \in R(\mathbf{M}),\ i \in [m]\big\}.$$

The copy operation also adds unary relation symbols $P_1, \ldots, P_m$ and a binary relation symbol $\simeq$, interpreted by

$$P_i(\mathrm{Copy}_m(\mathbf{M})) := M \times \{i\} \quad \text{and} \quad (a, i) \simeq (b, j) \quad \iff \quad a = b.$$

Thus, the predicates P_i distinguish the copies, while $\simeq$ identifies copies of the same original element.

A *transduction* is a composition of copy operations, unary colorings, and simple interpretations. By the standard normalization [45], we may write it as $\mathsf{T} = (m, \Sigma, \mathsf{I})$, where $m \geqslant 1$, Σ is a finite set

of fresh unary predicates, and I is a simple interpretation. Thus,

$$\mathsf{T}(\mathbf{M}) = \{\mathsf{I}(\mathbf{N}) : \mathbf{N} \in \Gamma_\Sigma(\mathrm{Copy}_m(\mathbf{M}))\}.$$

We call this transduction *non-copying* if $m = 1$, and *domain-preserving* if, in addition, its universe formula selects every input element.

We say that a structure $\mathbf{N}$ *can be transduced from* $\mathbf{M}$ *by* T if $\mathbf{N} \in \mathsf{T}(\mathbf{M})$. By extension, for a class $\mathscr{C}$ of structures, we define

$$\mathsf{T}(\mathscr{C}) := \bigcup_{\mathbf{M} \in \mathscr{C}} \mathsf{T}(\mathbf{M}).$$

If $\mathscr{D}$ is a class of τ-structures, we say that $\mathscr{D}$ *can be transduced from* $\mathscr{C}$ if there exists a transduction T such that $\mathscr{D} \subseteq \mathsf{T}(\mathscr{C})$. We remark that interpretations and transductions can be defined for all extensions of first-order logic that we consider in this paper.

We shall repeatedly carry auxiliary relations through a transduction. The following standard observation makes this precise.

Fact 3.2. *Let T be a non-copying, domain-preserving transduction from σ-structures to τ-structures, and let ρ be a finite binary relational signature disjoint from $\sigma \cup \tau$. Then T extends from $(\sigma \cup \rho)$-structures to $(\tau \cup \rho)$-structures by carrying every relation in ρ over unchanged.*

The next lemma says that, over a class excluding a fixed minor, all relational information supported by the Gaifman graph can be encoded by a fixed transduction.

Lemma 3.3 (special case of [14, Lemma 3.1])**.** *Let $\mathscr{C}$ be a class of graphs excluding a fixed minor and let σ be a finite binary relational signature. There is a domain-preserving transduction Expand_σ such that, for every $G \in \mathscr{C}$, $\mathsf{Expand}_\sigma(G)$ contains every σ-structure $\mathbf{M}$ with domain $V(G)$ and $\mathrm{Gaif}(\mathbf{M}) \subseteq G$.*

We will multiple times use the following special case of the above lemma.

Corollary 3.4. *Let $\mathscr{C}$ be a class of graphs excluding a fixed minor. There are fixed transductions $\mathsf{Monotone}$ and Orient such that, for every $G \in \mathscr{C}$, the class $\mathsf{Monotone}(G)$ contains every subgraph of G, and $\mathsf{Orient}(G)$ contains every orientation of G. Consequently every orientation of every subgraph of G belongs to $(\mathsf{Orient} \circ \mathsf{Monotone})(G)$.*

3.3.3. Dependence and monadic dependence

Let $\mathcal{L}$ be a logic extending first-order logic. An $\mathcal{L}$-formula $\psi(\bar{x}; \bar{y})$ is *independent* in a class $\mathscr{C}$ if, for every positive integer n, there are $\mathbf{M} \in \mathscr{C}$, tuples $\bar{a}_i \in M^{|\bar{x}|}$ for $i \in [n]$, and tuples $\bar{b}_J \in M^{|\bar{y}|}$ for $J \subseteq [n]$ such that

$$\mathbf{M} \models \psi(\bar{a}_i, \bar{b}_J) \qquad \Longleftrightarrow \qquad i \in J.$$

The class $\mathscr{C}$ is *$\mathcal{L}$-dependent* if no $\mathcal{L}$-formula is independent in $\mathscr{C}$. It is *monadically $\mathcal{L}$-dependent* if the class of all expansions of members of $\mathscr{C}$ by finitely many unary predicates is $\mathcal{L}$-dependent. When the logic is not displayed, dependence and monadic dependence refer to FO.

We mainly use the following characterization, which is specific here to first-order logic and first-order transductions.

Theorem 3.5 ([3])**.** *A class $\mathscr{C}$ of structures is monadically dependent if and only if the class of all finite graphs cannot be transduced from $\mathscr{C}$.*

We note the following corollary, which immediately follows from the definition of transductions.

Corollary 3.6. *Let $\mathscr{C}$ be a monadically dependent class of structures and let T be a transduction. Then $\mathsf{T}(\mathscr{C})$ is monadically dependent, too.*

For first-order logic, dependence and monadic dependence coincide on hereditary classes of relational structures [18].

3.4. Monadic dependence of tree-ordered binary structures

3.4.1. Linearly ordered structures and twin-width

We write $\mathrm{tww}(\mathbf{M})$ for the twin-width of $\mathbf{M}$. We do not use its definition directly, our arguments rely on the following characterization of ordered structures and its algorithmic consequence.

Theorem 3.7 ([10, Theorem 3])**.** *Let $\mathscr{C}$ be a class of finite linearly ordered binary structures over a finite signature. Then $\mathscr{C}$ has bounded twin-width if and only if $\mathscr{C}$ is monadically dependent.*

Theorem 3.8 ([10, Theorem 2] and [13, Theorem 1])**.** *There exist a computable function f and an absolute constant c such that, given a finite linearly ordered binary structure $\mathbf{M}$ and an FO-sentence φ, one can decide whether $\mathbf{M} \models \varphi$ in time*

$$f\big(|\varphi|, \mathrm{tww}(\mathbf{M})\big) \cdot \|\mathbf{M}\|^c.$$

For this statement, one first takes the reduct to the relation symbols occurring in φ together with the distinguished linear order. Thus, the dependence on the finite signature is absorbed by $|\varphi|$, while the polynomial exponent remains absolute.

Having bounded twin-width is preserved by transduction, as stated in the next lemma.

Lemma 3.9 ([13])**.** *Let T be a fixed transduction. There is a computable function g such that the following holds. If $\mathscr{C}$ is a class of binary structures with twin-width bounded by d, then $\mathsf{T}(\mathscr{C})$ has twin-width bounded by $g(d)$. In particular, every class of bounded twin-width is monadically dependent.*

The constructive version with the computable bound $g(d)$ follows from the constructive proof of [13, Theorem 8.1].

3.4.2. Forest-ordered graphs excluding a minor

The connection between forest-ordered classes with excluded minors and monadic dependence was made recently in [20]. A graph class $\mathscr{C}$ is *weakly sparse* if there is a $t \in \mathbb{N}$ such that no graph in $\mathscr{C}$ contains the complete bipartite graph $K_{t,t}$ as a subgraph.

Theorem 1.2 ([20, Theorem 1.4])**.** *Let $\mathscr{C}$ be a weakly sparse class of graphs. Then the following are equivalent:*
(1) $\mathscr{C}$ excludes some graph as a minor;
(2) $\mathscr{C}^{\Upsilon}$ is monadically dependent (for FO);
(3) $\mathscr{C}^{F}$ is monadically dependent (for FO).

We next consider minors of tree-ordered graphs. Let $(G, \preceq)$ be a tree-ordered graph (note that this tree order need not be related to the edges of G). An *elementary $\prec$-contraction* identifies a non-root vertex v with the unique element u such that $u \prec: v$. In the graph relation, the new vertex is adjacent to every vertex formerly adjacent to u or v (loops are discarded and parallel edges are identified). The order is the induced sub-order (i.e. the sub-order of $(V(G), \preceq)$ induced by $V(G) \setminus \{v\}$). A *$\prec$-contraction* is a possibly empty sequence of elementary $\prec$-contractions. A *tree-ordered minor* of $(G, \preceq)$ is any tree-ordered graph obtained by:

(i) taking an induced tree-ordered subgraph of $(G, \preceq)$ that contains the root r;
(ii) applying a $\prec$-contraction; and
(iii) deleting an arbitrary set of edges from the graph relation, while retaining the resulting order.

We write $\mathrm{Minor}_{\prec}(G, \preceq)$ for the set of all tree-ordered minors of $(G, \preceq)$, and for a class $\mathscr{C}$, we set $\mathrm{Minor}_{\prec}(\mathscr{C}) := \bigcup_{(G,\preceq)\in\mathscr{C}} \mathrm{Minor}_{\prec}(G, \preceq)$. The following was shown in [20]:

Theorem 3.10 ([20, Theorem 1.3])**.** *Let D be an integer and let $\mathscr{C}$ be a class of tree-ordered weakly sparse graphs whose $\prec$-reduct cover graphs have maximum degree at most D. Then the following are equivalent:*
(1) $\mathscr{C}$ is monadically dependent;
(2) $\mathscr{C}$ has bounded twin-width;
(3) the E-reduct of $\mathrm{Minor}_{\prec}(\mathscr{C})$ is weakly sparse.

The following is the constructive part of the implication (3) ⇒ (2) in the proof of Theorem 3.10. In that proof, the ordered-grid bound, the finite chain-antichain bounds, and the coloring bound for the conflict graph used there are effective. Tracing these bounds yields a computable function of D and r only.

Lemma 3.11. *There exists a computable function β such that the following holds. Let $(G, \preceq)$ be a tree-ordered graph whose cover graph has maximum degree at most D. If the E-reduct of every tree-ordered minor of $(G, \preceq)$ excludes $K_{r,r}$ as a subgraph, then*

$$\mathrm{tww}(G, \preceq) \leqslant \beta(D, r).$$

3.5. Tree decompositions

3.5.1. Definitions and terminology

A *rooted tree decomposition* of a graph G is a pair $\mathcal{T} = (T, \mathsf{bag})$, where T is a rooted tree and $\mathsf{bag}\colon V(T) \to 2^{V(G)}$ assigns to each node $x \in V(T)$ a set $\mathsf{bag}(x) \subseteq V(G)$, called the *bag* of x, such that:

- $V(G) = \bigcup_{x\in V(T)} \mathsf{bag}(x)$;
- for every edge $uv \in E(G)$, there exists $x \in V(T)$ with $\{u, v\} \subseteq \mathsf{bag}(x)$; and
- for every vertex $v \in V(G)$, the set $\{x \in V(T) \mid v \in \mathsf{bag}(x)\}$ induces a non-empty connected subtree of T.

Let $\mathcal{T} = (T, \mathsf{bag})$ be a rooted tree decomposition of a graph G. Let r be the root of T. We denote by $\mathsf{parent}(\cdot)$ the parent function induced on T and write $\mathsf{parent}(r) = \perp$ and set $\mathsf{bag}(\perp) = \emptyset$.

For every node $x \in V(T)$, we define

- its *adhesion* $\mathsf{adh}(x) := \mathsf{bag}(x) \cap \mathsf{bag}(\mathsf{parent}(x))$,
- its *margin* $\mathsf{mrg}(x) := \mathsf{bag}(x) \setminus \mathsf{adh}(x)$,
- its *component* $\mathsf{comp}(x) = \bigcup_{y \succeq_T x} \mathsf{mrg}(y)$, and
- its *torso* $\mathsf{torso}(x)$ as the graph on the vertex set $\mathsf{bag}(x)$ obtained from the induced subgraph $G[\mathsf{bag}(x)]$ by adding, for every neighbor y of x in T, all edges between vertices of $\mathsf{bag}(x) \cap \mathsf{bag}(y)$. In other words, for every neighbor y of x in the decomposition tree, the adhesion $\mathsf{bag}(x) \cap \mathsf{bag}(y)$ is turned into a clique.

The *adhesion* of $\mathcal{T}$ is the maximum size of an adhesion.

Definition 3.12. *A rooted tree decomposition $\mathcal{T} = (T, \mathsf{bag})$ of a graph G is* neat *if*

1. *the margin $\mathsf{mrg}(x)$ of every node x is non-empty,*
2. *the margin $\mathsf{mrg}(x)$ of every node x induces a connected subgraph of G,*
3. *every vertex of the adhesion $\mathsf{adh}(x)$ of every node x has a neighbor in $\mathsf{mrg}(x)$, and*
4. *for every non-root node x, the adhesion of x intersects the margin of its parent (i.e., $\mathsf{adh}(x) \cap \mathsf{mrg}(\mathsf{parent}(x)) \neq \emptyset$).*

The following fact is standard and based on axioms of tree decompositions and the assumption that margins in neat tree decompositions are non-empty.

Fact 3.13. *Let $\mathcal{T} = (T, \mathsf{bag})$ be a neat tree decomposition of a graph G. Then, for all nodes $x, y \in V(T)$, we have $\mathsf{mrg}(x) = \mathsf{mrg}(y)$ if and only if $x = y$, and the family $\{\mathsf{mrg}(x)\}_{x \in V(T)}$ is a partition of $V(G)$.*

Let $\mathcal{T} = (T, \mathsf{bag})$ be a neat rooted tree decomposition of a graph G. For every non-root node $x \in V(T)$, we will later carefully choose a vertex

$$\alpha(x) \in \mathsf{adh}(x) \cap \mathsf{mrg}(\mathsf{parent}(x)).$$

Since every vertex of $\mathsf{adh}(x)$ has a neighbor in $\mathsf{mrg}(x)$, we can and will further fix a vertex

$$\beta(x) \in \mathsf{mrg}(x)$$

adjacent to $\alpha(x)$, which we call the *vertex representing the margin of x*. For the root r, we fix an arbitrary vertex $\beta(r) \in \mathsf{mrg}(r)$.

It follows from Fact 3.13 that the map $\beta \colon V(T) \to V(G)$ is injective and that for every $v \in V(G)$, there exists a unique node $b(v) \in V(T)$ such that

$$v \in \mathsf{mrg}(b(v)).$$

The map $b : V(G) \to V(T)$ is called the *placement map* of $\mathcal{T}$.

3.5.2. Tree decompositions of graphs excluding a fixed minor

We now state the algorithmic form of the Robertson–Seymour graph minor structure theorem while avoiding the technical details as much as possible.

We use the usual bounded-width notion of an *almost-embedding*; see [36, Section 5]. For $k \in \mathbb{N}$, a surface Σ, a graph G, and a set $A \subseteq V(G)$ with $|A| \leqslant k$, we say that G is *k-almost embeddable in Σ with apex set A* if $G - A$ consists of a graph embedded in Σ together with at most k attached pieces, called *vortices*, each of width at most k. We leave the standard details to the cited definition.

We call an almost-embedding *explicit* if its standard finite witness, including the associated decompositions where required, is supplied. We call an almost-embedding *no-apex* if its apex set is empty. Finally, a graph is *k-almost embeddable* if it is k-almost embeddable in some surface of Euler genus at most k.

Theorem 3.14 (Robertson–Seymour; Grohe–Kawarabayashi–Reed)**.** *There is a computable function f and an algorithm with the following property. Given a graph J and a connected graph G, the algorithm either outputs a model of J as a minor of G, or computes a supergraph $G' \supseteq G$ on the same vertex set as G and a tree decomposition $\mathcal{T} = (T, \mathsf{bag})$ of G' of adhesion at most $k := f(|J|)$ such that:*

1. *for every edge $xy \in E(T)$, the adhesion $\mathsf{bag}(x) \cap \mathsf{bag}(y)$ induces a clique in G', in particular, for every node $x \in V(T)$, the torso of x is exactly $G'[\mathsf{bag}(x)]$;*
2. *for every node $x \in V(T)$, the algorithm computes a set $A_x \subseteq \mathsf{bag}(x)$ with $|A_x| \leqslant k$ and a graph P_x containing $G'[\mathsf{bag}(x)] - A_x$, together with an explicit no-apex k-almost-embedding of P_x in a surface of Euler genus at most k.*

Moreover, the decomposition can be required to be neat. The running time is $f(|J|) \cdot |G|^2$.

This follows from the structure theorem [55, Theorem 3.2] and its algorithmic version [55, Theorem 6.1 and Section 7]. The algorithm also supplies explicit almost-embeddings of the graphs P_x; see [55, Proviso 3.1].

For nonplanar excluded graphs, the underlying existence result goes back to Robertson and Seymour [75]. The additional neatness condition follows from the standard regularization with clique adhesions; see [6, Lemma 2.8]. Each new bag is contained in an old one, so we restrict the corresponding apex set and retain the old no-apex almost-embedding as the containing graph P_x.

Let $k \in \mathbb{N}$. An almost-embedding with apex set A is *strong* if every neighbor outside A of an apex vertex belongs to one of the attached pieces of the almost-embedding. Equivalently, apex vertices are adjacent only to other apex vertices and to vertices belonging to the attached pieces. A graph is *strongly k-almost embeddable* in a surface Σ if it admits a strong k-almost-embedding in Σ. It is *strongly k-almost embeddable* if this holds for some surface of Euler genus at most k.

Let G_1 and G_2 be graphs and let $K_1 \subseteq G_1$ and $K_2 \subseteq G_2$ be cliques of the same size. A *clique-sum* of G_1 and G_2 is any graph obtained by identifying pairs of vertices in K_1 and K_2 and optionally deleting some edges of the identified clique. A particular case is when K_1 and K_2 are single vertices v_1 and v_2 of G_1 and G_2, in which case the clique-sum consists of gluing G_1 and G_2 by identifying v_1 and v_2. Such a clique-sum is called a *1-clique-sum*.

More generally, a graph is obtained by *clique-sums* from a family of graphs if it can be constructed by repeatedly applying the clique-sum operation.

A graph G is an *apex graph* if there exists a vertex $v \in V(G)$ such that $G - v$ is planar. The following theorem of Dujmović, Morin, and Wood characterizes proper minor-closed classes of graphs excluding an apex graph.

Theorem 3.15 (Dujmović-Morin-Wood [36])**.** *Let $\mathscr{C}$ be a proper minor-closed class of graphs. Then the following are equivalent:*

1. *$\mathscr{C}$ excludes an apex graph as a minor;*
2. *there exists $k \in \mathbb{N}$ such that every graph in $\mathscr{C}$ can be constructed by clique-sums of strongly k-almost embeddable graphs;*

We use the following no-apex consequence of this characterization; see [11, Theorem 2.2].

Corollary 3.16 ([11, Theorem 2.2])**.** *Let $\mathscr{C}$ be a class of graphs and let $k \in \mathbb{N}$. If every maximal 2-connected subgraph with at least three vertices of every graph in $\mathscr{C}$ is a subgraph of a no-apex k-almost embeddable graph, then $\mathscr{C}$ excludes an apex graph as a minor.*

The following result for classes excluding an apex graph was proved by Milolidakis [67]. Its polynomial-time construction also uses the strengthened structure theorem of Dvořák and Thomas [38].

Theorem 3.17 ([67, Theorem 18] and [38])**.** *Let $\mathscr{C}$ be a class of graphs excluding an apex graph as a minor. Then there exists a class $\mathscr{C}'$ of subcubic graphs excluding an apex graph as a minor such that every graph $G \in \mathscr{C}$ is a minor of some graph $H \in \mathscr{C}'$.*
Furthermore, given $G \in \mathscr{C}$, such a graph $H \in \mathscr{C}'$ and a corresponding surjective minor model can be computed in polynomial time[a]*.*

[a] The polynomial-time claim uses the algorithmic structure theorem in [38]. For the uniform form used below, suppose that G is connected and, for each block with at least three vertices, an explicit no-apex k-almost-embedding of a containing graph is supplied. Then H can be chosen connected with $h(H) \leqslant q(k)$, and H and the minor-model map can be computed in time $p(k)N^{c_0}$, where p, q are computable functions, c_0 is an absolute constant, and N is the total encoding size of G and the supplied witnesses.

4. Rearranging tree orders

We present two useful general transductions allowing to rearrange tree orders.

4.1. Rewiring a tree order

We introduce a small transduction that will be used several times below to modify a tree order locally. The intuitive operation is the following. We mark some vertices as roots of subtrees that should no longer stay attached to their old parents. In each relevant part of the tree, we also mark one attachment vertex. The operation then cuts every marked root from its old parent and attaches it as a new child of the corresponding attachment vertex.

Let $\mathbf{X} = (X, \preceq)$ be a tree-ordered set. We use the unary marks S, A, R_{old} with the following intended meaning. The vertices marked by S are called *stop vertices*. They determine the parts of the tree in which rewiring is performed. Formally, we require that S is an antichain. For $s \in S$, we write $U_s := \{x \in X : s \preceq x\}$. Note that the sets U_s are pairwise-disjoint. We call them the *universes* of the rewiring.

The vertices marked by A are called *attachment vertices*. We require that every universe U_s contains exactly one vertex marked by A. We denote this vertex by a_s.

The vertices marked by R_{old} are the old roots of the pieces that will be moved. We require that every $r \in R_{\mathrm{old}}$ belongs to a unique universe U_s and that $\neg(r \preceq a_s)$. In particular, $r \neq a_s$, the vertex r is not a root of the original tree, and a_s does not belong to the piece rooted at r. Thus, deleting the cover edge from the parent of r to r and adding the edge $a_s r$ is well defined and creates no cycle.

The rewired tree is obtained from the cover tree of $\preceq$ as follows. For each $r \in R_{\mathrm{old}} \cap U_s$, delete the edge between r and its old parent and add the edge between a_s and r. All other cover edges are kept. Note that the vertices marked by R_{old} need not form an antichain. If two marked vertices lie on the same root-to-leaf path, then both are cut from their old parents and both are attached separately below the same attachment vertex. Thus, the lower marked vertex is no longer considered part of the piece moved with the higher marked vertex.

A tree-ordered set obtained in this way from $\mathbf{X}$ is called a *rewiring* of $\mathbf{X}$.

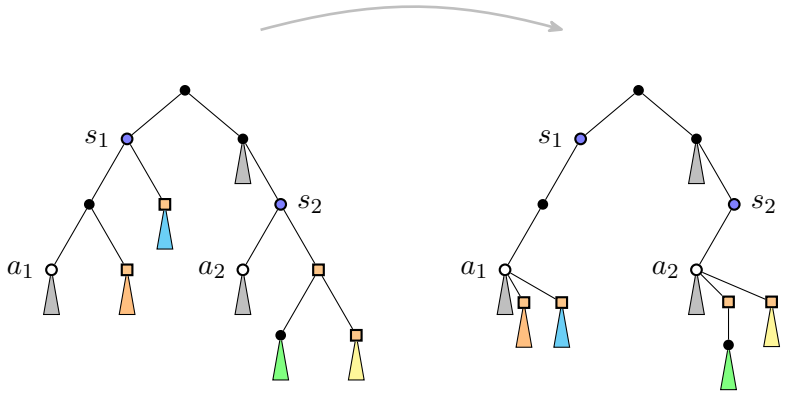


Figure 7: Rewiring some subtrees. The vertices s_1, s_2 are the stop vertices and the vertices a_1, a_2 are the corresponding attachment vertices. The vertices R_{old} are the four square vertices.

Lemma 4.1. *There is a transduction* Rewire *such that for every tree-ordered set* $\mathbf{X}$, Rewire($\mathbf{X}$) *contains all the rewirings of* $\mathbf{X}$.

Proof. The transduction guesses the unary predicates S, A, R_{old}. We first note some easily checked definability results:

- for an element u belonging to some universe, the unique stop vertex s such that $u \in U_s$ is definable;
- for every stop element s, the unique attachment vertex $a \in U_s$ is definable;
- for every element u marked R_{old}, the following set is definable: the set X_u of all vertices v with $u \preceq v$ such that no vertex $u' \neq u$ marked R_{old} satisfies $u \prec u' \preceq v$. These are the vertices below u that shall be moved with u.

The formula $\rho_{\preceq}(x, y)$ of the interpretation is now defined depending on several definable cases:

- If x and y are not in the same universe, then $\rho_{\preceq}(x, y) := (x \preceq y)$.
- Otherwise, let s be the stop vertex with $x, y \in U_s$ and let a be the only attachment vertex in U_s.
  - If x and y are in the same set X_u, then $\rho_{\preceq}(x, y) := (x \preceq y)$.
  - If x and y are in different sets ($x \in X_u$ and $y \in X_v$), then $\rho_{\preceq}(x, y) := \bot$.
  - If x is in a set X_u and y is not in any set X_v, then $\rho_{\preceq}(x, y) := \bot$.
  - If x is not in any set X_u and y is in a set X_v, then $\rho_{\preceq}(x, y) := (x \preceq a)$.
  - If neither x nor y are in any set X_u, then $\rho_{\preceq}(x, y) := (x \preceq y)$. □

4.2. Re-rooting a tree order

We next define a transduction that allows us to re-root selected subtrees of a tree order.

The intuitive operation is the following. We mark an antichain of vertices by a unary predicate R. These vertices are the old roots of the subtrees that we want to modify. For every such old root r, we also mark exactly one descendant n of r by a unary predicate N. The subtree rooted at r is then re-rooted at n: the path from r to n is reversed, all branches hanging from this path stay attached to the same vertex as before, and the whole subtree stays attached to the rest of the tree

at the same place; see Figure 8. The corresponding operation on (associated) tree orders is called a *re-rooting*.

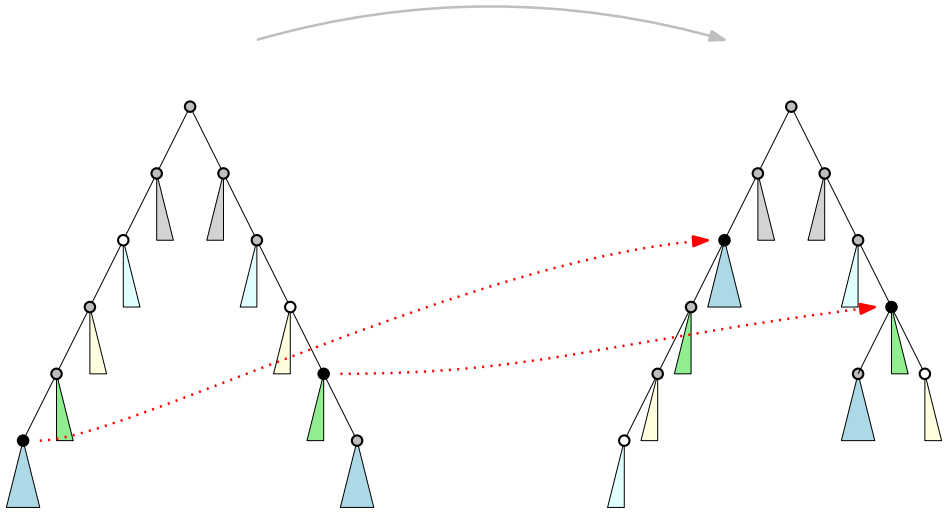


Figure 8: Re-rooting some subtrees. Old roots marked with R are white, new roots marked with N are black.

Lemma 4.2. *There is a transduction* T *such that every re-rooting of a tree-ordered set* $\mathbf{X}$ *belongs to* $\mathsf{T}(\mathbf{X})$.

Proof. The transduction guesses the unary predicates R and N. We then define, by a first-order interpretation, the new tree order.

Let Y be the original rooted tree and let Y^{new} be the new one. For a vertex v, denote by P_v (resp. P_v^{new}) the path from the root to v in Y (resp. in Y^{new}).

Then it is easily checked that

- if P_v does not contain a vertex marked R, then $P_v^{\text{new}} = P_v$;
- if P_v contains a (unique) vertex r_v marked R and if n_v is the (unique) descendant of r_v marked N, then (see Figure 9)

$$V(P_v^{\text{new}}) = V(P_v) \,\Delta\, \{z \colon (r_v \preceq z \preceq n_v) \wedge (z \neq (v \curlywedge n_v))\}.$$

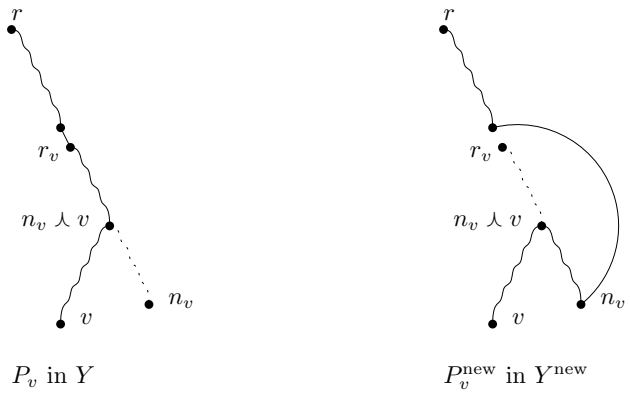


Figure 9: How the path from the root to v changes when re-rooting the tree order (case where P_v contains a vertex marked R).

We first define the auxiliary formula

$$\zeta(x, y, z, z') := R(z) \wedge N(z') \wedge (z \preceq x \preceq z') \wedge (z \preceq y),$$

which expresses that x and y are in the same subtree with root z marked R and x is on the path linking the root to the vertex z' marked N in this subtree. (Hence, $z = r_y$ and $z' = n_y$.)

It follows from the above case analysis that if there exist no z, z' with $\zeta(x, y, z, z')$, then $x \preceq y$ holds in the re-rooted tree order if and only if it holds in the original one. If there exist z, z' with $\zeta(x, y, z, z')$, then necessarily $z = r_y$ and $z' = n_y$, and $x \preceq y$ holds in the re-rooted tree order if and only if $y \curlywedge n_y \preceq x$ holds in the original tree order. Consequently, the new tree order is defined by

$$\rho_{\preceq}(x, y) := \big(\neg\exists z, z'\, \zeta(x, y, z, z') \wedge x \preceq y\big) \vee \big(\exists z, z'\, (\zeta(x, y, z, z') \wedge (y \curlywedge z' \preceq x))\big).$$

It is easily checked that the transduction $\mathsf{T} := (\{R, N\}, \mathsf{I})$ meets the requirements of the lemma. □

5. Proof of the main structure theorem

This section is devoted to the proof of the structure theorems, which we restate here.

Theorem 5.1. *Let $\mathscr{C}$ be a weakly sparse class of graphs. Then the following are equivalent:*
1. *$\mathscr{C}$ excludes some graph as a minor;*
2. *$\mathscr{C}^{\Upsilon}$ has bounded twin-width;*
3. *$\mathscr{C}^{F}$ has bounded twin-width.*

Moreover, if these conditions hold, an expansion of $\mathscr{C}^{F}$ by a linear order with bounded twin-width can be computed in polynomial time.

Theorem 5.2. *There exist computable functions ξ, f, a constant c and an algorithm that, for a given graph G and a rooted spanning forest F of G, computes in time $f(h(G)) \cdot |G|^c$ a linear order $<$ on G, such that $(G, \preceq_F, <)$ has twin-width at most $\xi(h(G))$.*

By Remark 3.1, it suffices to establish the following statement for connected graphs and spanning trees.

Theorem 5.3. *Let $\sigma = \{E, \preceq, <\}$ be the signature obtained by expanding the signature $\{E\}$ of graphs with two binary relations $\preceq$ and $<$. There exist computable functions ξ, f, a constant c, and an algorithm that, for a given connected graph G and a rooted spanning tree Y of G, computes in time $f(h(G)) \cdot |G|^c$ a σ-structure $\mathbf{G}$ such that*

- *the $\{E\}$-reduct of $\mathbf{G}$ is G,*
- *the $\preceq$-reduct of $\mathbf{G}$ is the tree order $\preceq_Y$,*
- *the $<$-reduct of $\mathbf{G}$ is a linear order,*
- *the twin-width of $\mathbf{G}$ is at most $\xi(h(G))$.*

Proof that Theorem 5.3 implies Theorem 5.1 and Theorem 5.2. Let ξ_0, f_0, and c_0 be as provided by Theorem 5.3, and let G be a graph equipped with a rooted spanning forest F. Apply Remark 3.1 to obtain a connected graph $\widehat{G} = G \oplus K_1$ with a rooted spanning tree Y. Thus, $h(\widehat{G}) \leqslant h(G)+1$, $|\widehat{G}| = |G|+1$, and restricting $\preceq_Y$ to $V(G)$ gives $\preceq_F$. Apply Theorem 5.3 to $(\widehat{G}, Y)$ and restrict the resulting linear order to $V(G)$. Since twin-width does not increase when passing to an induced substructure, the resulting structure $(G, \preceq_F, <)$ has twin-width at most $\xi_0(h(\widehat{G}))$. After increasing f_0 and c_0 if necessary to include the construction and restriction steps, define

$$\xi(k) = \max_{0 \leqslant i \leqslant k+1} \xi_0(i), \qquad f(k) = 2^{c_0} \max_{0 \leqslant i \leqslant k+1} f_0(i).$$

Then $\operatorname{tww}(G, \preceq_F, <) \leqslant \xi(h(G))$. Moreover, $(|G|+1)^{c_0} \leqslant 2^{c_0}|G|^{c_0}$, so the running time is at most $f(h(G)) \cdot |G|^{c_0}$. This proves Theorem 5.2.

We now prove Theorem 5.1. The implication from (2) to (1) follows from Theorem 1.2. For the converse, suppose that $\mathscr{C}$ excludes a graph H as a minor, and put $t = |V(H)|$. Then $h(G) < t$ for every $G \in \mathscr{C}$. By Theorem 5.2, for every $G \in \mathscr{C}$ and every rooted spanning forest F of G, there is a linear order $<$ such that $\operatorname{tww}(G, \preceq_F, <) \leqslant \xi(t-1)$. Finally, twin-width does not increase when relations are removed. Hence, $\operatorname{tww}(G, \preceq_F) \leqslant \operatorname{tww}(G, \preceq_F, <) \leqslant \xi(t-1)$. Thus, $\mathscr{C}^F$ has bounded twin-width, proving (1) implies (3). The implication from (3) to (2) follows from $\mathscr{C}^\Upsilon \subseteq \mathscr{C}^F$. □

Theorem 5.2 extends to binary structures.

Theorem 5.4. *Fix a finite relational signature σ of arity at most two. There exist computable functions f_σ, ξ_σ, an absolute constant c, and an algorithm that, given a σ-structure $\mathbf{M}$ and a rooted spanning forest F of $G := \operatorname{Gaif}(\mathbf{M})$, computes in time $f_\sigma(h(G)) \cdot \|\mathbf{M}\|^c$ a linear order $<$ such that $(\mathbf{M}, \preceq_F, <)$ has twin-width at most $\xi_\sigma(h(G))$.*

Proof. Apply Theorem 5.2 to $G := \operatorname{Gaif}(\mathbf{M})$ and F. We recover all relations of $\mathbf{M}$ from a suitable unary expansion of G by first-order formulas, while retaining both orders unchanged. This is the expansion construction of Lemma 3.3, together with Fact 3.2. In its effective form, the number of unary predicates and the defining formulas depend computably only on $h(G)$ and σ, and the required predicates can be computed within the stated running time. Preservation of bounded twin-width under first-order transductions then gives the required twin-width bound. □

As FO model checking is known to be fixed-parameter tractable on monadically dependent classes of linearly ordered binary structures (Theorem 3.8), we obtain the following corollary.

Corollary 5.5. *Fix a finite relational signature σ of arity at most two, and let $\preceq$ be a fresh binary relation symbol. There are a computable function f and an absolute constant c such that the following holds. Given a finite σ-structure $\mathbf{M}$, a rooted spanning forest F of $\mathrm{Gaif}(\mathbf{M})$, and an FO-sentence φ over $\sigma \cup \{\preceq\}$, one can decide whether*

$$(\mathbf{M}, \preceq_F) \models \varphi$$

in time

$$f\big(|\varphi|, h(\mathrm{Gaif}(\mathbf{M}))\big) \cdot \|\mathbf{M}\|^c.$$

As outlined in Section 2, the proof of Theorem 5.3 consists of 9 steps divided into 2 phases. For convenience, we reproduce the overview of the proof of Theorem 5.3 and the short description of the two phases given in Section 2.

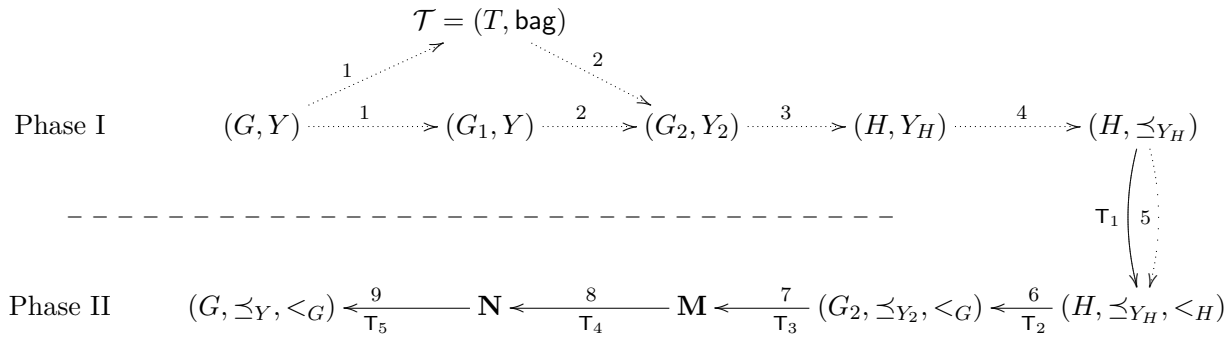


Figure 10: Outline of the proof of Theorem 5.3 (copy of Figure 1). Dotted arrows are polynomial-time computations and plain arrows are transductions. (Precisely, $G \xrightarrow[\mathsf{T}]{1} H$ means that at step 1 of the proof, we prove $H \in \mathsf{T}(G)$.) The main steps of the proof are: **1.** construction of a neat tree decomposition of G based on the structure theorem of Robertson and Seymour, as well as an augmentation of G so that every adhesion is a clique; **2.** construction of a subgraph whose nontrivial blocks have explicit no-apex almost-embedding witnesses; **3.** effective blockwise translation into a subcubic graph of bounded Hadwiger number; **4.** computation of the tree-ordered expansion; **5.** computation of a linear order that can be obtained by transduction T_1; **6.** transduction of the linearly ordered tree-ordered expansion of G_2 by T_2; **7.** transduction of a master structure that permits the graph to be handled margin-wise by T_3; **8.** recovery of the target tree order on the margins by transduction T_4; **9.** full recovery of the tree order by transduction T_5.

- **Phase I.** Given a connected graph $G \in \mathscr{H}_t$ with a rooted spanning tree Y, we construct in polynomial time a tree-ordered graph $(H, \preceq_{Y_H})$ with a linearly ordered expansion $(H, \preceq_{Y_H}, <_H)$ of $(H, \preceq_{Y_H})$ such that the class

$$\mathscr{M}_t = \{(H, \preceq_{Y_H}, <_H) \mid G \in \mathscr{H}_t, Y \text{ is a rooted spanning tree of } G\}$$

  is monadically dependent, and hence, as a linearly ordered class, has bounded twin-width.
- **Phase II.** From $(H, \preceq_{Y_H}, <_H)$ we transduce a linearly ordered expansion $(G, \preceq_Y, <_G)$ of the original tree-ordered graph. The order $<_G$ is inherited from the order $<_H$.

5.1. Phase I

Step 1. We first compute the Hadwiger number $h(G)$ of G in time $f(h(G)) \cdot |G|^{1+o(1)}$ using the minor containment testing algorithm given in [60], where f is some computable function.

We now set $J = K_{h(G)+1}$ and call the algorithm of Theorem 3.14. For this choice of J, the algorithm outputs a neat tree decomposition $\mathcal{T} = (T, \mathsf{bag})$ of a supergraph G_1 of G, together with apex sets A_x for all $x \in V(T)$. Each adhesion induces a clique in G_1, and $G_1[\mathsf{bag}(x)] - A_x$ is contained in a graph P_x supplied with an explicit no-apex k-almost-embedding, where k is the parameter from Theorem 3.14.

We shall need later that the supergraph G_1 still belongs to a minor-excluded class whose excluded minor depends only on $h(G)$. Every torso at a node x is contained in the k-almost embeddable graph obtained from P_x by adding A_x as apices and all their incident edges in the torso. Hence, every torso has Hadwiger number bounded by a computable function of k, see [59, Theorem 3.1]. Moreover, G_1 is obtained from its torsos by clique-sums, because all adhesions are cliques. Since the Hadwiger number of a clique-sum is at most the maximum of the Hadwiger numbers of its summands, see [59, Lemma 2.3], it follows that $h(G_1) \leqslant q_1(h(G))$ for a computable function q_1.

Our next goal is to define the vertices $\alpha(x)$ and $\beta(x)$ for all nodes x of the decomposition, which will represent these nodes and their connections. For this, we compute an acyclic orientation of G_1 as follows. Recall that a graph is *d-degenerate* if every non-empty subgraph contains a vertex of degree at most d. Equivalently, its vertices admit a degeneracy ordering, and such an ordering yields an acyclic orientation of the graph with maximum indegree at most d. Moreover, if d is fixed, such an orientation can be computed in linear time by a standard greedy algorithm.

Let us bound the degeneracy inside the margins. For every integer q, there is a (computable) number $d_{\mathrm{ae}}(q)$ such that every graph that is q-almost embeddable in a surface of Euler genus at most q is $d_{\mathrm{ae}}(q)$-degenerate. Indeed, as mentioned above, such graphs exclude a clique minor whose order is bounded by a function of q. A theorem of Kostochka and Thomason bounds the average degree of K_t-minor-free graphs [87], which hence bounds the degeneracy of these graphs. Since every torso is a subgraph of a k-almost embeddable graph as above, it is d_1-degenerate for $d_1 := d_{\mathrm{ae}}(k)$.

Since the graph induced by a margin is a subgraph of the corresponding torso, every margin $\mathsf{mrg}(x)$ induces a d_1-degenerate graph in G_1.

We now orient the edges of G_1. For every node $x \in V(T)$, we compute a degeneracy ordering of the graph $G_1[\mathsf{mrg}(x)]$ and orient all edges inside $\mathsf{mrg}(x)$ according to this ordering, so that every vertex has at most d_1 incoming edges from its own margin.

It remains to orient the edges whose endpoints lie in different margins. Let $u \in \mathsf{mrg}(x)$ and $v \in \mathsf{mrg}(y)$, where $x \neq y$. Since $\mathcal{T}$ is a tree decomposition, the nodes x and y are comparable in the rooted tree order of T whenever uv is an edge of G_1. If $x \prec_T y$, then we orient the edge from u to v; otherwise we orient it from v to u. In other words, all cross-margin edges are oriented down the decomposition tree.

The resulting orientation $\vec{G}_1$ is acyclic. Indeed, inside each margin the orientation is acyclic by construction, while every edge between two different margins goes strictly down the tree order on T. Hence, no directed cycle can use a cross-margin edge, and no directed cycle can be contained in a single margin.

Finally, we bound the indegree. A vertex has at most d_1 incoming edges from its own margin. Its remaining incoming edges come from ancestor margins. By the properties of tree decompositions, all such ancestors that are adjacent to a vertex in $\mathsf{mrg}(x)$ must lie in the adhesion through which the bag x is attached to its parent. This adhesion has size at most k. Thus, every vertex has at most $d := d_1 + k$ incoming edges in $\vec{G}_1$. For future reference, partition the arcs of $\vec{G}_1$ into binary relations $E_1, \ldots, E_d$ so that every vertex has at most one incoming arc in each E_i. Such a partition is obtained by assigning distinct indices to the incoming arcs at every vertex.

We now define vertices $\alpha(x)$ and $\beta(x)$ for every non-root node $x \in V(T)$. Let x be a non-root node and let $S_x := \mathsf{adh}(x) \cap \mathsf{mrg}(\mathsf{parent}(x))$. By neatness, S_x is non-empty.

Since $\mathsf{adh}(x)$ is a clique in G_1 and the orientation of G_1 is acyclic, the orientation induced

on $\mathsf{adh}(x)$ is a transitive tournament and therefore has a unique sink. Every vertex of $\mathsf{adh}(x) \setminus S_x$ belongs to the margin of a strict ancestor of $\mathsf{parent}(x)$. Hence every edge between $\mathsf{adh}(x) \setminus S_x$ and S_x is oriented towards S_x. Consequently, no vertex of $\mathsf{adh}(x) \setminus S_x$ can be the sink of the tournament induced on $\mathsf{adh}(x)$. We define $\alpha(x)$ to be this unique sink, thus $\alpha(x) \in \mathsf{adh}(x) \cap \mathsf{mrg}(\mathsf{parent}(x))$.

By neatness, every vertex of $\mathsf{adh}(x)$ has a neighbor in $\mathsf{mrg}(x)$. We choose such a neighbor of $\alpha(x)$ and denote it by $\beta(x)$. For the root r of T, we choose $\beta(r)$ arbitrarily in $\mathsf{mrg}(r)$.

As G is a spanning subgraph of G_1, the tree Y is also a spanning tree of G_1.

For every node $x \in V(T)$, let Y_x be the rooted forest whose order is the restriction of $\preceq_Y$ to $\mathsf{mrg}(x)$. We call Y_x the *trace of Y on the margin* $\mathsf{mrg}(x)$. Notice that a trace edge need not be an edge of G_1: it may shortcut a path of Y whose internal vertices lie outside $\mathsf{mrg}(x)$. All trace forests can be computed in linear time by traversing Y depth-first and maintaining, for each margin, its most recent vertex on the current root-to-vertex path.

Step 2. Our next goal is to delete edges (while ensuring that they can later be recovered by a transduction) so that every non-trivial block of the resulting graph is contained in a graph with an explicit no-apex almost-embedding witness, such that we can apply Corollary 3.16. This will yield the graph G_2.

Let G_0 be the graph obtained from G_1 by first deleting all edges between different margins, except the edges $\alpha(x)\beta(x)$, and then, for every $x \in V(T)$, deleting all edges with one endpoint in $\mathsf{mrg}(x)$ and the other one in $A_x \cap \mathsf{mrg}(x)$.

Fix a node $x \in V(T)$ and contract each connected component of $G_0[\mathsf{mrg}(x)]$ to one vertex. Let Q_x denote the resulting quotient of $G_1[\mathsf{mrg}(x)]$. By neatness, $G_1[\mathsf{mrg}(x)]$ is connected, so Q_x is connected. Moreover, every apex in $A_x \cap \mathsf{mrg}(x)$ is an isolated vertex of $G_0[\mathsf{mrg}(x)]$ and therefore gives a singleton quotient vertex.

Next, choose a spanning tree of Q_x. For every edge of this tree, choose an arbitrary corresponding edge of $E(G_1[\mathsf{mrg}(x)]) \setminus E(G_0[\mathsf{mrg}(x)])$. Let F_x be the set of chosen edges. Then $G_0[\mathsf{mrg}(x)]+F_x$ is connected and every edge of F_x is a bridge[3] in $G_0[\mathsf{mrg}(x)] + F_x$. Indeed, after contracting the connected components of $G_0[\mathsf{mrg}(x)]$, the graph $G_0[\mathsf{mrg}(x)] + F_x$ becomes precisely the chosen spanning tree of Q_x. See Figure 11.

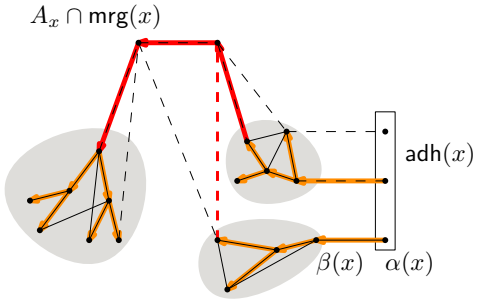


Figure 11: An illustration of chosen edges F_x (in red). The picture illustrates the graph $G_1[\mathsf{mrg}(x)]$. The dashed edges are the ones that are deleted from G_1 to get G_0. The connected components of $G_0[\mathsf{mrg}(x)]$ that are not apices correspond to the three marked regions. The edges of Y are highlighted and oriented away from the root of Y. The distinguished edges of Y (red arcs), together with the distinguished dashed edge (also in red), form the set F_x of chosen edges.

We call the edges in $\bigcup_{x \in V(T)} F_x$ the *apex connecting edges*. Let G_2 be the graph obtained from G_0 by adding all apex connecting edges.

[3] A *bridge* is an edge whose removal increases the number of connected components of the graph.

We prove the following auxiliary lemma.

Lemma 5.6. *Let G be a connected graph and let $X \subseteq V(G)$. Let G' be the graph obtained from G by deleting all edges with at least one endpoint in X. Let $F \subseteq E(G) \setminus E(G')$ be inclusion-wise minimal such that $G'' := G' + F$ is connected. Then the maximal 2-connected subgraphs of G'' with at least three vertices are exactly the maximal 2-connected subgraphs of G' with at least three vertices.*

Proof. By the inclusion-wise minimality of F, every edge of F is a bridge of G''. Every edge of a 2-connected graph with at least three vertices lies on a cycle, so no such subgraph of G'' contains an edge of F. Thus, G' and G'' have exactly the same 2-connected subgraphs with at least three vertices, and hence the same maximal ones. □

We next show that every 2-connected subgraph of G_2 with at least three vertices is contained in a single margin. Each graph $G_2[\mathsf{mrg}(x)]$ is connected by the construction of F_x. The only edges between distinct margins are the edges $\alpha(x)\beta(x)$ for non-root nodes x, each joining $\mathsf{mrg}(x)$ to $\mathsf{mrg}(\mathsf{parent}(x))$. Contracting all margins therefore gives exactly T. It follows that every cross-margin edge is a bridge of G_2, so no 2-connected subgraph with at least three vertices can use such an edge.

Now fix $x \in V(T)$. We apply Lemma 5.6 to the connected graph $G_1[\mathsf{mrg}(x)]$, with $X = A_x \cap \mathsf{mrg}(x)$, $G' = G_0[\mathsf{mrg}(x)]$, and $F = F_x$. The graph $G_0[\mathsf{mrg}(x)]$ is obtained from $G_1[\mathsf{mrg}(x)]$ by deleting precisely the edges with at least one endpoint in $A_x \cap \mathsf{mrg}(x)$. Moreover, F_x is inclusion-wise minimal subject to $G_0[\mathsf{mrg}(x)] + F_x$ being connected, since every edge of F_x is a bridge of this graph. As $G_2[\mathsf{mrg}(x)] = G_0[\mathsf{mrg}(x)] + F_x$, Lemma 5.6 yields that the maximal 2-connected subgraphs of $G_2[\mathsf{mrg}(x)]$ with at least three vertices are exactly the maximal 2-connected subgraphs of $G_0[\mathsf{mrg}(x)]$ with at least three vertices.

Every vertex of $A_x \cap \mathsf{mrg}(x)$ is isolated in $G_0[\mathsf{mrg}(x)]$. Therefore, every maximal 2-connected subgraph of $G_2[\mathsf{mrg}(x)]$ with at least three vertices is contained in $G_1[\mathsf{mrg}(x) \setminus A_x] \subseteq P_x$, for which Step 1 supplied an explicit no-apex k-almost-embedding. Since every such block of G_2 is contained in a single margin, Corollary 3.16 yields the following corollary.

Corollary 5.7. *The graph G_2 excludes some apex graph $\Gamma_{h(G)}$ as a minor.*

Proof. As shown above, every maximal 2-connected subgraph of G_2 with at least three vertices is contained in a graph with a no-apex k-almost-embedding. The remaining blocks consist of single vertices or single edges. Thus, Corollary 3.16 applies to the class of all graphs G_2 obtained for a fixed value of $h(G)$, and the assertion follows. □

We now define the rooted spanning tree Y_2. For every $x \in V(T)$, put

$$Y_x^0 := Y_x \cap G_0[\mathsf{mrg}(x)].$$

If C is a connected component of $G_0[\mathsf{mrg}(x)]$, then $Y_x^0[C]$ is a forest. Extend it to a spanning tree of C, and let $F_{Y,x}$ be the set of edges added in all these extensions. We call them the *spanning-tree connecting edges*. The tree Y_2 consists of all the resulting component trees, all edges in F_x for every $x \in V(T)$, and all edges $\alpha(x)\beta(x)$ for non-root nodes $x \in V(T)$. We root Y_2 at $\beta(x_0)$, where x_0 is the root of T.

Lemma 5.8. *Y_2 is a rooted spanning tree of G_2.*

Proof. Fix $x \in V(T)$. Inside each connected component of $G_0[\mathsf{mrg}(x)]$, we chose a spanning tree. By the construction of F_x, the graph obtained from F_x after contracting these components is a tree. Consequently, $Y_2[\mathsf{mrg}(x)]$ is connected and acyclic, and hence is a spanning tree of the margin.

Finally, contract every tree $Y_2[\mathsf{mrg}(x)]$ to a single vertex. The remaining edges are precisely the edges $\alpha(x)\beta(x)$ connecting every non-root margin to its parent margin, so the resulting quotient is T. It follows that Y_2 is connected and that no cycle can use edges between distinct margins. Since there is no cycle inside a margin either, Y_2 is a rooted spanning tree of G_2. □

Step 3. Every block of G_2 with at least three vertices is contained in a graph P_x supplied in Step 1 with an explicit no-apex k-almost-embedding. The other blocks have trivial such witnesses. Applying Theorem 3.17 to these witnesses computes a connected subcubic graph H and a surjective minor-model map $f\colon V(H) \to V(G_2)$ such that $h(H) \leqslant \xi_c(h(G))$ for a computable function ξ_c. This computation takes time $f_1(h(G)) \cdot |G|^{c_1}$ for a computable function f_1 and an absolute constant c_1.

We now delete every edge $ab \in E(H)$ with $f(a) \neq f(b)$ and $f(a)f(b) \notin E(G_2)$. The graph H remains connected and subcubic, its Hadwiger number does not increase, and f remains a surjective minor-model map. Contracting the branch sets now gives exactly G_2. We now construct a spanning tree Y_H of H as follows: For every $v \in V(G_2)$, choose a spanning tree of $H[f^{-1}(v)]$. For every edge $uv \in E(Y_2)$, add one edge of H between $f^{-1}(u)$ and $f^{-1}(v)$. Their union Y_H spans H, because the branch sets partition $V(H)$. Contracting the chosen branch-set trees turns Y_H into Y_2. Hence, Y_H is connected and acyclic, and is a spanning tree of H.

Step 4. We compute the expansion $(H, \preceq_{Y_H})$ of H by the tree order $\preceq_{Y_H}$ defined by Y_H, rooted at an arbitrary vertex of $f^{-1}(\beta(x_0))$, where x_0 is the root of T. Contracting this branch set sends the root to $\beta(x_0)$, which is the root of Y_2, hence the two root choices are compatible.

Step 5. A linear ordering $<_H$ of $V(H)$ is computed from a DFS-traversal of Y_H. This linear order can be obtained as a transduction of $(H, \preceq_{Y_H})$, as was stated without a proof in [20]. We give the proof for completeness.

Lemma 5.9 ([20]). *Let $\mathscr{C}$ be a class of connected graphs of maximum degree at most a fixed integer D. Then, for every $G \in \mathscr{C}$ and every rooted spanning tree Y of G, every DFS preorder of Y can be transduced from $(G, \preceq_Y)$.*

Proof. Let $G \in \mathscr{C}$, let Y be a rooted spanning tree of G, and fix a DFS preorder of Y. Every vertex has at most D children. Using unary predicates $C_1, \ldots, C_D$, color the ith child of each vertex by C_i, where the children are ordered according to the prescribed DFS preorder. These colors can be computed in linear time from the corresponding ordered tree. For distinct vertices x, y that are comparable in $\preceq_Y$, put the ancestor first. Otherwise, let $z = x \curlywedge y$ and compare the two children of z leading to x and y by their colors. The least common ancestor, these children, and their color comparison are first-order definable from the tree order and the unary predicates. This defines exactly the prescribed DFS preorder. □

All computations in Steps 1-5 take time $f_2(h(G)) \cdot |G|^{c_2}$ for a computable function f_2 and an absolute constant c_2. Indeed, Steps 1 and 3 have bounds of this form, while the remaining computations are polynomial in the sizes of their inputs with an absolute exponent. In particular, the size of H is bounded by the running time of Step 3, and its tree order and DFS preorder can be computed in quadratic time in $|H|$. We are now ready to conclude Phase I of the proof of Theorem 5.3.

Corollary 5.10. *There is a computable function d such that the class*

$$\mathscr{M}_t = \{(H, \preceq_{Y_H}, <_H) \mid G \in \mathscr{H}_t, Y \text{ is a rooted spanning tree of } G\}$$

has twin-width at most $d(t)$.

Proof. Fix t and let $(H, \preceq_{Y_H}, <_H) \in \mathscr{M}_t$. By Step 3, H is subcubic and $h(H) \leqslant \xi_c(t)$, where we may take ξ_c to be nondecreasing.

Observe that the graph reduct of every tree-ordered minor of $(H, \preceq_{Y_H})$ is an ordinary minor of H. Consequently, all these graph reducts exclude $K_{r,r}$ as a subgraph for $r := \xi_c(t) + 1$. As Y_H has maximum degree at most 3, Lemma 3.11 gives $\operatorname{tww}(H, \preceq_{Y_H}) \leqslant \beta(3, r)$. By Lemma 5.9, the order $<_H$ is obtainable by a fixed transduction. Applying Lemma 3.9 therefore yields a computable bound $d(t)$ on $\operatorname{tww}(H, \preceq_{Y_H}, <_H)$. □

5.2. Phase II

The first phase of constructing the class $\mathscr{M}_t$ with bounded twin-width was algorithmic: starting from (G, Y), we constructed a subcubic graph H with a rooted spanning tree Y_H and a DFS order. The purpose of the second phase is to show that we can reverse this construction by transductions. Starting from $(H, \preceq_{Y_H}, <_H)$, we successively recover the tree-ordered and linearly ordered expansion $(G, \preceq_Y, <_G)$. Since $\mathscr{M}_t$ has bounded twin-width and transductions preserve bounded twin-width, this will imply that the class of all produced structures $(G, \preceq_Y, <_G)$ has bounded twin-width. Additionally, as we obtain a linearly ordered class of bounded twin-width, we can do efficient model checking. We continue with the first step of the second phase.

Step 6. The minor model of G_2 in H consists of pairwise-disjoint connected branch sets $f^{-1}(v)$, one for each $v \in V(G_2)$, which partition $V(H)$. Contracting these branch sets yields precisely G_2, since all edges between branch sets corresponding to nonadjacent vertices of G_2 have already been deleted.

For $v \in V(G_2)$, let $\iota(v)$ be the unique $\preceq_{Y_H}$-minimum of $f^{-1}(v)$. This minimum exists because the branch set induces a subtree of Y_H. Contracting each branch set to its representative $\iota(v)$ turns Y_H into Y_2, by the construction of Y_H, and yields a tree-ordered graph whose E-reduct is precisely G_2.

We describe the required transduction. Mark the representatives $\iota(v)$ by a unary predicate Q and put

$$\chi(a, u) := Q(a) \wedge a \preceq_{Y_H} u \wedge \neg\exists b \left(Q(b) \wedge a \prec_{Y_H} b \preceq_{Y_H} u\right).$$

Because the branch sets partition $V(H)$ and each induces a subtree of Y_H, the formula $\chi(\iota(v), u)$ holds exactly when $u \in f^{-1}(v)$. Use Q as the output domain and interpret adjacency by

$$\rho_E(a, b) := a \neq b \wedge \exists u \exists v \left(\chi(a, u) \wedge \chi(b, v) \wedge E(u, v)\right).$$

The tree order is simply restricted to Q. We inherit the linear order from the representatives, that is, we define the linear order $<_G$ of $V(G)$ (recall that $V(G) = V(G_2)$) so that for every two distinct vertices $u, v \in V(G)$,

$$u <_G v \quad \iff \quad \iota(u) <_H \iota(v).$$

Thus, $(G_2, \preceq_{Y_2}, <_G)$ is obtained from $(H, \preceq_{Y_H}, <_H)$ by a fixed transduction.

Consequently, the class of all expansions $(G_2, \preceq_{Y_2}, <_G)$ obtained in this way from pairs (G, Y) with $G \in \mathscr{H}_t$ and Y a rooted spanning tree of G can be transduced from $\mathscr{M}_t$.

Step 7. In the next step, we make the tree decomposition visible inside a transduced *master structure*, which we obtain by a copying transduction. It contains one copy of the vertices of G_2 and a second kind of elements representing the bags of the decomposition. The bag element corresponding to a node x is represented by a copy of the distinguished vertex $\beta(x)$. After rewiring, the vertices of the margin $\mathsf{mrg}(x)$ become exactly the descendants of the bag element representing x. This is similar to the augmented trees introduced in [73].

Formally, the *master structure* $\mathbf{M} = \mathrm{Dec}(G_2, \preceq_{Y_2}, <_G, \mathcal{T})$ defined by $(G_2, \preceq_{Y_2}, <_G)$ and the tree decomposition $\mathcal{T}$ has signature $\sigma_{\mathrm{mas}} = \{V, B, R, P_\beta, E, \preceq, \trianglelefteq, <, \mathsf{rep}\}$. Here V, B, R, P_β are unary predicates, and V, B, R partition the domain. The set $R(\mathbf{M})$ contains a single element denoted ω. The relation $\preceq$ is a tree order on the whole domain with root ω, $<$ is a linear order on $V(\mathbf{M})$, $\trianglelefteq$ is a tree order on $B(\mathbf{M})$, and rep is a binary representative relation. The structure is defined by bijections $\zeta_V : V(G_2) \to V(\mathbf{M})$ and $\zeta_B : V(T) \to B(\mathbf{M})$ satisfying the following conditions.

- ζ_V defines an isomorphism of the $\{E, <\}$-reduct of $(G_2, \preceq_{Y_2}, <_G)$ and the $\{E, <\}$-reduct of $\mathbf{M}[V(\mathbf{M})]$;
- the set of children of ω (with respect to $\preceq$) is $B(\mathbf{M})$;
- for every node $x \in V(T)$, the restriction of ζ_V to $\mathsf{mrg}(x)$ defines an isomorphism of $(\mathsf{mrg}(x), \preceq_{Y_2})$ and $(\{v \colon v \succ \zeta_B(x)\}, \preceq)$;
- the mapping ζ_B induces an isomorphism of $(T, \preceq_T)$ and $(B(\mathbf{M}), \trianglelefteq)$.
- P_β marks exactly the vertices $\zeta_V(\beta(x))$, and $\mathsf{rep}(\zeta_B(x), \zeta_V(\beta(x)))$ holds for every $x \in V(T)$ and for no other pair.

From this definition, it follows that, for all $x \in V(T)$ and $u, v \in V(G_2)$,

$$\begin{aligned} v \in \mathsf{mrg}(x) \quad &\iff \quad \mathbf{M} \models \zeta_B(x) \prec \zeta_V(v), \\ b(u) = b(v) \quad &\iff \quad \mathbf{M} \models \zeta_V(u) \curlywedge \zeta_V(v) \neq \omega. \end{aligned}$$

Lemma 5.11. *There is a transduction* T *such that* $\mathrm{Dec}(G_2, \preceq_{Y_2}, <_G, \mathcal{T}) \in \mathsf{T}((G_2, \preceq_{Y_2}, <_G))$.

Proof. We first apply a first transduction consisting of the composition of a Copy_3 operation (with equivalence relation between the clones denoted by $\simeq$) and the transduction defined as follows:

- The universe of the structure will consist of three parts: a part marked V corresponding to the domain of G_2 (this will be the first copy), a part marked B corresponding to the image of β (in the second copy and in bijection with the domain of T), and a global root marked R (third copy of the root of the original tree-order). Hence, the domain is defined by the formula
$$\nu(x) := V(x) \vee B(x) \vee R(x).$$
- The relation E is restricted to V:
$$\rho_E(x, y) := V(x) \wedge V(y) \wedge E(x, y).$$
- The linear order $<$ is restricted to V:
$$\rho_<(x, y) := V(x) \wedge V(y) \wedge (x < y).$$
- The representative marker and relation are interpreted by
$$\rho_{P_\beta}(x) := V(x) \wedge \exists y \, (B(y) \wedge x \simeq y)$$
and
$$\rho_{\mathsf{rep}}(x, y) := B(x) \wedge V(y) \wedge x \simeq y.$$

- A tree order on bags is defined as the restriction of the tree order $\preceq$ to the image of β (which is essentially the set marked B):

$$\rho_{\trianglelefteq}(x,y) := B(x) \wedge B(y) \wedge (x \preceq y).$$

- Finally, a global tree order is defined as follows: the vertex marked R is the root (hence smaller than all other vertices), and the copy marked B of a vertex is introduced as a predecessor of its copy marked V in the tree order inherited on V:

$$\rho_{\preceq}(x,y) := R(x) \vee (x = y) \vee \big(B(x) \wedge V(y) \wedge x \simeq y\big) \vee \exists x' \exists y' \left(x' \simeq x \wedge y' \simeq y \wedge x' \prec y'\right).$$

The mapping ζ_V maps v to its first copy, ζ_B maps x to the second copy of $\beta(x)$ and ω is the only vertex marked R.

Then, we apply the transduction Rewire from Lemma 4.1. Marking the root both S and A and marking all the vertices marked B by R_{old}, the rewiring detaches all the subtrees rooted at a vertex marked B and moves them as children of the root. This witnesses that our transduction produces the master structure.

The whole process is displayed on Figure 12. □

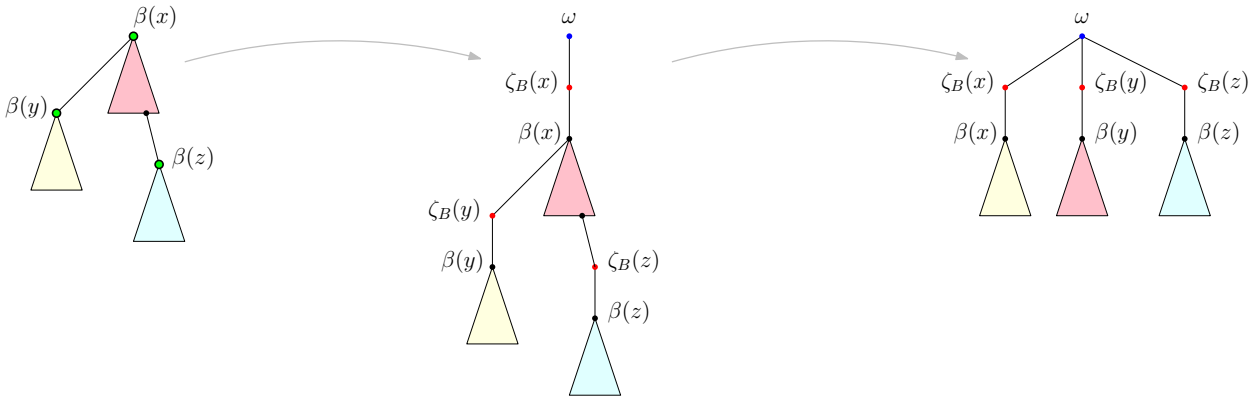


Figure 12: The master structure as a transduction. In the drawing, the graph G_2 is decomposed into three bags, x (in pink), y (in yellow), and z (in blue), with respective minimum vertices $\beta(x), \beta(y)$, and $\beta(z)$. These minimum vertices are duplicated as $\zeta_B(x), \zeta_B(y)$, and $\zeta_B(z)$ and these new vertices are moved as direct children of a super-root ω.

Step 8. At this point, the master structure still contains the simplified graph G_2 and the simplified tree order Y_2. We now undo the local simplifications inside each margin. We restore the edges of G_1 incident with the boundedly many apices and replace the auxiliary tree on each margin by the part of the trace forest that is carried by actual edges of G_1.

The structure $\mathbf{N}$ has the same signature and domain as $\mathbf{M}$. The predicates V, B, R, P_β, the relations $\trianglelefteq, <, \mathsf{rep}$, and the root ω are unchanged. We next explain how E and $\preceq$ change. For every $x \in V(T)$, put

$$Y_x^{\text{loc}} := Y_x \cap G_1[\mathsf{mrg}(x)]$$

and let $\preceq_x^{\text{loc}}$ be the forest-order induced by this forest. Inside $\mathsf{mrg}(x)$, the relation $\preceq$ of $\mathbf{N}$ is $\preceq_x^{\text{loc}}$, and every component root of this forest is a child of the bag element $\zeta_B(x)$. The order on the bag elements and the order between distinct margin universes are retained from $\mathbf{M}$. For vertices u, v marked V, set

$$\mathbf{N} \models E(u,v) \quad \Longleftrightarrow \quad \begin{cases} uv \in E(G_1), & \text{if } b(u) = b(v), \\ \{u,v\} = \{\alpha(x), \beta(x)\} \text{ for some non-root } x \in V(T), & \text{if } b(u) \neq b(v). \end{cases}$$

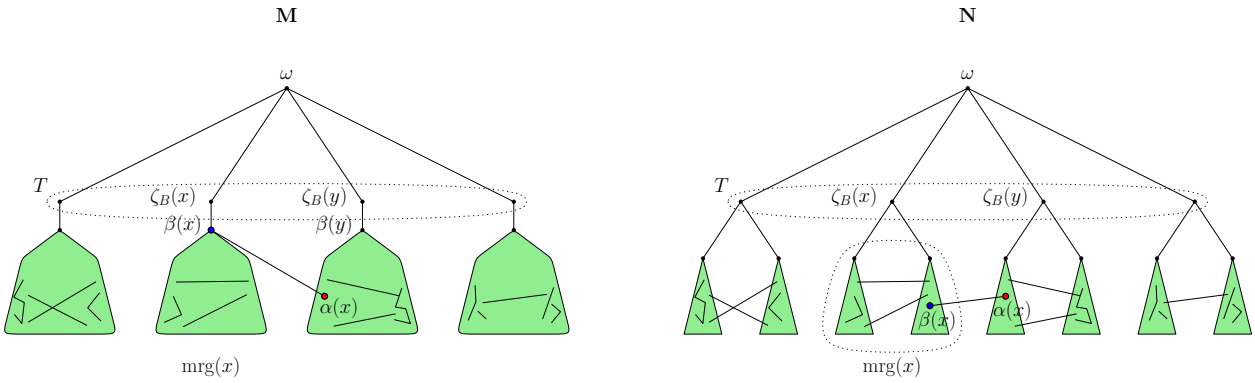


Figure 13: In **M** (left), the first level under the root ω contains a copy of the tree-decomposition order and, under each node x, the subtree rooted at $\beta(x)$ contains the vertices of $\mathsf{mrg}(x)$. In **N** (right), the order on each margin is the local trace order induced by Y_x^{loc}, and the edge relation inside each margin is the one induced by G_1.

Thus, the restriction of **N** to a margin carries the local trace order and the edge relation induced by G_1. Between distinct margins, the only graph edges retained at this stage are the distinguished attachment edges $\alpha(x)\beta(x)$.

Lemma 5.12. *There is a transduction* T *such that* $\mathbf{N} \in \mathsf{T}(\mathbf{M})$.

Proof. For each of the at most k apex positions, use a unary predicate to select the corresponding apex in every margin and further unary predicates to select its neighbors in G_1 within that margin. Membership in a common margin is first-order definable in **M**. A fixed number of interpretation steps therefore restores exactly the edge relation of G_1 inside every margin.

We next reconstruct the local trace forests. In every margin, mark by R_{old} the child endpoint, with respect to the current tree order, of every edge belonging to $F_{Y,x}$ or F_x. Mark every bag element $\zeta_B(x)$ both S and A. The rewiring transduction from Lemma 4.1 deletes the corresponding parent–child edges and attaches every resulting component directly below $\zeta_B(x)$. The edges left inside the margin are exactly those of Y_x^0. The component roots now form an antichain, so Lemma 4.2 applies simultaneously to all components. We mark the minimum of each component in $\preceq_Y$ as its new root, obtaining the orientation inherited from Y_x.

It remains to add the edges of $Y_x^{\mathrm{loc}} \setminus Y_x^0$. Every such edge is incident with a vertex of $A_x \cap \mathsf{mrg}(x)$, because $G_0[\mathsf{mrg}(x)]$ was obtained from $G_1[\mathsf{mrg}(x)]$ precisely by deleting the edges incident with these local apices. Assign the local apices at most k fixed positions. For one apex position at a time, first restore every trace edge whose child is that apex: mark the apex as the component root to be moved and mark its unique parent in Y_x^{loc} as the attachment vertex. In a second application, restore all trace edges whose parent is that apex and whose child is not an apex: mark the apex as the attachment vertex and the corresponding component roots as the roots to be moved. Edges between two apices are handled in the first case, according to the position of their child.

These operations use at most $2k$ applications of the fixed rewiring transduction and are performed simultaneously in all margins. The resulting restriction of $\preceq$ to $\mathsf{mrg}(x)$ is exactly $\preceq_x^{\mathrm{loc}}$. Since the same colors are reused in every margin, the whole construction is a fixed transduction depending only on k. □

The local trace order need not yet be the full restriction of $\preceq_Y$. Indeed, a cover of the restriction of $\preceq_Y$ to a margin need not be an edge of G_1: the corresponding path in Y may leave the margin

and later return to it. The following lemma shows that every cover missing from the local trace forest has an adhesion vertex on its corresponding path in Y. This accounts for all comparisons still missing within a margin, as we show in $(*)$ below. In Step 9, we recover these comparisons by recording the order relations between each margin vertex and the vertices of its adhesion. Here, the *open u-v path* is the unique u-v path in Y with its endpoints u and v removed.

Lemma 5.13. *Let $x \in V(T)$, and let $u \prec_Y v$ be a cover in the restriction of $\preceq_Y$ to $\mathsf{mrg}(x)$. If the open u-v path in Y avoids $\mathsf{adh}(x)$, then $uv \in E(G_1)$.*

Proof. If $uv \in E(Y)$, there is nothing to prove. Otherwise, the open u-v path is non-empty and avoids $\mathsf{bag}(x) = \mathsf{mrg}(x) \dot\cup \mathsf{adh}(x)$, where $\mathsf{adh}(x) = \emptyset$ if x is the root. Indeed, it avoids $\mathsf{mrg}(x)$ because $u \prec_Y v$ is a cover in the restricted order, and it avoids $\mathsf{adh}(x)$ by assumption. Let C be the component of $G_1 - \mathsf{bag}(x)$ containing this open path. The nodes whose bags meet C form a connected subtree of $T - x$, and hence lie in one component D of $T - x$. If q is the neighbor of x in D, the connectedness of the bags containing each vertex implies $N_{G_1}(C) \cap \mathsf{bag}(x) \subseteq \mathsf{bag}(x) \cap \mathsf{bag}(q)$. Both u and v belong to this set. The node q cannot be $\mathsf{parent}(x)$, since $u, v \in \mathsf{mrg}(x)$. Thus, q is a child of x, and $u, v \in \mathsf{adh}(q)$. As $\mathsf{adh}(q)$ is a clique in G_1, we obtain $uv \in E(G_1)$. □

Consequently, for all $u, v \in \mathsf{mrg}(x)$,

$$u \preceq_Y v \quad \Longleftrightarrow \quad u \preceq_x^{\mathrm{loc}} v \ \vee\ \exists a \in \mathsf{adh}(x)\ \big(u \preceq_Y a \preceq_Y v\big). \tag{$*$}$$

For the forward implication, the case $u = v$ is immediate, so suppose that $u \prec_Y v$. If the u-v segment meets $\mathsf{adh}(x)$, we choose a vertex $a \in \mathsf{adh}(x)$ on that segment. Otherwise, let $u = z_0 \prec_Y z_1 \prec_Y \cdots \prec_Y z_\ell = v$ be the vertices of $\mathsf{mrg}(x)$ on the segment, listed in their Y-order. For every $i < \ell$, the pair $z_i \prec_Y z_{i+1}$ is a cover in the restriction of $\preceq_Y$ to $\mathsf{mrg}(x)$, and its open path avoids $\mathsf{adh}(x)$. By Lemma 5.13, we have $z_i z_{i+1} \in E(G_1)$. Hence, by the definition of Y_x^{loc}, every such pair belongs to Y_x^{loc}. Taking the transitive closure gives $u \preceq_x^{\mathrm{loc}} v$.

For the reverse implication, recall that $\preceq_x^{\mathrm{loc}}$ is, by construction, contained in the restriction of $\preceq_Y$ to $\mathsf{mrg}(x)$. Thus, $u \preceq_x^{\mathrm{loc}} v$ implies $u \preceq_Y v$. If instead there exists $a \in \mathsf{adh}(x)$ such that $u \preceq_Y a \preceq_Y v$, then $u \preceq_Y v$ follows directly from the transitivity of $\preceq_Y$.

Step 9. In the final step, we restore the edges and the tree order between different margins. We first need more than a definition of the set $\mathsf{adh}(x)$: a fixed formula must be able to identify each of its elements separately. This is possible because every adhesion has size at most k. The first coordinate in the index set below locates the margin containing an adhesion vertex, the second identifies that vertex using the bounded-indegree orientation from Step 1.

Lemma 5.14. *Let $I = [2k] \times \{0, \dots, d\}$. There is a monadic expansion $\mathbf{N}^+$ of a fixed transduction of $\mathbf{N}$, injections $\iota_x : \mathsf{adh}(x) \longrightarrow I$ for $x \in V(T)$, and first-order formulas $\varphi_{c,i}^{\mathsf{adh}}(v, z)$, for $(c, i) \in I$, such that*

$$\mathbf{N}^+ \models \varphi_{c,i}^{\mathsf{adh}}(v, \zeta_B(x)) \quad \Longleftrightarrow \quad v \in \mathsf{adh}(x) \text{ and } \iota_x(v) = (c, i).$$

For the root of T, whose adhesion is empty, all these formulas are false.

Proof. The E-reduct of $\mathbf{N}$ is a subgraph of G_1 together with the isolated bag elements and ω, and therefore excludes a fixed minor. Apply Lemma 3.3 to this reduct and carry the other relations through by Fact 3.2. Using the bounded-indegree orientation constructed in Step 1, we obtain

directed relations $E_1, \ldots, E_d$ which agree with the corresponding oriented edges of G_1 currently present in $\mathbf{N}$. Every vertex has at most one incoming E_i-edge for each i.

For $v \in V(G_1)$, let $X_v := \{x \in V(T) : v \in \mathsf{bag}(x)\}$. This is a non-empty connected subtree of T, and its minimum is the placement node $b(v)$. Consequently,

$$v \in \mathsf{mrg}(x) \iff x = b(v), \qquad v \in \mathsf{adh}(x) \iff x \in X_v \setminus \{b(v)\}.$$

Let $\mathscr{X}$ be the indexed family of all connected components of $T[X_v \setminus \{b(v)\}]$, over all vertices v. Every node x belongs to exactly one member of $\mathscr{X}$ for each vertex of $\mathsf{adh}(x)$, and therefore to at most k members of $\mathscr{X}$.

We color the subtrees in $\mathscr{X}$ with colors in $[2k]$ so that subtrees of the same color neither intersect nor contain adjacent vertices of T. This is done as follows. We process them in non-decreasing order of the depth of their minimum. When a subtree X with minimum x is processed, every previously processed subtree intersecting or containing a vertex adjacent to X contains x or $\mathsf{parent}(x)$. At most $k-1$ previously processed subtrees contain x, and at most k contain $\mathsf{parent}(x)$. Hence, one of the $2k$ colors is available.

For $x \in V(T)$, let $\Lambda(x)$ be the set of colors of the members of $\mathscr{X}$ containing x, encoded by unary predicates on $\zeta_B(x)$. If $c \in \Lambda(x)$, there is a unique color-c subtree containing x. Let $\eta(x,c)$ denote its minimum. The relation $y = \eta(x,c)$ is first-order definable from $\trianglelefteq$ and the unary color predicates: y is the first vertex of the maximal color-c interval on the ancestor chain ending at x. Moreover,

$$\{\eta(x,c) : c \in \Lambda(x)\} = \{y \preceq_T x : \ y \text{ is non-root and } \mathsf{adh}(x) \cap \mathsf{mrg}(\mathsf{parent}(y)) \neq \emptyset\}.$$

Fix $c \in \Lambda(x)$ and put $y := \eta(x,c)$. The color c identifies a unique vertex $v \in \mathsf{adh}(x) \cap \mathsf{mrg}(\mathsf{parent}(y))$. The relation rep explicitly identifies $\beta(y)$ from $\zeta_B(y)$, even though $\beta(y)$ need not be the unique child of $\zeta_B(y)$ after Step 8. Using a unary color on $\zeta_B(y)$, record the index $j \in [d]$ for which $\alpha(y)$ is the unique E_j-in-neighbor of $\beta(y)$. Thus, $\alpha(y)$ is first-order definable from $\zeta_B(y)$.

Since $v, \alpha(y) \in \mathsf{adh}(y)$ and $\mathsf{adh}(y)$ is a clique of G_1, either $v = \alpha(y)$ or the orientation contains the arc $v\alpha(y)$. The latter holds because $\alpha(y)$ is the sink of the transitive tournament on $\mathsf{adh}(y)$. Set $i = 0$ in the first case, and otherwise let $i \in [d]$ be the unique index for which $E_i(v, \alpha(y))$ holds. A unary predicate on $\zeta_B(x)$ records this index for each c. Define $\iota_x(v) := (c,i)$. Distinct vertices of $\mathsf{adh}(x)$ correspond to distinct members of the indexed family $\mathscr{X}$ through x, and these receive distinct colors, hence ι_x is injective. The construction is expressed by the formulas $\varphi^{\mathsf{adh}}_{c,i}$. □

We can now restore all edges of G_1 between different margins. Let $v \in \mathsf{mrg}(x)$. Every neighbor of v in an ancestor margin belongs to $\mathsf{adh}(x)$. For every $(c,i) \in I$, use a unary predicate on v to record whether v is adjacent in G_1 to the unique vertex of $\mathsf{adh}(x)$ with index (c,i). The placement node x is definable from v through its bag representative $\zeta_B(x)$, so Lemma 5.14 interprets exactly all missing cross-margin edges. Together with the edges already present inside the margins, this recovers the full edge relation of G_1.

It remains to restore the global tree order. Order $I = [2k] \times \{0, \ldots, d\}$ lexicographically. For each $x \in V(T)$, let $\overline{\mathsf{adh}}(x)$ be the I-indexed tuple whose coordinate λ is the unique vertex of $\mathsf{adh}(x)$ with index λ, or a formal absence marker if no such vertex exists. For $x \preceq_T y$, let $\theta(x,y)$ record which coordinates of these two tuples are present, all equalities between present coordinates, and all comparisons in $\preceq_Y$ between present coordinates. We call this finite record the *order type* of $\overline{\mathsf{adh}}(y)$ over $\overline{\mathsf{adh}}(x)$.

Claim 1. *If $x \preceq_T y \preceq_T z$, then $\theta(x,z)$ is determined by $\theta(x,y)$ and $\theta(y,z)$.*

Proof of the claim. Let $a \in \mathsf{adh}(x)$ and $c \in \mathsf{adh}(z)$. Put $W_y := \{w \in V(G_1) : y \preceq_T b(w)\}$. The adhesion $\mathsf{adh}(y)$ separates W_y from all vertices outside $W_y \cup \mathsf{adh}(y)$. We have $a \notin W_y$. Moreover, either $c \in W_y$ or $c \in \mathsf{adh}(y)$, since the bags containing c form a connected subtree and contain z. It follows that the a-c path in Y meets $\mathsf{adh}(y)$, possibly at an endpoint. Consequently,

$$a \preceq_Y c \quad \iff \quad \exists b \in \mathsf{adh}(y)\ a \preceq_Y b \preceq_Y c.$$

The same argument with the order reversed determines whether $c \preceq_Y a$. Both comparisons are determined by $\theta(x,y)$ and $\theta(y,z)$, and equality holds exactly when both do. The internal comparisons in the two end tuples are already recorded in the corresponding intermediate types. $\lhd$

After padding the adhesion tuples by dummy coordinates, the set of order types is finite, and the claim gives a finite transition table for propagating order types along ancestor paths. The types $\theta(x,x)$ and those on parent-child pairs can be encoded by finitely many unary predicates on T. The resulting propagation is MSO-definable by the standard equivalence between finite automata and monadic second-order logic on finite paths [85, 30, 86]. By Colcombet's theorem [23, Theorem 2], all relations $\theta(x,y)$ are therefore first-order definable after a further monadic expansion of $(T, \preceq_T)$, and hence of $\mathbf{N}$ through the bag representatives. These translations are effective, so the required formulas and unary predicates depend computably only on k.

We also record the order type of every margin vertex over its indexed adhesion. For $\lambda \in I$, write $a_\lambda(x)$ for the vertex of $\mathsf{adh}(x)$ with index λ, when it exists. Add unary predicates $P_\lambda^{\uparrow}$ and $P_\lambda^{\downarrow}$ with the meanings

$$\begin{aligned} P_\lambda^{\uparrow}(w) \quad &\iff \quad w \in \mathsf{mrg}(x),\ a_\lambda(x) \text{ exists, and } w \preceq_Y a_\lambda(x), \\ P_\lambda^{\downarrow}(w) \quad &\iff \quad w \in \mathsf{mrg}(x),\ a_\lambda(x) \text{ exists, and } a_\lambda(x) \preceq_Y w. \end{aligned}$$

Only $2|I|$ unary predicates are required, so this is a fixed monadic expansion.

We now define the desired order in three stages. First, let

$$R_1(u,v) :\iff b(u) = b(v) \ \wedge \left(u \preceq v \ \vee \bigvee_{\lambda \in I} \left(P_\lambda^{\uparrow}(u) \wedge P_\lambda^{\downarrow}(v) \right) \right),$$

where $\preceq$ in the first disjunct is the local trace order carried by $\mathbf{N}$. By $(*)$,

$$R_1(u,v) \quad \iff \quad b(u) = b(v) \text{ and } u \preceq_Y v.$$

Next suppose that $b(v) \prec_T b(u)$, and let z be the unique child of $b(v)$ with $z \preceq_T b(u)$. To recover the order type of v over $\mathsf{adh}(z)$, split this adhesion as

$$\mathsf{adh}(z) = \big(\mathsf{adh}(z) \cap \mathsf{adh}(b(v))\big) \,\dot\cup\, \big(\mathsf{adh}(z) \setminus \mathsf{adh}(b(v))\big).$$

Comparisons with the first part are given by the unary code for the type of v over $\mathsf{adh}(b(v))$. Since $b(v) = \mathsf{parent}(z)$, the second part is contained in $\mathsf{mrg}(b(v))$, and its comparisons with v are supplied by R_1. Put $y = b(u)$. The adhesion separators give

$$u \preceq_Y v \quad \iff \quad \exists a \in \mathsf{adh}(y)\ \exists b \in \mathsf{adh}(z)\ u \preceq_Y a \preceq_Y b \preceq_Y v.$$

For the forward implication, the u-v path in Y meets $\mathsf{adh}(y)$ and then $\mathsf{adh}(z)$, possibly at the same vertex. The reverse implication follows from transitivity. The comparisons of u with a are given by

its unary code, those of a with b by the composed type $\theta(z, y)$, and those of b with v by the type just recovered. Thus, the displayed condition is first-order definable. The case $b(u) \prec_T b(v)$ is analogous, exchanging u and v and reversing the comparisons. Together with R_1 for equal placement nodes, this defines a relation R_2 such that

$$R_2(u, v) \iff \big(b(u) \preceq_T b(v) \vee b(v) \preceq_T b(u)\big) \wedge u \preceq_Y v.$$

The last case uses the following separator observation.

Lemma 5.15. *Let $u, v \in V(G)$ have incomparable placement nodes, and let $z = b(u) \curlywedge_T b(v)$. The u-v path in Y meets $\mathsf{bag}(z)$. Every $w \in \mathsf{bag}(z)$ satisfies*

$$b(w) \preceq_T z \preceq_T b(u), b(v).$$

Consequently, if $u \preceq_Y v$, then there is a vertex w with $b(w) \preceq_T b(u), b(v)$ and $u \preceq_Y w \preceq_Y v$.

Proof. The two placement nodes lie in distinct child subtrees of z. The tree-decomposition axioms imply that $\mathsf{bag}(z)$ separates the vertices placed in these two parts of T. Since $Y \subseteq G \subseteq G_1$, the u-v path in Y must therefore meet $\mathsf{bag}(z)$. If $w \in \mathsf{bag}(z)$, the connected set of bags containing w includes z, so its minimum $b(w)$ is an ancestor of z. Finally, when $u \preceq_Y v$, every vertex w on the u-v segment of Y satisfies $u \preceq_Y w \preceq_Y v$. □

It follows that the full tree order is defined by

$$R_3(u, v) :\iff R_2(u, v) \vee \exists w \, \big(b(w) \preceq_T b(u) \wedge b(w) \preceq_T b(v) \wedge R_2(u, w) \wedge R_2(w, v)\big).$$

Indeed, the forward implication in the incomparable-placement case follows from Lemma 5.15, while the reverse implication follows from transitivity of $\preceq_Y$. Thus, $R_3(u, v)$ holds exactly when $u \preceq_Y v$.

We have now recovered $(G_1, \preceq_Y, <_G)$. Fix t and consider inputs G with $h(G) \leqslant t$. By the bound proved in Phase I, $h(G_1) \leqslant q_1(t)$ for the computable function q_1 from Phase I. Hence, the effective transduction from Lemma 3.3, extended as in Fact 3.2, recovers the edge relation of G from G_1 while retaining $\preceq_Y$ and $<_G$.

All transductions used in Steps 6–9 are given by explicit formulas, and the numbers of copies, colors, and formulas used in them are bounded computably in t. Their composition with the preceding expansion transduction therefore forms an effectively computable family $(\mathsf{R}_t)_{t\in\mathbb{N}}$. Since $\mathrm{tww}(\mathscr{M}_t) \leqslant d(t)$ for a computable bound $d(t)$, we overall get a computable bound $\xi(t)$ from Lemma 3.9. This completes the proof. □

We also obtain Theorem 2.1. For every fixed bound t, given a connected graph G with $h(G) \leqslant t$ and a rooted spanning tree Y of G, the construction computes a connected subcubic graph H with a rooted spanning tree Y_H and a DFS preorder $<_H$. The Hadwiger number of H is bounded by a computable function of t. Step 5 supplies the fixed transduction L, while Steps 6–9 supply the transduction R_t, such that

$$(H, \preceq_{Y_H}, <_H) \in \mathsf{L}(H, \preceq_{Y_H}) \qquad \text{and} \qquad (G, \preceq_Y, <_G) \in \mathsf{R}_t(H, \preceq_{Y_H}, <_H)$$

for the linear order $<_G$ produced by the construction. The family $(\mathsf{R}_t)_{t\in\mathbb{N}}$ is effectively computable, and composing R_t with L and forgetting both orders recovers G.

For an arbitrary graph G with $h(G) \leqslant t$ and a rooted spanning forest F of G, apply the connected construction with bound $t + 1$ to the connected extension $(\widehat{G}, Y)$ from Remark 3.1.

Deleting the marked universal vertex from the interpreted output recovers $(G, \preceq_F, <_G)$, where $<_G$ is the restriction of the computed linear order. The auxiliary graph H is unchanged and remains connected and subcubic.

We now consider the special case of monotone classes of graphs.

Theorem 5.16. *Let $\mathscr{C}$ be a monotone class of graphs. Then the following are equivalent:*

1. *$\mathscr{C}$ excludes some graph as a minor;*
2. *$\mathscr{C}^{\Upsilon}$ has bounded twin-width;*
3. *$\mathscr{C}^{F}$ has bounded twin-width;*
4. *for every $G \in \mathscr{C}$, there is a choice of a full rooted spanning forest F_G of G such that the class $\{(G, \preceq_{F_G}) \mid G \in \mathscr{C}\}$ is monadically dependent.*

Proof. The first condition implies the third by Theorem 5.1, since a minor-excluded class is weakly sparse. The third implies the second because $\mathscr{C}^{\Upsilon} \subseteq \mathscr{C}^{F}$. The second implies the fourth by choosing, for every graph, any full rooted spanning forest and using that bounded twin-width implies monadic dependence.

It remains to prove that the fourth condition implies the first. Suppose for a contradiction that $\mathscr{C}$ is monotone and excludes no fixed minor. Choose a sequence $(H_n)_{n\in\mathbb{N}}$ of connected cubic graphs of unbounded twin-width. Such a sequence exists because the class of cubic graphs has unbounded twin-width [9].

For every n, some graph in $\mathscr{C}$ contains H_n as a minor. Taking an edge-minimal minor model and using monotonicity, we conclude that $\mathscr{C}$ contains a subdivision S_n of H_n. Let F_n be the full rooted spanning forest supplied by the fourth condition. Since S_n is connected, F_n is a rooted spanning tree.

Every subdivision vertex has degree two and is incident with an edge of the spanning tree F_n. On a path of S_n that subdivides one edge of H_n, the tree F_n omits at most one edge: omitting two would disconnect the vertices between them. If it omits one, contract every cover edge of F_n on that path; if it omits none, leave one cover edge uncontracted and contract the others. These choices can be made simultaneously so that every contraction class contains exactly one original vertex of H_n. Contracting the selected cover edges therefore collapses the internal vertices of every subdivided path while retaining one edge between its two original endpoints. It produces a tree-ordered copy $(H_n, \preceq_{T_n})$, where T_n is a spanning tree of H_n and hence has maximum degree at most three. The selected cover edges can be guessed by unary colors and the contractions are realized by the fixed tree-order contraction transduction from [20]. Thus, the resulting E-reduct is exactly H_n (with parallel edges, if any, suppressed by the convention that graphs are simple).

Thus, the class $\{(H_n, \preceq_{T_n}) : n \in \mathbb{N}\}$ is a transduction of $\{(S_n, \preceq_{F_n}) : n \in \mathbb{N}\}$ and would be monadically dependent. It is a weakly sparse class of tree-ordered graphs whose cover graphs have maximum degree at most three. By Theorem 3.10, it would consequently have bounded twin-width. This is impossible because its E-reducts are the graphs H_n, which have unbounded twin-width. Hence, $\mathscr{C}$ must exclude a fixed minor. □

Note that Theorem 5.16 does not extend to hereditary weakly sparse classes. This can be seen by considering the weakly sparse example used in [12] in order to prove that Theorem 1.3 does not extend to hereditary classes.

6. Sub-connectivity logic

We now introduce *sub-connectivity logic* $\mathsf{FO{+}subconn}$ on binary relational structures. Let σ be a finite binary relational signature containing a distinguished binary relation symbol Z. The logic is evaluated on σ-structures $\mathbf{M}$ in which $Z(\mathbf{M})$ is symmetric and irreflexive. We identify $\mathbf{M}\restriction_Z$ with the simple undirected graph with vertex set M and edge relation $Z(\mathbf{M})$. In particular, $\mathbf{M}\restriction_Z$ is a subgraph of $\mathrm{Gaif}(\mathbf{M})$.

The logic $\mathsf{FO{+}subconn}[\sigma]$ is obtained from $\mathsf{FO}[\sigma]$ by allowing, for every tuple $\bar{z}$, the additional atomic formula

$$\mathsf{subconn}(x, y; \bar{z}).$$

On a σ-structure $\mathbf{M}$, the atom $\mathsf{subconn}(x, y; z_1, \ldots, z_k)$ holds if x and y are vertices of the same connected component of $\mathbf{M}\restriction_Z - \bar{z}$. In particular, the atom is false if x or y is among the deleted vertices. Paths are allowed to have length zero, so $\mathsf{subconn}(x, x; \bar{z})$ holds precisely when x is not deleted. If $\bar{z}$ is empty, we simply write $\mathsf{subconn}(x, y)$.

6.1. Equivalence to FO with forest orders

To characterize the expressive power of $\mathsf{FO{+}subconn}$, we note the following key lemma about $\mathsf{FO{+}conn}$.

Lemma 6.1. *For every nonnegative integer k, there exists a formula $\psi_k(x, y, \bar{z})$ in $\mathsf{FO}[\{E, \preceq\}]$ with $|\bar{z}| = k$ and the following property: for every rooted depth-first search forest F of a graph G and every $u, v \in V(G)$ and $\bar{w} \in V(G)^k$, we have*

$$G \models \mathsf{conn}(u, v, \bar{w}) \quad \iff \quad (G, \preceq_F) \models \psi_k(u, v, \bar{w}).$$

On the other hand, there exists a formula $\varrho(x, y)$ of $\mathsf{FO{+}conn}$ (for graphs with a unary predicate R), such that if Y is a forest, R marks one root in each connected component of Y, and $\preceq$ is the associated forest order, then

$$(Y, \preceq) \models u \preceq v \quad \iff (Y, R) \models \varrho(u, v).$$

Proof. Let G be a graph, let $k \geqslant 0$, and let F be a rooted depth-first spanning forest of G. We write $\preceq$ for the forest order $\preceq_F$. We use the standard DFS property for the graph G: every edge of G has comparable endpoints (with respect to $\preceq$).

We first define, in first-order logic over $\{E, \preceq\}$, when two vertices lie in the same component of the forest $F - \{z_1, \ldots, z_k\}$. Define

$$\mathsf{Comp}_k(x, y; \bar{z}) := \exists a \Big(\mathsf{lca}(a; x, y) \wedge \bigwedge_{i=1}^{k} \neg(a \preceq z_i \wedge z_i \preceq x) \wedge \bigwedge_{i=1}^{k} \neg(a \preceq z_i \wedge z_i \preceq y) \Big).$$

This formula holds exactly when x and y lie in the same tree of F and the unique tree path from x to y avoids all vertices occurring in $\bar{z}$. Hence,

$$(G, \preceq_F) \models \mathsf{Comp}_k(u, v; \bar{w})$$

if and only if u and v lie in the same connected component of $F - \{w_1, \ldots, w_k\}$.

Next, define a one-step relation between components of $F - \{z_1, \ldots, z_k\}$:

$$\begin{aligned}\mathsf{Step}_k(x, y; \bar{z}) := \mathsf{Comp}_k(x, y; \bar{z}) \\ \vee \exists u \, \exists w \big(\mathsf{Comp}_k(x, u; \bar{z}) \wedge E(u, w) \wedge \mathsf{Comp}_k(w, y; \bar{z})\big).\end{aligned}$$

Thus, $\mathsf{Step}_k(x, y; \bar{z})$ expresses that x and y are either in the same component of $F - \{z_1, \ldots, z_k\}$, or that one can move from the component of x to the component of y using one edge of $G - \{z_1, \ldots, z_k\}$.

If $k = 0$, then x and y are connected in G exactly when they lie in the same tree of F, which is expressed by $\mathsf{Comp}_0(x, y)$. For $k \geqslant 1$, we claim that x and y are connected in $G - \{z_1, \ldots, z_k\}$ if and only if they can be connected by at most $2k$ applications of Step_k. The backward implication is immediate from the definition of Step_k: each step is realized by a path in $G - \{z_1, \ldots, z_k\}$.

For the forward implication, work in the connected component of G containing x and y, restrict F to this component, and ignore every z_i outside it. The restricted forest is a rooted depth-first spanning tree of this component. We say that a connected component of $F - \{z_1, \ldots, z_k\}$ is *internal* if it contains the parent (in $\preceq$) of some z_i. Hence, there are at most k internal connected components. Let P be an x-y path in $G - \{z_1, \ldots, z_k\}$ minimizing the number of edges whose endpoints lie in different connected components of $F - \{z_1, \ldots, z_k\}$. Note that every connected component appears at most once. Moreover, if uv is an inter-component edge of P, then, since $uv \in E$ and F is a DFS forest of G, the vertices u and v are comparable in $\preceq$. As u and v lie in different components of $F - \{z_1, \ldots, z_k\}$, the $\preceq$-chain between them contains at least one z_i. It follows that at least one of the components containing u and v in $F - \{z_1, \ldots, z_k\}$ is internal. As the k internal components are each visited at most once in P, and each touches at most 2 inter-component edges, this yields at most $2k$ inter-component edges in P, and at most $2k+1$ connected components of $F \setminus \{z_1, \ldots, z_k\}$.

Consequently, x and y are connected in $G - \{z_1, \ldots, z_k\}$ if and only if the following first-order formula holds: For $k = 0$, put

$$\mathsf{Conn}_0(x, y) := \mathsf{Comp}_0(x, y).$$

For $k \geqslant 1$, put

$$\mathsf{Conn}_k(x, y; \bar{z}) := \exists v_1 \cdots \exists v_{2k+1} \Big((v_1 = x) \wedge (v_{2k+1} = y) \wedge \bigwedge_{j=1}^{2k} \mathsf{Step}_k(v_j, v_{j+1}; \bar{z})\Big).$$

Since Step_k is reflexive on the undeleted vertices, a sequence with fewer than $2k$ steps can be padded by repetitions.

Thus, for every $u, v \in V(G)$ and $\bar{w} \in V(G)^k$, we have

$$G \models \mathsf{conn}(u, v, \bar{w}) \quad \iff \quad (G, \preceq_F) \models \mathsf{Conn}_k(u, v, \bar{w}).$$

This completes the forward direction. For the converse direction, assume that $(Y, \preceq)$ is a forest equipped with a rooted forest order defined by some rooting of Y, and let R mark the roots.

The relation $u \preceq v$ holds if and only if v is connected to the marked root of its tree and deleting u destroys this connection. Thus, define

$$\varrho(x, y) := \exists r \big(R(r) \wedge \mathsf{conn}(r, y) \wedge \neg \mathsf{conn}(r, y; x)\big).$$

Then

$$(Y, \preceq) \models u \preceq v \quad \iff \quad (Y, R) \models \varrho(u, v). \qquad \square$$

The next theorem yields the promised equivalence for arbitrary binary relational structures. All relations from the original signature are carried through the translations unchanged.

Theorem 6.2.

FO+subconn *is equivalent to* FO *with DFS spanning forest orders in the following sense.*

FO with DFS spanning forest orders can simulate FO+subconn:

Let σ be a finite binary relational signature containing a distinguished binary relation symbol Z and not containing $\preceq$.
For every formula $\varphi(\bar{x})$ of $\mathsf{FO{+}subconn}[\sigma]$, there is an effectively computable formula $\psi(\bar{x}) \in \mathsf{FO}[\sigma \cup \{\preceq\}]$ such that the following holds. Let $\mathbf{M}$ be a σ-structure in which $Z(\mathbf{M})$ is symmetric and irreflexive, and let F be any rooted depth-first search forest of $\mathbf{M}\restriction_Z$. Then, for every tuple $\bar{v} \in M^{|\bar{x}|}$,

$$\mathbf{M} \models \varphi(\bar{v}) \quad \Longleftrightarrow \quad (\mathbf{M}, \preceq_F) \models \psi(\bar{v}).$$

FO+subconn can simulate FO with spanning forest orders:

Let τ be a finite binary relational signature containing neither Z, R, nor $\preceq$.
For every formula $\chi(\bar{x}) \in \mathsf{FO}[\tau \cup \{\preceq\}]$, there is an effectively computable formula $\theta(\bar{x})$ of $\mathsf{FO{+}subconn}[\tau \cup \{Z, R\}]$, where Z and R are fresh binary and unary relation symbols, respectively, with the following property. Let $\mathbf{M}$ be a τ-structure, let $\preceq$ be a rooted forest order on M, let $Z_\preceq$ be the set of undirected cover edges of $\preceq$, and let $R_\preceq$ mark exactly the roots of $\preceq$. Let $\mathbf{M}_\preceq$ be the expansion of $\mathbf{M}$ in which $Z(\mathbf{M}_\preceq) = Z_\preceq$ and $R(\mathbf{M}_\preceq) = R_\preceq$. Then, for every $\bar{v} \in M^{|\bar{x}|}$,

$$(\mathbf{M}, \preceq) \models \chi(\bar{v}) \quad \Longleftrightarrow \quad \mathbf{M}_\preceq \models \theta(\bar{v}).$$

Proof. For each k, let ψ_k^Z be obtained from ψ_k in Lemma 6.1 by replacing every atom $E(u, v)$ by $Z(u, v)$. For the first translation, replace every atom $\mathsf{subconn}(x, y; z_1, \ldots, z_k)$ by $\psi_k^Z(x, y; \bar{z})$. Ordinary atomic formulas, Boolean connectives, and quantifiers are left unchanged.

For the converse, leave the symbols of τ unchanged and replace each atom $x \preceq y$ by

$$\exists r\big(R(r) \wedge \mathsf{subconn}(y, r) \wedge \neg\mathsf{subconn}(y, r; x)\big).$$

Because $Z(\mathbf{M}_\preceq)$ is exactly the symmetric edge relation of the cover forest of $\preceq$, correctness follows from the second part of Lemma 6.1. □

6.2. Model checking

We now derive the model-checking result for classes excluding a fixed minor, which follows immediately from Theorem 6.2 and Corollary 5.5.

Theorem 6.3. *There exist a computable function f and an absolute constant c such that the following holds. Given a finite binary relational structure $\mathbf{M}$ over a finite signature containing the distinguished binary relation symbol Z, where $Z(\mathbf{M})$ is symmetric and irreflexive, and an* FO+subconn*-sentence φ over its signature, one can decide whether $\mathbf{M} \models \varphi$ in time*

$$f\big(|\varphi|, h(\mathrm{Gaif}(\mathbf{M}))\big) \cdot \|\mathbf{M}\|^c,$$

where $\|\mathbf{M}\|$ denotes the size of the input encoding. In particular, FO+subconn *model checking is fixed-parameter tractable, parameterized by formula length, on every class of binary structures whose Gaifman graphs exclude a fixed minor.*

Proof. Let σ_φ be the set of relation symbols other than Z that occur in φ, and let $\mathbf{M}_\varphi$ be the $(\sigma_\varphi \cup \{Z\})$-reduct of $\mathbf{M}$. This does not change whether φ holds, while $|\sigma_\varphi| \leqslant |\varphi|$, $\|\mathbf{M}_\varphi\| \leqslant \|\mathbf{M}\|$, and

$$h(\mathrm{Gaif}(\mathbf{M}_\varphi)) \leqslant h(\mathrm{Gaif}(\mathbf{M})).$$

Compute a rooted depth-first search forest F_Z of $\mathbf{M}_\varphi \restriction_Z$, including every isolated vertex as a one-vertex tree. Since $\mathbf{M}_\varphi \restriction_Z$ is a subgraph of $\mathrm{Gaif}(\mathbf{M}_\varphi)$, the forest F_Z is a rooted spanning forest of $\mathrm{Gaif}(\mathbf{M}_\varphi)$. By Theorem 6.2, one can effectively compute a first-order sentence ψ such that

$$\mathbf{M} \models \varphi \quad \Longleftrightarrow \quad (\mathbf{M}_\varphi, \preceq_{F_Z}) \models \psi.$$

Moreover, $|\psi|$ is bounded by a computable function of $|\varphi|$. Applying Corollary 5.5 to the right-hand side proves the claimed running time. The computation of the reduct, the Gaifman graph, and the DFS forest takes polynomial time with an absolute exponent and is absorbed into f and c. □

6.3. Monadic dependence

We now show that on weakly sparse graph classes FO+subconn characterizes classes with excluded minors. Recall that a class is *monadically dependent for* FO+subconn if every expansion by finitely many unary predicates is FO+subconn-dependent, that is, no FO+subconn-formula has the independence property on the class of all such expansions.

For a class $\mathscr{K}$ of relational structures, write $\mathrm{Gaif}(\mathscr{K}) := \{\mathrm{Gaif}(\mathbf{M}) \mid \mathbf{M} \in \mathscr{K}\}$. For a graph class $\mathscr{C}$, let

$$\mathscr{C}_Z := \{(G, Z) \mid G \in \mathscr{C},\ Z \subseteq E(G)\}.$$

Theorem 6.4. *Let $\mathscr{K}$ be a class of finite binary relational structures over a fixed signature containing the distinguished binary relation symbol Z, such that $Z(\mathbf{M})$ is symmetric and irreflexive for every $\mathbf{M} \in \mathscr{K}$. If $\mathrm{Gaif}(\mathscr{K})$ excludes a fixed minor, then $\mathscr{K}$ is monadically dependent for* FO+subconn*.*
Moreover, if $\mathscr{C}$ is a weakly sparse class of graphs, then

$$\mathscr{C}_Z \text{ is monadically dependent for FO+subconn} \quad \Longleftrightarrow \quad \mathscr{C} \text{ excludes a fixed minor.}$$

Proof. Assume first that $\mathrm{Gaif}(\mathscr{K})$ excludes a fixed minor. For every $\mathbf{M} \in \mathscr{K}$, choose a rooted depth-first search forest $F_{\mathbf{M}}$ of $\mathbf{M} \restriction_Z$, and let

$$\mathscr{D} := \{(\mathbf{M}, \preceq_{F_{\mathbf{M}}}) : \mathbf{M} \in \mathscr{K}\}.$$

Each $F_{\mathbf{M}}$ is a rooted spanning forest of $\mathrm{Gaif}(\mathbf{M})$. By Theorem 5.1, the class of all forest-ordered expansions of the graphs in $\mathrm{Gaif}(\mathscr{K})$ has bounded twin-width and is therefore monadically dependent.

By Lemma 3.3 and Fact 3.2, the class $\mathscr{D}$ is contained in a fixed transduction of that forest-ordered graph class. Consequently, $\mathscr{D}$ is monadically dependent as well.

Suppose, towards a contradiction, that $\mathscr{K}$ is monadically independent for FO+subconn. Then some FO+subconn-formula $\varphi(\bar{x};\bar{y})$ has the independence property on a class of monadic expansions of $\mathscr{K}$. Equip every underlying structure with the fixed DFS forest order chosen above. The added unary predicates make the resulting structures a monadic expansion of $\mathscr{D}$. By the first translation in Theorem 6.2, the formula φ is replaced uniformly by a first-order formula $\varphi^\dagger$ with exactly the same truth values on the witnessing tuples. Hence, $\varphi^\dagger$ has the independence property on a monadic expansion of $\mathscr{D}$, contradicting the monadic dependence of $\mathscr{D}$. This proves the first assertion.

Now let $\mathscr{C}$ be a weakly sparse graph class. If $\mathscr{C}$ excludes a fixed minor, then $\mathrm{Gaif}(\mathbf{M}) = G$ for every $\mathbf{M} \in \mathscr{C}_Z$ whose E-reduct is G, and the first assertion applies. Conversely, suppose that $\mathscr{C}$ does not exclude a minor. By Theorem 1.2, some first-order formula has the independence property on a monadic expansion of $\mathscr{C}^\Upsilon$. For every $(G, \preceq_F) \in \mathscr{C}^\Upsilon$, let $\mathbf{M}_{G,F} \in \mathscr{C}_Z$ be the expansion of G in which $Z(\mathbf{M}_{G,F})$ is the symmetric edge relation of F. Keep the unary predicates of the witnessing expansion and add a fresh unary predicate R marking the roots of F. The second translation in Theorem 6.2 replaces the witnessing formula by an FO+subconn-formula with exactly the same truth values on the witnessing tuples. Hence, this formula has the independence property on a monadic expansion of $\mathscr{C}_Z$, so $\mathscr{C}_Z$ is not monadically dependent for FO+subconn. □

We now show hardness of FO+subconn model checking on monotone classes that do not exclude a minor, under the efficient encoding hypothesis below. The encoding algorithm need not output a minor model. We use an alternative construction that avoids having to find one by using a subcubic coding graph whose minor-minimal host is necessarily a subdivision.

Theorem 6.5. *Assume that $\mathscr{C}$ is monotone, does not exclude any fixed minor, and admits efficient encoding, that is, there is a polynomial-time algorithm that, given a graph H, computes a graph $F \in \mathscr{C}$ such that $H \leqslant_m F$ and $H \not\leqslant_m F'$ for every proper subgraph F' of F. Then deciding, given a graph $F \in \mathscr{C}$, a subset $Z_F \subseteq E(F)$, and an* FO+subconn*-sentence φ, whether $(F, Z_F) \models \varphi$ is* AW[⋆]*-hard, parameterized by $|\varphi|$.*

Proof. We reduce from first-order model checking on finite linearly-ordered partial matchings. Thus, the input structure has the form

$$\mathbf{M} = (M, E(\mathbf{M}), <(\mathbf{M})),$$

where $E(\mathbf{M})$ is the edge relation of a graph of maximum degree at most 1 and $<(\mathbf{M})$ is a linear order on M. First-order model checking on this class is AW[⋆]-hard by [39, Theorem 1].

Enumerate M as $M = \{v_1, \dots, v_n\}$ so that

$$(v_i, v_j) \in <(\mathbf{M}) \quad \Longleftrightarrow \quad i < j.$$

We first construct from $\mathbf{M}$ a connected graph $G(\mathbf{M})$ all of whose vertices have degree either 1 or 3.

Start with the path whose vertex sequence is

$$r, v_1, \dots, v_n, t.$$

For each $i \in [n-1]$ such that at least one of v_i and v_{i+1} is incident with an edge of $E(\mathbf{M})$, subdivide the edge v_iv_{i+1} once, and attach a new leaf to the subdivision vertex. If both v_i and v_{i+1} are isolated in $\mathbf{M}$, leave the edge v_iv_{i+1} unchanged. Add the edge v_iv_j for every $v_iv_j \in E(\mathbf{M})$, and attach one new leaf to every vertex v_i that is isolated in $\mathbf{M}$. Finally, attach two new leaves to t, and attach

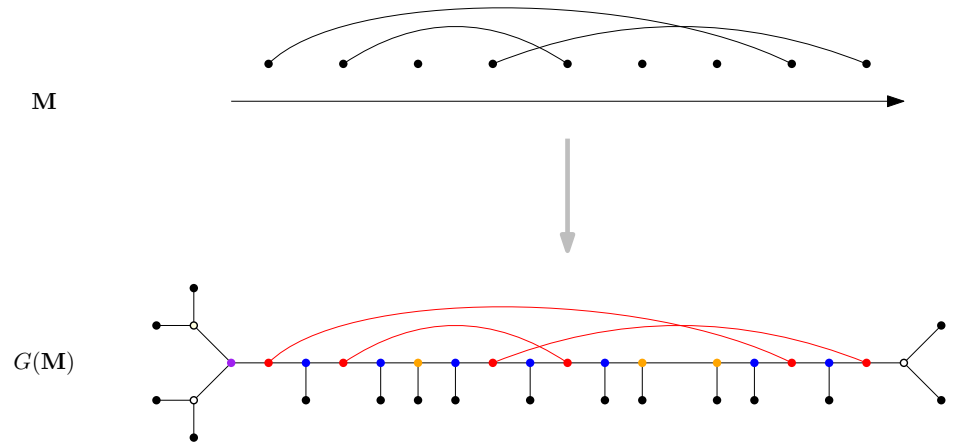


Figure 14: The coding of an ordered partial matching $\mathbf{M}$ by $G(\mathbf{M})$. Degree-1 vertices are black; degree-3 vertices with one black neighbor are preliminarily blue, and those with two black neighbors are white. The unique vertex with two white neighbors is purple, and all remaining degree-3 vertices are red. Finally, a preliminarily blue vertex with no red neighbor is recolored orange. The red and orange vertices form the interpreted domain, the red-red edges are precisely the matching edges, and the purple vertex anchors the order encoded by the decorated spine. In the subdivision $F(\mathbf{M})$, the admissible edge set is obtained by deleting one edge from each path corresponding to a red-red edge.

to r two new vertices, each of which receives two new leaf neighbors. Every vertex of $G(\mathbf{M})$ has degree either 1 or 3.

The elements of M can be recognized intrinsically in $G(\mathbf{M})$. First color the degree-1 vertices black. Among the degree-3 vertices, color those with exactly one black neighbor preliminarily blue and those with exactly two black neighbors white. Color the unique vertex with exactly two white neighbors purple, and color all remaining degree-3 vertices red. Finally, recolor orange every preliminarily blue vertex having no red neighbor.

The purple vertex is r. The red vertices are precisely the vertices v_i that are incident with an edge of $E(\mathbf{M})$, whereas the orange vertices are precisely the isolated vertices of $\mathbf{M}$. Indeed, an auxiliary subdivision vertex is introduced between v_i and v_{i+1} only if at least one of them is matched, and hence it has a red neighbor and remains blue. On the other hand, an unmatched vertex v_i has no red neighbor: whenever one of its neighbors in the order is matched, the corresponding spine edge is subdivided. Consequently, the red and orange vertices are exactly $v_1, \dots, v_n$. Moreover, the edges with two red endpoints are exactly the edges of $E(\mathbf{M})$. In particular, $G(\mathbf{M})$ with its red-red edges deleted is a tree.

Apply the assumed encoding algorithm to $G(\mathbf{M})$, and denote the resulting graph by $F(\mathbf{M})$. Thus,

$$F(\mathbf{M}) \in \mathscr{C}, \qquad G(\mathbf{M}) \leqslant_m F(\mathbf{M}),$$

and no proper subgraph of $F(\mathbf{M})$ contains $G(\mathbf{M})$ as a minor. We use the following elementary observation, which is well known.

Claim. Let K be a connected graph all of whose vertices have degree either 1 or 3. If $K \leqslant_m F$, but $K \not\leqslant_m F'$ for every proper subgraph F' of F, then F is a subdivision of K.

By the claim, $F(\mathbf{M})$ is a subdivision of $G(\mathbf{M})$. Notice that the minor model was used only to prove this structural fact, the reduction does not compute it. Since $G(\mathbf{M})$ has no degree-2 vertices, its vertices correspond exactly to the vertices of $F(\mathbf{M})$ whose degree is 1 or 3. We can therefore recover the suppressed graph and all its subdivided edge paths in linear time: starting at every vertex of degree 1 or 3, traverse each incident maximal path whose internal vertices have degree 2.

The resulting suppressed graph is isomorphic to $G(\mathbf{M})$, and the coloring described above can be computed on it without finding an isomorphism with the labeled copy of $G(\mathbf{M})$.

For every red-red edge of the suppressed graph, choose one edge on its corresponding path in $F(\mathbf{M})$. Let $D_{\mathbf{M}}$ be the set of chosen edges and put

$$Z_{\mathbf{M}} := E(F(\mathbf{M})) \setminus D_{\mathbf{M}}.$$

Let $\mathbf{A}_{\mathbf{M}}$ be the $\{E, Z\}$-structure with

$$E(\mathbf{A}_{\mathbf{M}}) = E(F(\mathbf{M})) \qquad \text{and} \qquad Z(\mathbf{A}_{\mathbf{M}}) = Z_{\mathbf{M}}.$$

Since deleting the red-red edges from $G(\mathbf{M})$ leaves a tree, $\mathbf{A}_{\mathbf{M}} \restriction_Z$ is a spanning tree of $F(\mathbf{M})$. Indeed, every path replacing a matching edge is cut once, and its two remaining pieces hang from the subdivision of that tree.

We now explain how to recover $\mathbf{M}$ by a fixed $\mathsf{FO{+}subconn}$-interpretation. The branch vertices are precisely the vertices of degree 1 or 3 in $F(\mathbf{M})$. Since the admissible graph is a tree, two distinct branch vertices are consecutive on an admissible path if deleting any other branch vertex leaves them connected. This is expressible in $\mathsf{FO{+}subconn}$ and recovers the suppressed graph with its red-red edges removed. We can then apply the coloring described above, using degrees in $F(\mathbf{M})$ for the degree tests and the recovered adjacency for the tests on neighbors. The omitted red-red edges do not affect these tests on neighbors. Thus, the red and orange vertices give the domain of the interpreted structure.

To recover the matching, consider an edge outside $Z_{\mathbf{M}}$. Its endpoints lie in the two remaining halves of a subdivided matching edge. Each endpoint is joined to a unique branch vertex by an admissible path containing no other branch vertex. This also includes the case where the endpoint is itself a branch vertex. The same deletion test expresses this condition in $\mathsf{FO{+}subconn}$. Two red vertices are therefore matched precisely when they correspond in this way to the endpoints of an edge outside $Z_{\mathbf{M}}$.

Finally, the purple vertex r marks the beginning of the spine. For distinct red or orange vertices x, y, we have

$$x < y \quad \iff \quad \mathsf{subconn}(r, x; y).$$

Indeed, deleting y leaves r connected to x exactly when x occurs before y on the spine.

Given a first-order sentence φ, we replace its matching and order atoms by these definitions and relativize its quantifiers to the red and orange vertices. The resulting $\mathsf{FO{+}subconn}$-sentence $\varphi^\dagger$ satisfies

$$\mathbf{M} \models \varphi \quad \iff \quad \mathbf{A}_{\mathbf{M}} \models \varphi^\dagger.$$

All formulas defining the interpretation are fixed, so $|\varphi^\dagger| = O(|\varphi|)$.

The graph $G(\mathbf{M})$ has size linear in $|\mathbf{M}|$, and the encoding algorithm computes $F(\mathbf{M})$ in polynomial time. Suppressing the degree-2 paths, computing the intrinsic coloring, and constructing $Z_{\mathbf{M}}$ all take linear time in $|F(\mathbf{M})|$. We have therefore obtained an fpt-reduction from first-order model checking on ordered partial matchings, which proves the theorem. □

7. Combinatorial reconfiguration

We now present the algorithmic application to combinatorial reconfiguration, discussed in the introduction.

Fix a graph G and sets $S, T \subseteq V(G)$. We think of S as a set of initial tokens placed on the graph. A *token sliding sequence of length* ℓ from S to T is a sequence $S = S_0, \ldots, S_\ell = T$ satisfying, for every $i \in [\ell]$,

$$S_i = (S_{i-1} \setminus \{u_i\}) \cup \{v_i\}$$

for some $u_i \in S_{i-1}$, $v_i \in V(G) \setminus S_{i-1}$, and $u_i v_i \in E(G)$. This means that each step slides one token to an unoccupied neighbor.

Let $\mathcal{L}$ be a logic over the signature of colored graphs. The $\mathcal{L}$-Modulator-Reconfiguration problem is defined as follows. Given a (possibly colored) graph G, sets $S, T \subseteq V(G)$, a movement budget $b \in \mathbb{N}$, and an $\mathcal{L}$-sentence ψ, decide whether there exists a token sliding sequence $S = S_0, \ldots, S_\ell = T$ from S to T of length $\ell \leqslant b$ such that

$$G - S_i \models \psi \qquad \text{for every } 0 \leqslant i \leqslant \ell.$$

When the formula ψ is fixed, we also speak of the ψ-Modulator-Reconfiguration problem.

Let us show how this captures some standard reconfiguration problems, and why the use of FO+subconn is natural here. Let P_S be a fresh unary relation symbol, and let $\mathbf{G}_S$ be the expansion of G in which

$$P_S(\mathbf{G}_S) = S \qquad \text{and} \qquad Z(\mathbf{G}_S) := \{(u,v) \in E(\mathbf{G}_S) : u \notin S \text{ and } v \notin S\}.$$

Thus, $\mathbf{G}_S \restriction_Z$ is obtained from $G - S$ by adding the vertices of S as isolated vertices. In the following formulas, Z and P_S denote the corresponding relation symbols, interpreted in $\mathbf{G}_S$ as above. Over the course of the reconfiguration sequence, the newly occupied vertices will be deleted in the subconn atoms, while connections through vertices that have become free will be handled explicitly. The examples state the desired properties for the initial token configuration.

As a first example, Feedback Vertex Set Reconfiguration is obtained by requiring that $G - S$ is acyclic. In FO+subconn this can be expressed as

$$\psi_{\mathrm{fvs}} := \forall x \forall y \forall z \Big(Z(x,y) \wedge Z(x,z) \wedge y \neq z \to \neg \mathsf{subconn}(y,z;x) \Big).$$

This forbids cycles in $\mathbf{G}_S \restriction_Z$. Since the vertices of S are isolated in this graph, this is equivalent to saying that $G - S$ is acyclic, and hence says exactly that S is a feedback vertex set.

As a second example, suppose that two distinct terminals are marked by unary relation symbols P_s and P_t. Then s-t Cut Reconfiguration is captured by the FO+subconn sentence

$$\begin{aligned}\psi_{s,t} := \exists s \exists t \Big(& P_s(s) \wedge P_t(t) \wedge s \neq t \wedge \neg P_S(s) \wedge \neg P_S(t) \\ & \wedge \forall x (P_s(x) \to x = s) \wedge \forall x (P_t(x) \to x = t) \wedge \neg \mathsf{subconn}(s,t) \Big).\end{aligned}$$

The subformula $\neg\mathsf{subconn}(s,t)$ says that there is no s-t path in $G - S$, while the conjuncts $\neg P_S(s)$ and $\neg P_S(t)$ ensure that the terminals themselves are not deleted.

We show the following meta theorem.

Theorem 7.1. *There exist a computable function f and an absolute constant c such that, given a possibly unary-colored graph G, sets $S, T \subseteq V(G)$, a budget $b \in \mathbb{N}$, and a sentence $\psi \in$ FO+conn, one can decide the ψ-Modulator-Reconfiguration problem in time*

$$f(|\psi|, h(G), b) \cdot |G|^c.$$

Proof. Let P_S and P_T be fresh unary relation symbols and Z a fresh binary relation symbol. Given an instance (G, S, T, b, ψ), let $\mathbf{G}_{S,T}$ be the expansion of G in which $P_S(\mathbf{G}_{S,T}) = S$, $P_T(\mathbf{G}_{S,T}) = T$, and $Z(\mathbf{G}_{S,T}) := \{uv \in E(G) : \{u, v\} \cap S = \emptyset\}$. The Gaifman graph of $\mathbf{G}_{S,T}$ is exactly G.

Let S' be a configuration occurring after at most b token slides, and put $F := S \setminus S'$ and $B := S' \setminus S$. Since every slide vacates one occupied vertex and occupies one adjacent unoccupied vertex,

$$S' = (S \setminus F) \cup B, \qquad F \subseteq S, \quad B \subseteq V(G) \setminus S, \quad |F| = |B| \leqslant b.$$

We represent F and B by tuples of at most b vertices; membership is then expressed by disjunctions of equalities. For every such pair (F, B), the assertion $G - \big((S \setminus F) \cup B\big) \models \psi$ is expressible by an FO+subconn-formula whose length depends only on $|\psi|$ and b. For a connectivity atom of the FO+conn-sentence ψ, observe that every simple path in the remaining graph can be split into at most $2b$ individual edges incident with F and at most $2b + 1$ subpaths using only edges from Z and avoiding B. It can therefore be expressed by a chain of at most $4b+1$ edge or sub-connectivity atoms.

It is now entirely standard to express a token sliding sequence of length at most b. For every $\ell \leqslant b$, quantify representations (F_i, B_i) as above of configurations $S_i = (S \setminus F_i) \cup B_i$, require $S_0 = S$ and $S_\ell = T$, and require $G - S_i \models \psi$ for every $0 \leqslant i \leqslant \ell$ using the translation above. For every $i \in [\ell]$, require adjacent vertices $u_i \in S_{i-1}$ and $v_i \notin S_{i-1}$ such that $S_i = (S_{i-1} \setminus \{u_i\}) \cup \{v_i\}$.

Taking the disjunction over all $\ell \leqslant b$ gives an effectively computable FO+subconn-sentence $\varphi_{\psi,b}$ such that

$$\mathbf{G}_{S,T} \models \varphi_{\psi,b} \quad \Longleftrightarrow \quad (G, S, T, b) \text{ is a yes-instance of } \psi\text{-Modulator-Reconfiguration}.$$

Note that the length of $\varphi_{\psi,b}$ depends only on $|\psi|$ and b. Since the Gaifman graph of $\mathbf{G}_{S,T}$ is G, the claimed running time follows from Theorem 6.3. □

The examples given above give the following corollary.

Corollary 7.2. *Vertex-Cut-Reconfiguration and Feedback-Vertex-Set-Reconfiguration are fixed-parameter tractable when parameterized by b on graph classes excluding a fixed minor.*

7.1. Reconfiguration variants and solution discovery

Our algorithm for the ψ-Modulator-Reconfiguration problem (Theorem 7.1) reduces the problem to model checking a FO+subconn formula. This approach is highly flexible and shows the power of algorithmic meta-theorems: with simple modifications to the constructed formula, we can easily adapt the algorithm to other variations of the reconfiguration framework. A common variation is to change the rules for how one token set S_i can be transformed to the next set S_{i+1}. Popular models include

- the *token sliding* model, where in each step a token is moved along a single edge [70],
- the *token jumping* model, where in each step a token moves to an arbitrary vertex [70],
- the *token addition and removal* model, where in each step either a token is added to an arbitrary vertex, or an existing token is removed [70], and
- the *token sliding along a path*[4] model, where in each step a token moves along an arbitrarily long path of unoccupied vertices [22, 24].

[4]To the best of our knowledge, there is no established name for this model.

We have given the proof of Theorem 7.1 for the token sliding model, but it can be easily adapted to the other models. Especially the last model fits FO+subconn perfectly.

Another emerging variant of the reconfiguration framework is the solution discovery framework, which was recently introduced in [41] and further studied in [16, 53, 54, 76, 89]. Here S is viewed as a tentative solution that may fail to satisfy the target property and should be transformed into an a priori unknown solution T. More formally, in the ψ-Modulator-Discovery problem the input consists of G, S, b, and ψ, and the question is whether, for some $\ell \leqslant b$, there exists a token sliding sequence

$$S = S_0, \ldots, S_\ell = T$$

such that $G - T \models \psi$. No feasibility requirement is imposed on the intermediate sets. The proof of Theorem 7.1 can be easily adapted to prove the following theorem. The result entails fixed-parameter tractability of Feedback-Vertex-Set-Discovery and Vertex-Cut-Discovery on classes excluding a fixed minor, parameterized by the budget b.

Theorem 7.3. *There exist a computable function f and an absolute constant c such that, given a possibly unary-colored graph G, a set $S \subseteq V(G)$, a budget $b \in \mathbb{N}$, and a sentence $\psi \in$ FO+conn, one can decide the ψ-Modulator-Discovery problem in time*

$$f(|\psi|, h(G), b) \cdot |G|^c.$$

8. Further applications and discussion

In this section, we discuss two extensions of sub-connectivity logic and its connection with regular path queries. Throughout, Z denotes a fresh binary relation symbol. If Z is interpreted by a symmetric and irreflexive relation $F \subseteq M^2$, we write $\mathbf{M}[Z := F]$ for the resulting expansion. The applicability and running-time bound of the positive model-checking theorem depend on this expansion through $h(\mathrm{Gaif}(\mathbf{M}[Z := F]))$; unless explicitly required below, $Z(\mathbf{M}[Z := F]) = F$ need not be contained in any relation of the original signature.

8.1. Guarded−FO+subconn

We now describe a new logic, which we call Guarded−FO+subconn, which generalizes FO+subconn while still admitting efficient model checking. The idea is to fix the Gaifman graph of a structure and allowing to update the Z-relation in a controlled way.

We consider a binary relational signature σ containing a special binary relation symbol E, and let $Z \notin \sigma$ be a fresh binary relation symbol. We restrict ourselves to σ-structures $\mathbf{M}$ in which $E(\mathbf{M})$ is symmetric and irreflexive and, for every binary relation symbol $R \in \sigma$,

$$R(\mathbf{M}) \setminus \{(v, v) : v \in M\} \subseteq E(\mathbf{M}).$$

Thus, the E-reduct is precisely the Gaifman graph of $\mathbf{M}$, which we consider as fixed.

We define the syntax inductively. Put $\mathcal{L}_0 := \mathsf{FO}[\sigma]$. Inductively, let $\mathcal{L}_{r+1}$ be the Boolean closure of $\mathcal{L}_r$ together with all formulas

$$[\varphi]\, \psi(\bar{x}),$$

where $\varphi(p, q) \in \mathcal{L}_r$ has precisely the free variables p and q and $\psi(\bar{x}) \in \mathsf{FO+subconn}[\sigma \cup \{Z\}]$, with Z as the distinguished sub-connectivity relation symbol of this full signature. Set

$$\text{Guarded−FO+subconn}[\sigma] := \bigcup_{r \geqslant 0} \mathcal{L}_r.$$

The semantics is inductive on r. For $\varphi(p,q) \in \mathcal{L}_r$, define

$$Z_{\varphi,\mathbf{M}} := \{(a,b) \in M^2 : \mathbf{M} \models E(a,b) \wedge (\varphi(a,b) \vee \varphi(b,a))\}.$$

Then

$$\mathbf{M} \models [\varphi]\,\psi(\bar{v}) \quad \Longleftrightarrow \quad \mathbf{M}[Z := Z_{\varphi,\mathbf{M}}] \models \psi(\bar{v}).$$

Theorem 8.1. *There exist a computable function g and an absolute constant C such that, given a finite binary signature σ, a σ-structure $\mathbf{M}$ of the kind just described, and a sentence $\chi \in$ Guarded-FO+subconn$[\sigma]$, one can decide whether $\mathbf{M} \models \chi$ in time*

$$g\big(|\chi|, |\sigma|, h(\mathrm{Gaif}(\mathbf{M}))\big) \cdot \|\mathbf{M}\|^C.$$

Proof. Evaluate the subformulas in increasing rank and memoize the binary relation defined by every guard. To compute the relation defined by a guard $\varphi(p,q)$, evaluate it on all ordered pairs $(a,b) \in M^2$. Assigned vertices are encoded by fresh unary singleton predicates. Rank-zero formulas are evaluated by Theorem 6.3 after expanding $\mathbf{M}$ by the empty interpretation of Z. Every scheme occurrence $[\eta]\,\psi(\bar{x})$ is evaluated, once $Z_{\eta,\mathbf{M}}$ has been computed, by applying Theorem 6.3 to $\mathbf{M}[Z := Z_{\eta,\mathbf{M}}]$ with the appropriate singleton predicates. Because the construction uses only Boolean closure, a scheme occurrence inside a guard with free variables p,q has no other free variables; its truth table therefore has at most $|M|^2$ entries. A top-level scheme occurrence in the input sentence has no free variables and requires only one evaluation. Memoization ensures that each occurrence is processed only once.

Every relation $Z_{\eta,\mathbf{M}}$ is contained in $E(\mathbf{M})$, and the additional singleton predicates are unary. Hence, every structure supplied to Theorem 6.3 has the same Gaifman graph as $\mathbf{M}$. Let c be the absolute exponent from that theorem. The factor $|M|^2$ needed to tabulate a binary guard raises the polynomial exponent to at most $c+2$, independently of χ, σ, and the excluded minor. All remaining dependence is absorbed into a computable function of $|\chi|$, $|\sigma|$, and $h(\mathrm{Gaif}(\mathbf{M}))$, which proves the stated bound with the absolute constant $C := c+2$. □

For example, let $\eta_1(x,y)$ and $\eta_2(x,y)$ be first-order formulas defining two equivalence relations on the vertices of a graph, and let $\psi(\bar{x})$ be an FO+subconn-formula. If we think about equivalence classes as vertex colors, then, for each $i \in \{1,2\}$, $[\eta_i]\,\psi(\bar{x})$ interprets the $\mathsf{subconn}(x,y;\bar{z})$ atom as asserting the existence of a monochromatic path linking x to y while avoiding $\bar{z}$, with respect to the colors defined by η_i. For example, the sentence

$$\big[\eta_1\big]\,\forall x \forall y\,\big(\eta_1(x,y) \rightarrow \mathsf{subconn}(x,y)\big) \;\wedge\; \big[\eta_2\big]\,\forall x \forall y\,\big(\eta_2(x,y) \rightarrow \mathsf{subconn}(x,y)\big)$$

states that for each of the two equivalence relations, all vertices in each equivalence class are pairwise connected by monochromatic paths. Thus, every class induces a connected subgraph. The two conjuncts are evaluated independently, each with its own sub-connectivity relation, which remains fixed throughout the evaluation of its quantified formula.

8.2. More than one set of admissible edges

Let k be a positive integer and let $\tau_k := \{E, Z_1, \ldots, Z_k\}$, where $Z_1, \ldots, Z_k$ are distinct binary relation symbols. We consider τ_k-structures $\mathbf{M}$ in which $E(\mathbf{M})$ and every $Z_i(\mathbf{M})$ are symmetric and irreflexive. For $i \in [k]$, let $G_{Z_i}(\mathbf{M})$ be the graph with vertex set M and edge set

$$E(G_{Z_i}(\mathbf{M})) := \big\{\{u,v\} \in \tbinom{M}{2} : (u,v) \in Z_i(\mathbf{M})\big\}.$$

The logic $\mathsf{FO}{+}\mathsf{subconn}^k$ is obtained from $\mathsf{FO}[\tau_k]$ by adding, for every $i \in [k]$, atoms $\mathsf{subconn}_i(x, y; \bar{z})$ asserting that x and y belong to the same connected component of $G_{Z_i}(\mathbf{M}) - V(\bar{z})$.

For a graph class $\mathscr{C}$, let $\mathscr{C}_{\bar{Z}}$ be the class of all τ_k-structures $\mathbf{M}$ whose E-reduct is a graph $G \in \mathscr{C}$ and for which

$$E(G_{Z_i}(\mathbf{M})) \subseteq E(G) \qquad \text{for every } i \in [k].$$

The interpretations of the symbols Z_i need not be disjoint.

Theorem 8.2. *Let $k \geqslant 2$. For every monotone class of graphs $\mathscr{C}$, the class $\mathscr{C}_{\bar{Z}}$ is $\mathsf{FO}{+}\mathsf{subconn}^k$-dependent if and only if $\mathscr{C}$ has bounded tree-width.*

Proof. Assume first that $\mathscr{C}$ has unbounded tree-width. For $n \geqslant 2$, fix an ordering of $2^{[n]}$ whose first element is $\emptyset$, and let H_n be the $n \times 2^n$ grid with vertices $v_{i,J}$ indexed by $[n] \times 2^{[n]}$ in this order. Attach a pendant vertex $w_{i,J}$ to $v_{i,J}$ whenever $i \in J$, and put $W := \{w_{i,J} : i \in J \subseteq [n]\}$.

For every fixed planar graph H, the class of H-minor-free graphs has bounded tree-width [74].Hence, some graph in $\mathscr{C}$ contains H_n as a minor. By monotonicity, $\mathscr{C}$ contains an inclusion-minimal subgraph H'_n supporting such a model. In this minimal model, every branch set is a tree and exactly one edge is retained for each edge of H_n. Let $f\colon V(H'_n) \to V(H_n)$ be the map specifying the resulting surjective minor model. Minimality implies that each pendant branch set $f^{-1}(w)$, $w \in W$, is a single degree-one vertex, and that H'_n has no other degree-one vertices.

Define edge sets $\mathcal{Z}_1$ and $\mathcal{Z}_2$ as follows. For each $xy \in E(H'_n)$:

- if $x, y \in f^{-1}(v_{i,J})$ for some grid vertex $v_{i,J}$, then xy belongs to both $\mathcal{Z}_1$ and $\mathcal{Z}_2$;
- if $x \in f^{-1}(v_{i,J})$ and $y \in f^{-1}(v_{i,J'})$, where $v_{i,J}$ and $v_{i,J'}$ are horizontally adjacent in H_n, then $xy \in \mathcal{Z}_1$;
- if $x \in f^{-1}(v_{i,J})$ and $y \in f^{-1}(v_{i',J})$, where $v_{i,J}$ and $v_{i',J}$ are vertically adjacent in H_n, then $xy \in \mathcal{Z}_2$.

Let $\mathbf{M}_n \in \mathscr{C}_{\bar{Z}}$ be the expansion of H'_n in which $Z_1(\mathbf{M}_n)$ and $Z_2(\mathbf{M}_n)$ are the symmetric relations induced by $\mathcal{Z}_1$ and $\mathcal{Z}_2$, respectively; if $k > 2$, interpret the remaining symbols Z_i by the empty relation. Let

$$\varphi(x, y) := (\exists z)\,\big(\mathsf{subconn}_1(x, z) \wedge \theta(z) \wedge \mathsf{subconn}_2(z, y)\big),$$

where

$$\theta(z) := \exists w\big(E(z, w) \wedge \forall u\,(E(w, u) \to u = z)\big)$$

expresses that z has a neighbor of degree 1 in the E-reduct.

For each $i \in [n]$, select $a_i \in f^{-1}(v_{i,\emptyset})$, and for each $J \subseteq [n]$, select $b_J \in f^{-1}(v_{1,J})$. Horizontal $Z_1(\mathbf{M}_n)$-connectivity fixes the row, vertical $Z_2(\mathbf{M}_n)$-connectivity fixes the column, and $v_{i,J}$ has a pendant neighbor precisely when $i \in J$. Consequently,

$$\mathbf{M}_n \models \varphi(a_i, b_J) \quad \iff \quad i \in J.$$

Thus, φ has the independence property on $\mathscr{C}_{\bar{Z}}$.

Conversely, assume that $\mathscr{C}$ has bounded tree-width. Since every $Z_i(\mathbf{M})$ is contained in $E(\mathbf{M})$, the Gaifman graphs of the structures in $\mathscr{C}_{\bar{Z}}$ have bounded tree-width as well. Every atom $\mathsf{subconn}_i(x, y; \bar{z})$ is expressible in monadic second-order logic, and hence every $\mathsf{FO}{+}\mathsf{subconn}^k$-formula translates into an MSO-formula. The standard tree encoding of structures of bounded tree-width, together with the type bounds for MSO on trees [71], implies dependence also after adding finitely many arbitrary unary predicates. Consequently, $\mathscr{C}_{\bar{Z}}$ is monadically $\mathsf{FO}{+}\mathsf{subconn}^k$-dependent. □

We remark that if the interpretations of $Z_1, \dots, Z_k$ are required to be pairwise disjoint, the preceding proof no longer applies because every internal branch-set edge was placed in both $\mathcal{Z}_1$ and $\mathcal{Z}_2$. For a minor-closed class the argument can be repaired: unbounded tree-width implies that the class contains every planar graph H_n itself, whose horizontal and vertical grid edges can be assigned to Z_1 and Z_2, respectively, and are disjoint.

8.3. Connections to RPQs

We now compare our logic with the undirected endpoint semantics of *regular path queries* (RPQs). Let Σ be a finite label alphabet and let $\tau_\Sigma := \{E\} \cup \{E_a : a \in \Sigma\}$, where all displayed objects are binary relation symbols. Consider a τ_Σ-structure $\mathbf{G}$ in which $E(\mathbf{G})$ and all $E_a(\mathbf{G})$ are symmetric and irreflexive, and $E_a(\mathbf{G}) \subseteq E(\mathbf{G})$ for every $a \in \Sigma$; an edge may carry more than one label. Given a regular expression Q over Σ, a pair (x, y) is an answer to Q if there are a walk $x = v_0, v_1, \dots, v_m = y$ in the E-reduct of $\mathbf{G}$ and labels $a_1, \dots, a_m \in \Sigma$ such that $(v_{i-1}, v_i) \in E_{a_i}(\mathbf{G})$ for every $i \in [m]$ and $a_1 \cdots a_m \in L(Q)$. Regular path queries play a central role in graph databases and are used in query languages such as SPARQL.

RPQs and FO+subconn serve different expressive purposes. RPQs can constrain the complete label word of a witnessing walk, for example by $B^*R^*B^*$; such a constraint is not available as a single sub-connectivity atom. Conversely, FO+subconn combines its connectivity atoms with the full first-order logical overlay, which is not part of an RPQ.

With at least two labels whose edge relations may overlap, allowing arbitrary RPQ atoms subsumes reachability with respect to two possibly overlapping admissible-edge relations. Hence, Section 8.2 shows that, on monotone graph classes, unbounded tree-width implies monadic independence. Conversely, every fixed RPQ endpoint relation is MSO-definable via its finite automaton and a standard product-reachability construction. Thus, expansions of bounded-tree-width graph classes remain monadically dependent, and bounded tree-width is the resulting dependence frontier. An AW[$\star$]-hardness statement on classes of unbounded tree-width additionally requires an efficient construction of the relevant grid-minor encodings, including their minor models.

Building on the query-log analysis of Bonifati, Martens, and Timm [8], Martens and Trautner [65, Definition 2] call the following expressions *simple transitive expressions* (STEs). A nonempty set $A \subseteq \Sigma$ is an *atomic expression*; an expression of the form $A_1 \cdots A_m$ or $A_1? \cdots A_m?$ is *bounded*, where $m \geqslant 0$ and every A_i is atomic. An STE has the form

$$B_{\text{pre}}\, T^*\, B_{\text{suff}},$$

where B_{pre} and B_{suff} are bounded expressions and T is either ε or an atomic expression. For a fixed STE, the endpoint relations defined by B_{pre} and B_{suff} are first-order definable; denote the corresponding formulas by β_{pre} and β_{suff}. If T is a nonempty set of labels, put

$$\theta_T(p, q) := \bigvee_{a \in T} E_a(p, q).$$

Then the undirected endpoint semantics of the STE is expressed in Guarded−FO+subconn by

$$\Psi(x, y) := [\theta_T]\, \exists u \exists w \left(\beta_{\text{pre}}(x, u) \wedge \mathsf{subconn}(u, w) \wedge \beta_{\text{suff}}(w, y)\right).$$

If $T = \varepsilon$, the middle condition is simply $u = w$, and the query is first-order definable.

The subsequent analysis in [65] reports that 99.992% of the 247,404 property paths in the analyzed query logs are STEs or are trivially equivalent to STEs. Moreover, every remaining expression type in the reported table except $(ab)^*$ is a union of STEs. Thus, the guarded logic covers nearly all of those logged property-path shapes under the undirected endpoint semantics.

Dealing with RPQs not expressible in the guarded fragment of the logic. There is no known Guarded−FO+subconn formula expressing that a set is bipartite, or a path to be of even length, and these properties are likely not expressible, similarly to FO+conn [81, Theorem 3.12]. However, we can always use the following technique. First compute in linear time a bounded amount of unary (or 0-ary) predicates, second use Theorem 8.1 on the graph equipped with these new predicates. We now showcase this reasoning on the parity-restricted endpoint relation without vertex deletions, and show that it is still tractable. Indeed, given the admissible-edge graph, mark by a fresh unary predicate N all vertices lying in non-bipartite connected components, and choose one bipartition class in every bipartite connected component and mark one side by a fresh unary predicate B. Then the existence of an even-length admissible-edge walk is expressed by

$$\mathsf{Even}_Z(x,y) := \mathsf{subconn}(x,y) \wedge \big(N(x) \vee (B(x) \leftrightarrow B(y))\big).$$

The predicates N and B can be computed in linear time and, being unary, do not change the Gaifman graph, and the formula can be model-checked efficiently with Theorem 8.1 on minor-excluded classes.

Going further in this direction, we may also consider atoms

$$\mathsf{even\text{-}scon}(x, y; \bar{z})$$

asserting the existence of an even-length walk in the admissible-edge graph after deleting the vertices occurring in $\bar{z}$. Whether model checking for this deletion-aware extension remains tractable on minor-excluded classes is a natural question, which remains open.

Appendix A. Dependence of separator logic on monotone graph classes excluding a topological minor

Theorem 1.1 (restated)**.** *Let $\mathscr{C}$ be a monotone class of graphs. Then the following are equivalent:*

(1) $\mathscr{C}$ excludes some graph as a topological minor;
(2) $\mathscr{C}$ is monadically dependent (for FO+conn*);*
(3) $\mathscr{C}$ is dependent (for FO+conn*).*

Moreover, any of these conditions imply that FO+conn *model checking is* FPT *on $\mathscr{C}$, while if $\mathscr{C}$ is independent and admits efficient encoding[a] of graphs, then model checking on $\mathscr{C}$ is* AW[$*$]*-hard.*

[a] For logics more expressive than first-order logic, independence alone need not rule out fixed-parameter tractable model checking, even on hereditary graph classes. For example, sufficiently long subdivisions can make the encoded graph small relative to the input, leaving enough time to evaluate a sentence by brute force on the encoded instance. Such phenomena occur for monadic second-order logic [46, 61] and separator logic [73].

To obtain hardness, we therefore impose an efficiency condition on the encoding. An *efficient encoding of graphs* consists of a polynomial-time construction that maps every finite graph to a monadic expansion of a structure in the class, from which a fixed simple interpretation in the logic recovers the input graph. The interpretation then gives an AW[$\star$]-hardness reduction for model checking on these monadic expansions. Our hardness theorem uses a specific efficient minor-encoding hypothesis, stated below.

For (1) $\Rightarrow$ (2), graphs in $\mathscr{C}$ admit tree decompositions that can be encoded by rooted trees equipped with their ancestor order and auxiliary structures on sibling sets. For each fixed FO+conn-formula, Lemma 8.3 and Remark 7.1 of the full version of [73] provide such an encoding, with finitely many unary colors, on which the formula is equivalent to a plain first-order formula, preserving all assignments to its free variables. The auxiliary structures belong to a fixed bounded-expansion class and are therefore monadically dependent.

More generally, trees augmented in this way with structures from any fixed monadically dependent class are monadically dependent. The counting argument follows the same principle as the tree-automata argument of Paszke and Pilipczuk [71, Section 3], who bound the VC density of definable set systems on trees and, through tree encodings, on graphs of bounded clique-width.

We briefly sketch the argument using Theorem 7.1 of the full version of [73]. Every first-order formula on these augmented trees can be translated into an equivalent automaton, independently of any structural assumptions on the auxiliary structures. The automaton depends only on the formula, its transitions are specified by first-order formulas evaluated on the auxiliary structures colored by the states already computed at their vertices.

By the standard one-object-variable criterion for NIP [84], it suffices to consider formulas $\psi(x;\bar{y})$ with x a single variable and $\bar{y}$ an arbitrary tuple. Note that this criterion also applies to classes of finite structures. Fix such a formula, its automaton, and a nonempty finite parameter set A. The ψ-type of a vertex over A records precisely which tuples from $A^{|\bar{y}|}$ make the automaton accept when that vertex is marked as x and the tuple is marked as $\bar{y}$. Thus, counting types amounts to counting the different acceptance patterns obtained by moving the x-mark.

Consider the union of the paths from the root to A. Retaining the root and the vertices of A and suppressing all other vertices with only one child leaves $\mathcal{O}(|A|)$ vertices and connecting paths. Color the auxiliary structures by the states of the automaton run with no free-variable markers. Along each connecting path, the effect of the x-mark is described by one of finitely many transformations of automaton states. At a retained vertex, a parameter tuple changes the states of at most $|\bar{y}|$ children. The interaction with the child containing the x-mark is therefore described by finitely

many first-order formulas on the colored auxiliary structure. Monadic dependence bounds the number of resulting patterns over the children leading to A by $\mathcal{O}(|A|^d)$, where d depends only on the formula and the class of auxiliary structures. These local patterns determine the state passed towards the root and hence acceptance for every parameter tuple. Including the $\mathcal{O}(|A|)$ possibilities where x is itself a retained vertex and summing over all vertices and paths gives $\mathcal{O}(|A|^{d+1})$ possible ψ-types over $A^{|\bar{y}|}$.

The argument allows arbitrary unary colors and therefore establishes monadic dependence of the augmented trees. Applying this to the encodings above proves (1) $\Rightarrow$ (2). The implication (2) $\Rightarrow$ (3) is immediate.

For $\neg(1) \Rightarrow \neg(3)$, we recall the subdivision argument underlying Lemma 9.1 of the same paper. Let H be any graph and attach three private leaves to each vertex, obtaining H^+. By monotonicity, $\mathscr{C}$ contains a subdivision S of H^+. The original vertices of H are precisely the vertices of degree at least three in S, subdivision vertices have degree two, and the added leaves have degree one. Two distinct original vertices x and y are adjacent in H if and only if they are adjacent in S, or a component of $S - \{x, y\}$ consists entirely of vertices of degree two in S and contains a neighbor of each of x and y. This is expressible by a fixed $\mathsf{FO{+}conn}$ formula. Applying it to subdivisions of incidence graphs realizing arbitrary finite incidence patterns proves independence.

References

[1] Hans Adler and Isolde Adler. Interpreting nowhere dense graph classes as a classical notion of model theory. *European Journal of Combinatorics*, 36:322–330, 2014.

[2] Stefan Arnborg, Jens Lagergren, and Detlef Seese. Easy problems for tree-decomposable graphs. *Journal of Algorithms*, 12(2):308–340, 1991.

[3] John T. Baldwin and Saharon Shelah. Second-order quantifiers and the complexity of theories. *Notre Dame Journal of Formal Logic*, 26(3):229–303, 1985.

[4] Pablo Barceló, Leonid Libkin, Anthony W. Lin, and Peter T. Wood. Expressive languages for path queries over graph-structured data. *ACM Transactions on Database Systems*, 37(4):31:1–31:46, 2012.

[5] Mikołaj Bojańczyk. Separator logic and star-free expressions for graphs, 2021. arXiv:2107.13953.

[6] Mikołaj Bojańczyk and Michał Pilipczuk. Definability equals recognizability for graphs of bounded treewidth. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2016)*, pages 407–416, 2016.

[7] Mikołaj Bojańczyk, Michał Pilipczuk, Wojciech Przybyszewski, Marek Sokołowski, and Giannos Stamoulis. Low rank MSO. In *41st Annual Symposium on Logic in Computer Science (LICS 2026)*, volume 380 of *Leibniz International Proceedings in Informatics*, pages 22:1–22:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026.

[8] Angela Bonifati, Wim Martens, and Thomas Timm. An analytical study of large SPARQL query logs. *The VLDB Journal*, 29(2–3):655–679, 2020.

[9] Édouard Bonnet, Colin Geniet, Eun Jung Kim, Stéphan Thomassé, and Rémi Watrigant. Twin-width II: small classes. *Combinatorial Theory*, 2(2):10:1–10:42, 2022.

[10] Édouard Bonnet, Ugo Giocanti, Patrice Ossona de Mendez, Pierre Simon, Stéphan Thomassé, and Szymon Toruńczyk. Twin-width IV: ordered graphs and matrices. *Journal of the ACM*, 71(3):21:1–21:45, 2024.

[11] Édouard Bonnet and Robert Hickingbotham. Induced minors and region intersection graphs. *Innovations in Graph Theory*, 2:313–327, 2025.

[12] Édouard Bonnet, Eun Jung Kim, Amadeus Reinald, and Stéphan Thomassé. Twin-width VI: the lens of contraction sequences. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1036–1056. SIAM, 2022.

[13] Édouard Bonnet, Eun Jung Kim, Stéphan Thomassé, and Rémi Watrigant. Twin-width I: tractable FO model checking. *Journal of the ACM*, 69(1):3:1–3:46, 2022.

[14] Édouard Bonnet, Jaroslav Nešetřil, Patrice Ossona de Mendez, Sebastian Siebertz, and Stéphan Thomassé. Twin-width and permutations. *Logical Methods in Computer Science*, 20(3):4:1–4:25, 2024.

[15] Nicolas Bousquet, Felix Hommelsheim, Yusuke Kobayashi, Moritz Mühlenthaler, and Akira Suzuki. Feedback vertex set reconfiguration in planar graphs. *Theoretical Computer Science*, 979:114188, 2023.

[16] Nicolas Bousquet, Amer E. Mouawad, Stephanie Maaz, Naomi Nishimura, and Sebastian Siebertz. On algorithmic meta-theorems for solution discovery: Tractability and barriers, 2025. arXiv:2510.17344.

[17] Nicolas Bousquet, Amer E. Mouawad, Naomi Nishimura, and Sebastian Siebertz. A survey on the parameterized complexity of reconfiguration problems. *Computer Science Review*, 53:100663, 2024.

[18] Samuel Braunfeld and Michael C. Laskowski. Existential characterizations of monadic NIP. *Communications of the American Mathematical Society*, 6:244–267, 2026.

[19] Samuel Braunfeld, Jaroslav Nešetřil, Patrice Ossona de Mendez, and Sebastian Siebertz. Decomposition horizons and a characterization of stable hereditary classes of graphs. *European Journal of Combinatorics*, 129:104130, 2025.

[20] Hector Buffière, Yuquan Lin, Jaroslav Nešetřil, Patrice Ossona de Mendez, and Sebastian Siebertz. Characterizations of monadically dependent tree-ordered weakly sparse structures, 2026. arXiv:2601.16039.

[21] Hector Buffière, Yuquan Lin, Jaroslav Nešetřil, Patrice Ossona de Mendez, and Sebastian Siebertz. On merge-models, 2026. arXiv:2603.26570.

[22] Gruia Călinescu, Adrian Dumitrescu, and János Pach. Reconfigurations in graphs and grids. *SIAM Journal on Discrete Mathematics*, 22(1):124–138, 2008.

[23] Thomas Colcombet. A combinatorial theorem for trees: Applications to monadic logic and infinite structures. In *Automata, Languages and Programming – 34th International Colloquium (ICALP 2007)*, volume 4596 of *Lecture Notes in Computer Science*, pages 901–912. Springer, 2007.

[24] Alexandre Cooper, Stephanie Maaz, Amer E. Mouawad, and Naomi Nishimura. Parameterized complexity of reconfiguration of atoms. *Algorithmica*, 86(10):3284–3308, 2024.

[25] Bruno Courcelle. The monadic second-order logic of graphs. I. Recognizable sets of finite graphs. *Information and Computation*, 85(1):12–75, 1990.

[26] Bruno Courcelle. The monadic second-order logic of graphs. III. Tree-decompositions, minors and complexity issues. *RAIRO – Theoretical Informatics and Applications*, 26(3):257–286, 1992.

[27] Bruno Courcelle. The monadic second-order logic of graphs VII: Graphs as relational structures. *Theoretical Computer Science*, 101(1):3–33, 1992.

[28] Isabel F. Cruz, Alberto O. Mendelzon, and Peter T. Wood. A graphical query language supporting recursion. In *Proc. of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, pages 323–330. ACM, 1987.

[29] Guilherme de Castro Mendes Gomes, Sérgio H. Nogueira, and Vinícius Fernandes dos Santos. Some results on vertex separator reconfiguration. *RAIRO Operations Research*, 59(1):725–742, 2025.

[30] John Doner. Tree acceptors and some of their applications. *Journal of Computer and System Sciences*, 4(5):406–451, 1970.

[31] Jan Dreier, Ioannis Eleftheriadis, Nikolas Mählmann, Rose McCarty, Michał Pilipczuk, and Szymon Toruńczyk. First-order model checking on monadically stable graph classes. In *Proc. of the 65th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 21–30, 2024.

[32] Jan Dreier, Nikolas Mählmann, and Sebastian Siebertz. First-order model checking on structurally sparse graph classes. In *Proc. of the 55th Annual ACM Symposium on Theory of Computing (STOC)*, pages 567–580, 2023.

[33] Jan Dreier, Nikolas Mählmann, Sebastian Siebertz, and Szymon Toruńczyk. Indiscernibles and Flatness in Monadically Stable and Monadically NIP Classes. In *50th International Colloquium on Automata, Languages, and Programming (ICALP 2023)*, volume 261 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 125:1–125:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023.

[34] Jan Dreier, Nikolas Mählmann, and Szymon Toruńczyk. Flip-breakability: A combinatorial dichotomy for monadically dependent graph classes. In *Proc. of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1550–1560, 2024.

[35] Jan Dreier and Szymon Toruńczyk. Merge-width and first-order model checking. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 1944–1955. ACM, 2025.

[36] Vida Dujmović, Pat Morin, and David R. Wood. Layered separators in minor-closed graph classes with applications. *Journal of Combinatorial Theory, Series B*, 127:111–147, 2017.

[37] Zdeněk Dvořák, Daniel Král', and Robin Thomas. Testing first-order properties for subclasses of sparse graphs. *Journal of the ACM*, 60(5):36:1–36:24, 2013.

[38] Zdeněk Dvořák and Robin Thomas. List-coloring apex-minor-free graphs, 2014. arXiv:1401.1399.

[39] Kord Eickmeyer, Jan van den Heuvel, Ken-ichi Kawarabayashi, Stephan Kreutzer, Patrice Ossona de Mendez, Michał Pilipczuk, Daniel A. Quiroz, Roman Rabinovich, and Sebastian Siebertz. Model-checking on ordered structures. *ACM Transactions on Computational Logic*, 21(2):11:1–11:28, 2020.

[40] Solomon Feferman and Robert L. Vaught. The first order properties of products of algebraic systems. *Fundamenta Mathematicae*, 47(1):57–103, 1959.

[41] Michael R. Fellows, Mario Grobler, Nicole Megow, Amer E. Mouawad, Vijayaragunathan Ramamoorthi, Frances A. Rosamond, Daniel Schmand, and Sebastian Siebertz. On solution discovery via reconfiguration. *Journal of Computer and System Sciences*, 157:103747, 2026.

[42] Jörg Flum and Martin Grohe. *Parameterized Complexity Theory*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2006.

[43] Dominik D. Freydenberger and Nicole Schweikardt. Expressiveness and static analysis of extended conjunctive regular path queries. *Journal of Computer and System Sciences*, 79(6):892–909, 2013.

[44] Markus Frick and Martin Grohe. Deciding first-order properties of locally tree-decomposable structures. *Journal of the ACM*, 48(6):1184–1206, 2001.

[45] Jakub Gajarský, Stephan Kreutzer, Jaroslav Nešetřil, Patrice Ossona de Mendez, Michał Pilipczuk, Sebastian Siebertz, and Szymon Toruńczyk. First-order interpretations of bounded expansion classes. *ACM Transactions on Computational Logic*, 21(4):29:1–29:41, 2020.

[46] Robert Ganian, Petr Hliněný, Joachim Kneis, Daniel Meister, Jan Obdržálek, Peter Rossmanith, and Somnath Sikdar. Are there any good digraph width measures? *Journal of Combinatorial Theory, Series B*, 116:250–286, 2016.

[47] Robert Ganian, Petr Hliněný, Jaroslav Nešetřil, Jan Obdržálek, and Patrice Ossona de Mendez. Shrub-depth: Capturing height of dense graphs. *Logical Methods in Computer Science*, 15(1):7:1–7:25, 2019.

[48] Robert Ganian, Petr Hliněný, Jaroslav Nešetřil, Jan Obdržálek, and Patrice Ossona de Mendez. Shrub-depth: Capturing height of dense graphs. *Logical Methods in Computer Science*, 15(1):7:1–7:25, 2019.

[49] Colin Geniet and Stéphan Thomassé. First order logic and twin-width in tournaments and dense oriented graphs. *European Journal of Combinatorics*, 132:104247, 2026.

[50] Tatsuya Gima, Takehiro Ito, Yasuaki Kobayashi, and Yota Otachi. Algorithmic meta-theorems for combinatorial reconfiguration revisited. *Algorithmica*, 86(11):3395–3424, 2024.

[51] Petr A. Golovach, Giannos Stamoulis, and Dimitrios M. Thilikos. Model-checking for first-order logic with disjoint paths predicates in proper minor-closed graph classes. In *Proc. of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 3684–3699. SIAM, 2023.

[52] Guilherme C. M. Gomes, Clément Legrand-Duchesne, Reem Mahmoud, Amer E. Mouawad, Yoshio Okamoto, Vinícius Fernandes dos Santos, and Tom C. van der Zanden. Minimum separator reconfiguration. *Journal of Computer and System Sciences*, 146:103574, 2024.

[53] Mario Grobler, Stephanie Maaz, Nicole Megow, Amer E. Mouawad, Vijayaragunathan Ramamoorthi, Daniel Schmand, and Sebastian Siebertz. Solution discovery via reconfiguration for problems in P. In *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*, volume 297 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 76:1–76:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024.

[54] Mario Grobler, Stephanie Maaz, Amer E. Mouawad, Naomi Nishimura, Vijayaragunathan Ramamoorthi, and Sebastian Siebertz. Kernelization complexity of solution discovery problems. In *35th International Symposium on Algorithms and Computation (ISAAC 2024)*, volume 322 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 36:1–36:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024.

[55] Martin Grohe, Ken-ichi Kawarabayashi, and Bruce Reed. A simple algorithm for the graph minor decomposition – logic meets structural graph theory. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 414–431. SIAM, 2013.

[56] Martin Grohe, Stephan Kreutzer, and Sebastian Siebertz. Deciding first-order properties of nowhere dense graphs. *Journal of the ACM*, 64(3):17:1–17:32, 2017.

[57] Martin Grohe and Gy. Turán. Learnability and definability in trees and similar structures. *Theory of Computing Systems*, 37(1):193–220, 2004.

[58] Jonathan L. Gross, Jay Yellen, and Mark Anderson. *Graph theory and its applications*. Chapman and Hall/CRC, 3rd edition, 2018.

[59] Gwenaël Joret and David R. Wood. Complete graph minors and the graph minor structure theorem. *Journal of Combinatorial Theory, Series B*, 103(1):61–74, 2013.

[60] Tuukka Korhonen, Michał Pilipczuk, and Giannos Stamoulis. Minor containment and disjoint paths in almost-linear time. In *Proc. of the 2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 53–61. IEEE Computer Society, 2024.

[61] Stephan Kreutzer and Siamak Tazari. Lower bounds for the complexity of monadic second-order logic. In *2010 25th Annual IEEE Symposium on Logic in Computer Science*, pages 189–198. IEEE, 2010.

[62] Leonid Libkin. *Elements of finite model theory*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004.

[63] Leonid Libkin, Wim Martens, and Domagoj Vrgoč. Querying graphs with data. *Journal of the ACM*, 63(2):14:1–14:53, 2016.

[64] Daniel Lokshtanov, Amer E. Mouawad, Fahad Panolan, M. S. Ramanujan, and Saket Saurabh. Reconfiguration on sparse graphs. *Journal of Computer and System Sciences*, 95:122–131, 2018.

[65] Wim Martens and Tina Trautner. Bridging theory and practice with query log analysis. *ACM SIGMOD Record*, 48(1):6–13, 2019.

[66] Alberto O. Mendelzon and Peter T. Wood. Finding regular simple paths in graph databases. *SIAM Journal on Computing*, 24(6):1235–1258, 1995.

[67] Orestis C. Milolidakis. Splittability within minor-closed classes to graphs of low maximum degree. Master's thesis, National and Kapodistrian University of Athens, 2025.

[68] Amer E. Mouawad, Naomi Nishimura, Venkatesh Raman, Narges Simjour, and Akira Suzuki. On the parameterized complexity of reconfiguration problems. *Algorithmica*, 78(1):274–297, 2017.

[69] Amer E. Mouawad, Naomi Nishimura, Venkatesh Raman, and Marcin Wrochna. Reconfiguration over tree decompositions. In *Parameterized and Exact Computation – IPEC 2014*, volume 8894 of *Lecture Notes in Computer Science*, pages 246–257. Springer, 2014.

[70] Naomi Nishimura. Introduction to reconfiguration. *Algorithms*, 11(4):52, 2018.

[71] Adam Paszke and Michał Pilipczuk. VC Density of Set Systems Definable in Tree-Like Graphs. In *45th International Symposium on Mathematical Foundations of Computer Science (MFCS 2020)*, volume 170 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 78:1–78:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020.

[72] Michał Pilipczuk. Graph classes through the lens of logic, 2025. arXiv:2501.04166.

[73] Michał Pilipczuk, Nicole Schirrmacher, Sebastian Siebertz, Szymon Toruńczyk, and Alexandre Vigny. Algorithms and data structures for first-order logic with connectivity under vertex failures. In *49th International Colloquium on Automata, Languages, and Programming, ICALP 2022*, volume 229 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 102:1–102:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[74] Neil Robertson and P. D. Seymour. Graph minors. V. excluding a planar graph. *Journal of Combinatorial Theory, Series B*, 41(1):92–114, 1986.

[75] Neil Robertson and Paul D. Seymour. Graph minors. XVI. Excluding a non-planar graph. *Journal of Combinatorial Theory, Series B*, 89(1):43–76, 2003.

[76] Rin Saito, Anouk Sommer, Tatsuhiro Suga, Takahiro Suzuki, and Yuma Tamura. Solution discovery for vertex cover, independent set, dominating set, and feedback vertex set. In *SOFSEM 2026: Theory and Practice of Computer Science – 51st International Conference on Current Trends in Theory and Practice of Computer Science*, volume 16448 of *Lecture Notes in Computer Science*, pages 432–446. Springer, 2026.

[77] Sylvain Salvati and Sophie Tison. Containment of Regular Path Queries Under Path Constraints. In *27th International Conference on Database Theory (ICDT 2024)*, volume 290 of *Leibniz International Proceedings in Informatics*, pages 17:1–17:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024.

[78] Ignasi Sau, Nicole Schirrmacher, Sebastian Siebertz, Giannos Stamoulis, Dimitrios M. Thilikos, and Alexandre Vigny. Model checking for low monodimensionality fragments of CMSO on topological-minor-free graph classes. In *41st Annual Symposium on Logic in Computer Science (LICS 2026)*, volume 380 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 80:1–80:27. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026.

[79] Ignasi Sau, Giannos Stamoulis, and Dimitrios M. Thilikos. Parameterizing the quantification of CMSO: model checking on minor-closed graph classes. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 3728–3742. SIAM, 2025.

[80] Nicole Schirrmacher, Sebastian Siebertz, Giannos Stamoulis, Dimitrios M. Thilikos, and Alexandre Vigny. Model checking disjoint-paths logic on topological-minor-free graph classes. In *Proc. of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 68:1–68:12. ACM, 2024.

[81] Nicole Schirrmacher, Sebastian Siebertz, and Alexandre Vigny. First-order logic with connectivity operators. *ACM Transactions on Computational Logic*, 24(4):30:1–30:23, 2023.

[82] Detlef Seese. Linear time computable problems and first-order descriptions. *Mathematical Structures in Computer Science*, 6(6):505–526, 1996.

[83] Sebastian Siebertz. Reconfiguration on nowhere dense graph classes. *The Electronic Journal of Combinatorics*, 25(3):P3.24, 2018.

[84] Pierre Simon. A note on stability and NIP in one variable, 2021. arXiv:2103.15799.

[85] James W. Thatcher and Jesse B. Wright. Generalized finite automata theory with an application to a decision problem of second-order logic. *Mathematical Systems Theory*, 2(1):57–81, 1968.

[86] Wolfgang Thomas. Languages, automata, and logic. In *Handbook of Formal Languages*, volume 3, pages 389–455. Springer, Berlin, 1997.

[87] Andrew Thomason. The extremal function for complete minors. *Journal of Combinatorial Theory, Series B*, 81(2):318–338, 2001.

[88] Jan van den Heuvel. The complexity of change. In *Surveys in Combinatorics 2013*, volume 409 of *London Mathematical Society Lecture Note Series*, pages 127–160. Cambridge University Press, 2013.

[89] Hanno von Bergen, Larissa Fastenau, Enna Gerhard, Nicola Lorenz, Stephanie Maaz, Amer E. Mouawad, Roman Rabinovich, Nicole Schirrmacher, Daniel Schmand, Sebastian Siebertz, and Mai Trinh. Separating feasibility and movement in solution discovery: The case of path discovery. In *51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026)*, volume 386 of *Leibniz International Proceedings in Informatics*, pages 82:1–82:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026.

[90] Peter T. Wood. Query languages for graph databases. *ACM SIGMOD Record*, 41(1):50–60, 2012.